Counting Complexity of PureCircuit -1

CS907 Dissertation Project

Christos Demetriou 2018918

Supervisor: Dr. Christian Ikenmeyer

Department of Computer Science

University of Warwick

2024-2025

Disclaimer

We declare that there are several exerts/figures or tables have been copied directly or modified slightly from the interim report. We annotate explicitly any figures or tables that have been copied directly. Furthermore, we also declare that our introduction and project objectives have been inspired by the interim report. Additionally, several definitions and exerts in our literature review section have been carried over or slightly modified from the interim report. Any additional exerts that were used will be further annotated in the footnotes.

We declare the usage of AI under the current university regulations for the completion of the dissertation project assignment. Specifically we enumerate the following use case:

1. Further researching, brainstorming and explanation of difficult concepts such as
    (a) PPAD problems
    (b) Kleene logic
    (c) Counting Complexity
2. Proof reading under the current university's proof reading regulations. For example
    (a) Content validation and verification
    (b) Identification of textual inconsistencies
3. Assistance in the debugging of Rust code

All use cases enumerated fall under the university guidelines. All methodology, diagrams, proofs and code remain entirely my own work, unless stated otherwise, and I confirm this declaration to be accurate to the best of my knowledge.

Abstract

Associating a combinatorial meaning to abstract numbers has been a driving force in mathematics, with recent works by several researchers demonstrating how counting complexity can unlock deep structural insights [43, 33]. This paper builds upon their notion in order to study the combinatorial structure of the **#PPAD** -1 complexity class using the *PureCircuit*, a circuit-like problem that uses three-valued logic to find satisfying assignments. The main contribution builds upon the relation of *PureCircuit* with topological structures and its potential usages for parsimonious reductions across **PPAD** problems.

We first demonstrate how to embed the three-valued logic into the **PPAD** framework and the different parsimonious reductions we can construct across the *EndOfLine*, *SourceOrExcess*$(2, 1)$ and *PureCircuit* problems. These reductions allow us to analyse unbalanced digraphs through a topological lens, uncovering previously unexplored combinatorial classes within **#PPAD** -1. Moreover, our analysis extends to multi-source graph configurations, where we characterise their combinatorial structures and demonstrate their connections between combinatorial and topological interpretations.

We complement our theoretical work with computational tools for visualising *PureCircuit* instances. The paper demonstrates the application of several meta-heuristic algorithms to approximate solutions as well as a specialised backtracking algorithm for solution enumeration, demonstrating practical improvements over brute-force approaches.

Keywords: Counting complexity, Kleene logic, Combinatorial Interpretations, Search Problems, PPAD, Hazard-free Circuits, PureCircuit, Complexity Theory

Acknowledgements

I would like to express my deepest appreciation to my dissertation supervisor Christian Ikenmeyer for his constant support over the past 9 months. I would also like to thank my friends and family who stood by me, especially Emily Miranthi, Evelina Theodosiou, Mohammad Edelbi and my flatmates Sukritha P M and Sepideh B for their constant support and reassurance.

Abbreviations

Turing Machine	TM
Non-Deterministic Turing Machine	NTM
StrongSperner	SS

Contents

List of Figures

List of Tables

List of Algorithms

List of Symbols

$[n]$ List of numbers $1, ..., n$.

$[n]_0$ Defined as $[n] \cup \{0\}$.

$\mathbb{N}$ Set of natural numbers excluding 0.

$\mathbb{N}_0$ Natural numbers including 0.

x_{-i} Given $x \in A^d$, we use x_{-i} to denote all members excluding index i.

$x_{-i}(e) \equiv x(x_{-i}, e)$ Given $x \in A^d$, we denote $x_{-i}(e)$ or $x(x_{-i}, e)$ to indicate n dimensional vector, where

$$\forall j \in [n] : [x_{-i}(e)]_j = [x(x_{-i}, e)]_j = \begin{cases} e & \text{if } j = i \\ x_i & \text{otherwise} \end{cases}$$

Will be used to indicate element replacement.

$[x]_j$ Given $x \in A^d$ for some $j \in [d]$, we have $[x]_j = x_j$.

$\mathbb{B}$ The binary set $\{0, 1\}$.

$\mathbb{T}$ Defines the set $\{0, 1, \perp\}$ which depicts the kleene algebra.

Chapter 1

Introduction

1.1 Introduction

Combinatorics is a field of study in mathematics that primarily focuses on the notion of counting objects with certain properties. Over time, this notion has shifted, especially in the subfield of algebraic combinatorics, where there is no clear notion of the object that we are counting, and the numbers express something more abstract [43]. Researchers have questioned whether it is possible to address the inverse question, or more informally, to associate a sequence of numbers to sets of objects [33, 43, 34].

To understand this notion, we provide an example from the papers by Ikenmeyer and Pak [43, 33]: Assume that $f(G)$ counts the number of 3-colourings of a graph G. A natural question follows: what does $h(G) \doteq f(G)/6$ count? As shown in [43, 33], for every colouring χ, there are 3! additional colourings since we can permute the colours we assigned. Therefore, we only need to count one of the permutations, or the smallest colouring. We can therefore argue that $h(G)$ counts the smallest possible 3-colourings of a graph G. We denote this as a "combinatorial interpretation of $h(G)$".

However not all non-negative functions have such a clear interpretation. We can utilise an example by Ikenmeyer et al. [33], where we recall a combinatorial variant of the *Pigeonhole principle*, given a circuit $C : \{0,1\}^n \to \{0,1\}^n$, find x such that $C(x) = 0^n$ or distinct $x, y \in \{0,1\}^n$ such that $C(x) = C(y)$ and $C(C(x)) \neq 0$. C essentially maps $\{0,1\}^n \to \{0,1\}^n \setminus \{0^n\}$. Every point that is mapped to 0^n is considered a violation. The problem counts every pair of points that is mapped to the same bin or to a point that is a violation. We add the restriction of $C(C(x)) \neq 0$ to avoid double counting. We can clearly observe that, if $e(C)$ counts the number of solutions, under the pigeonhole principle, it must be the case $e(C) \geqslant 1$ since we either have a violation or there exists at least one pair of points that is mapped to the same bin by the pigeonhole principle. This raises the question: does the function $e'(C) \doteq e(C) - 1$ have a combinatorial interpretation? Ikenmeyer et al. [33] demonstrated that this problem may not have a combinatorial interpretation. This leads to the question of how can we formally represent the meaning of a combinatorial interpretation, and what can we do to verify whether a function has one?

In recent years, there has been a revolutionary initiative to answer the aforementioned questions, with the help of counting complexity theory and the complexity class of **#P**. In fact, being able to find such definitions or interpretations is crucial, as it allows us to utilise tools from enumerative combinatorics to understand and reveal hidden structures and properties of such functions [43, 33]. Moreover, there are several problems or numbers, such as *Kronecker coefficients* [39], whose combinatorial interpretation could lead to groundbreaking breakthroughs or resolutions of key conjectures such as $\mathbf{P} \neq \mathbf{NP}$ [33, 32].

In our current work, we focus on extending the work done by Ikenmeyer et al. [33], where they focused on the creation of frameworks that determine whether $f \in^? \mathbf{\#P}$, by looking at the subclasses of $\mathbf{\#TFNP} - 1$ problems, a class of problems which guarantee a solution. More specifically, we look into the **PPAD** class which seems to have several *complete* problems where under decrementation, may or may not have a combinatorial interpretation [33]. This implies that class **PPAD** acts as some sort of barrier to **#P** which raises questions as to its combinatorial nature. To accomplish this task, we studied *PureCircuit* problem which was created by Deligkas et al. [21], as we hope its **PPAD**-completeness and circuit-like nature may give us useful insights to the counting complexity of $\mathbf{\#PPAD} - 1$.

We will now give an extensive introduction to the literature and related problems and consequently offer formal and precise objectives that capture our goal. We will then go over the main results of our theory research with the conjectures and possible directions that we uncovered. Moreover, we will give an overview of the visualisation software we developed to verify our theoretical results and give an overview of its capabilities and limitations. Lastly we will give brief analysis of our project management, our personal reflections and the future directions.

1.2 Theoretical Preliminaries and Literature Overview

In the current section, we introduce all the necessary preliminaries with respect to search problems and counting complexity theory. Moreover, we give an overview of the *PureCircuit* problem, and its connections with search problems and relations with *Kleene logic*.

1.2.1 Search problems

In complexity theory, we are interested in different kinds of problems. Common examples are problems found in **P** and **NP** classes, known as decision problems, where given an input, the output is a boolean value of whether the specific instance is in the language or not. Function problems or search problems are extensions of decision problems, where we represent functions or relations as explained in definition 1.2.1.

Definition 1.2.1: Search Problems [3]

Search problems can be defined as relations $R \subseteq \mathbb{B}^* \times \mathbb{B}^*$, where given $x \in \mathbb{B}^*$, we want to find $y \in \mathbb{B}^*$ such that xRy.
Total Search problems are search problems such that for each input, there must exist at least one solution.

We mainly focus on the search variants of **NP** known as **FNP** and its subclass for *total problems* known as **TFNP** using the definitions in 1.2.2.

Definition 1.2.2: FNP and TFNP

FNP are *search problems* such that there exists poly-time *TM* $M : \mathbb{B}^* \to \mathbb{B}$ and a polynomial function $p : \mathbb{N} \to \mathbb{N}$ such that:

$$\forall x \in \mathbb{B}^*, y \in \mathbb{B}^{p(|x|)} : xRy \iff M(x, y) = 1$$

Lastly $\textbf{TFNP} = \{L \in \textbf{FNP} \mid L \text{ is total}\}$

Additionally, in the decision problem space, it is common to reduce problems to each other with the help of *Kerp reductions*, in order to yield deeper connections. We introduce its search problem variant known as *Levin Reductions* using the definition 1.2.3.

Definition 1.2.3: Levin Reductions [3]

Let A and B be two search problems with relations $R_A \subseteq \{0,1\}^* \times \{0,1\}^*$ and $R_B \subseteq \{0,1\}^* \times \{0,1\}^*$, respectively.
We say that A is *Levin reducible* to B, denoted $A \leqslant_L B$, if there exist polynomial-time computable functions $f : \{0,1\}^* \to \{0,1\}^*$ and $g : \{0,1\}^* \times \{0,1\}^* \to \{0,1\}^*$ such that:

1. f applies a *Kerp reduction* from problems of A to problems of B such that:

$$\forall x \in \{0,1\}^* : x \in S_{R_A} \implies f(x) \in S_{R_B}$$

2. Using an element $R_B(f(x))$, we can construct a solution to R_A such that:

$$\forall x \in S_{R_A}, y' \in R_B(f(x)) : (x, g(x,y)) \in R_A$$

Where we denote the sets S_R and $R(\cdot)$ as the domain and co-domain of the relations R, or more formally

$$S_R \triangleq \{x \mid \exists y : xRy\}$$
$$R(x) \triangleq \{y \mid \exists x : xRy\}$$

The **TFNP** hierarchy contains several subclasses (see figure 1.2.1) as defined by various researchers [45, 35, 16, 24]. Each of these problems take advantage of a different combinatorial property such as, the pigeonhole principle or sinks in DAGs. In the current project we are primarily interested in **PPAD**, known as "Polynomial Parity of Augmented DiGraphs", as coined by Papadimitriou [45]. We first introduce the *EndOfLine* problem 1.2.4 which takes advantage of the combinatorial fact that if there exists an unbalanced node in a directed graph, there must always exist a different one.

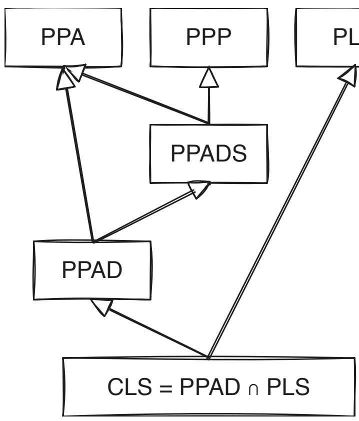


Figure 1.2.1: **TFNP** hierarchy. Figure re-created by [33].

Definition 1.2.4: *EndOfLine* problem [45]

Given poly-sized circuits $S, P \in \mathbb{B}^n \to \mathbb{B}^n$, we define a digraph $G = (V, E)$ such that $V = \mathbb{B}^n$ and E defined as:

$$E = \{(x,y) \in V^2 : S(x) = y \wedge P(y) = x\}$$

We define source nodes as $v \in V : deg(v) = (0,1)$ and sink nodes as $deg(v) = (1,0)$. We also syntactically ensure that the 0^n node is always a source, meaning $S(P(0^n)) \neq 0^n \wedge P(S(0^n)) = 0^n$. A node $v \in V$ is a solution *iff in-deg*$(v) \neq$ *out-deg*(v).

An illustrative example of an *EndOfLine* instance can be seen in figure 1.2.2. The key observation

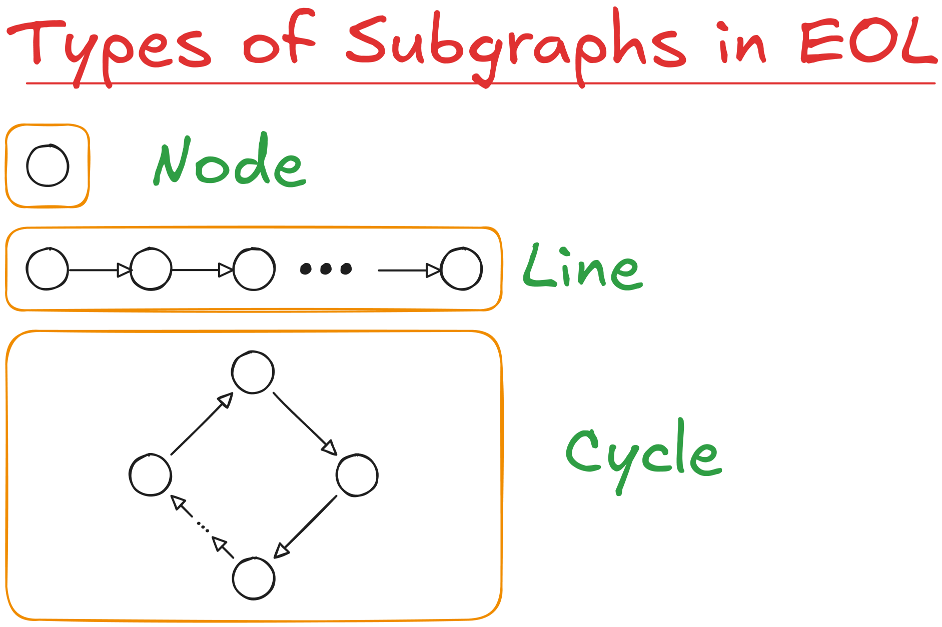


Figure 1.2.2: Types of sub-graphs in *EndOfLine*. Figure from interim report.

is that solutions will always be at the ends of a line, hence its name. We also always guarantee a solution since we ensure that the 0^n is always a source and therefore there must exist a pairing sink. From this problem, we define the **PPAD** complexity class 1.2.5.

Definition 1.2.5: *PPAD* complexity class [45]

PPAD is defined as the set of search problems that are *levin reducible* 1.2.3 to the *EndOfLine* problem 1.2.4.

EndOfLine variants and extensions

We introduce several key variants of the *EndOfLine* that have been explored in the past. We first highlight the works by Papadimitriou [44], Beame et al. [4] and Alexandros et al. [29] since they demonstrated that in the more general case of superpolynomial graphs, if we bound the in-degree and out-degree of each node to some polynomial of the input size, then the problem remains **PPAD**. We introduce the *SourceOrExcess*$(2,1)$ problem by Ikenmeyer et al. [33], since it lacks a combinatorial interpretation under decrementation ("-1"), as further demonstrated in section 1.2.4.

Definition 1.2.6: *SourceOrExcess* problem [33]

We define as *SourceOrExcess*$(k,1)$ for $k \in \mathbb{N}_{\geqslant 2}$ as such: Given a poly-sized successor circuit $S : \mathbb{B}^n$ and a set of predecessor poly-sized circuits $\{P_i\}_{i\in[k]}$, we define the graph $G = (V, E)$ such that, $V = \mathbb{B}^n$ and E as:

$$\forall x, y \in V : (x, y) \in E \iff (S(x) = y) \wedge \bigvee_{i\in[k]} P_i(y) = x$$

We ensure that 0^n is as source, meaning $\deg(0^n) = (0, 1)$. A valid solution is a vertex v such that *in-deg*$(v) \neq$ *out-deg*(v).

In addition to the aforementioned adjustments, researchers were able to give a topological interpretation to the *EndOfLine* 1.2.4 problem, by creating embeddings of the graph into 2D or 3D spaces [11, 29, 15]. We will mainly focus on the *2D-EOL* created by Chen et al. [11] and

Hollender et al. [29].

Definition 1.2.7: *2D-EOL* problem [11, 29]

Input: Two poly-sized circuits $P, S : \{0,1\}^{2n} \to \{0,1\}^{2n}$ where $P(0^n, 0^n) = (0^n, 0^n) \neq S(0^n, 0^n)$.
Output: A point $x \in \{0,1\}^{2n}$ such that one of the following conditions are satisfied:

1. Sinks: $P(S(x)) \neq x$
2. Sources: $S(P(x)) \neq x \wedge x \neq 0^n$
3. Invalid successor: $x - S(x) \notin \{(0,0),(0,1),(0,-1),(1,0),(-1,0)\}$
4. Invalid predecessor: $x - P(x) \notin \{(0,0),(0,1),(0,-1),(1,0),(-1,0)\}$

The above description defines a superpolynomial graph such that an edge $(x, y) \in E$ if and only if $S(x) = y \wedge P(y) = x$ and $\|x - y\|_1 \leqslant 1$. We refer to this problem in later sections to discuss the topological representations of our graphs.

Multi-Source variants Lastly we introduce variants of the *EndOfLine* problem where we extend the number of known sources. These multi-source problems were created by Hollender et al. [29] and were demonstrated to be **PPAD**-complete.

Definition 1.2.8: *PolyS-EoL* definition

Input: We are given triplet $(S, P, \bar{f})$, where S, P are successors, predecessor circuits of an *EndOfLine* problem with n input-output bits, and $\bar{f}$ is a poly-sized circuit that computes some polynomial function $f : \mathbb{N} \to \mathbb{N}$. We impose the following restriction:

$$\forall i \in [f(n)]_0 : S(P(i)) \neq i \wedge P(S(i)) = i$$

Output: A node $x \notin [f(n)]_0$ such that $S(P(x)) \neq x \oplus P(S(x)) \neq x$.

The PureCircuit problem

We first start with *ϵ-GCircuit* which was firstly introduced by Chen et al. [21, 12]. *ϵ-GCircuit* is defined as a directed graph $T = (V, G)$ where V denote value nodes and G denote gate nodes. Each value node is assigned a value in $[0,1]$ and the gates can denote the standard set of arithmetic operations $\{+, -, *\}$ with other nodes or some constant, or boolean operators $\{\wedge, \vee, \neg, <, >, =\}$ with other nodes or some constant. The goal of the problem is to assign every value node with a value such that the assignment is correct up to a factor of $\pm\epsilon$. We refer figure 1.2.3 as example. This problem was crucial to demonstrate the hardness of approximating of **PPAD** problems since Rubinstein demonstrated that there exists $\epsilon > 0$ for which *ϵ-GCircuit* is **PPAD**-hard [21, 46].

PureCircuit is the problem that we get when taking the limit $\epsilon \to 1$ of the *ϵ-GCircuit* problem. This results in a circuit which uses the set of values $\{0, \perp, 1\}$ to denote the range of values $(0), (0,1), (1)$. Incidentally, the above triplet of values coincides with *Kleene algebra* which extends the traditional binary logic [37]. We will explore the above algebra in greater depth in section 1.2.3. This problem was created by Deligkas et al. [21] to demonstrate that for all $\epsilon < 1/10$, *ϵ-GCircuit* is **PPAD**-hard and it was shown to be **PPAD**-complete.

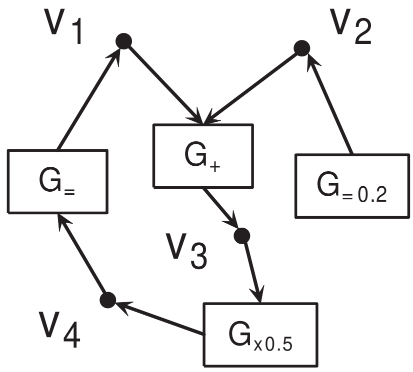


Figure 1.2.3: ϵ-*GCircuit* example. A valid assignment would be $(0.2, 0.2, 0.2, 0.2)$ since each gate will be satisfied up to an ϵ factor. Image provided by Chen et al. [12].

Definition 1.2.9: PureCiruit Problem Definition [21]

An instance of *PureCircuit* is given by vertex set $V = [n]$ and gate set G such that $\forall g \in G : g = (T, u, v, w)$ where $u, v, w \in V$ and $T \in \{\text{NOR}, \text{Purify}\}$. Each gate is interpreted as:

1. NOR: Takes as input u, v and outputs w
2. Purify: Takes as input u and outputs v, w

And each vertex is ensured to have in-deg$(v) \leqslant 1$. A solution to input instance (V, G) is denoted as an assignment $\mathbf{x} : V \to \{0, \perp, 1\}$ such that for all gates $g = (T, u, v, w)$ we have:

1. NOR:

$$\begin{aligned} \mathbf{x}[u] = \mathbf{x}[v] = 0 &\implies \mathbf{x}[w] = 1 \\ (\mathbf{x}[u] = 1 \vee \mathbf{x}[v] = 1) &\implies \mathbf{x}[w] = 0 \\ \text{otherwise} &\implies \perp \end{aligned}$$

2. Purify:

$$\begin{aligned} \forall b \in \mathbb{B} : \mathbf{x}[u] = b &\implies \mathbf{x}[v] = b \wedge \mathbf{x}[w] = b \\ \mathbf{x}[u] = \perp &\implies (\mathbf{x}[v], \mathbf{x}[w]) \in \{(0, \perp), (\perp, 1), ((0, 1))\} \end{aligned}$$

In addition to the gates above, we will make use of the standard set of gates $\{\vee, \wedge, \neg\}$, whose behaviour is depicted in the tables 1.2.1. Moreover, we expand our gate set with the Copy gate which we can define as $\text{Copy}(x) = \neg(\neg x)$. These gates are well-defined based on the NOR gate as shown by Deligkas et al. [21] and do not directly affect the complexity of the problem. Furthermore, with respect to the Purify gate, the original problem states that one of two outputs must contain a pure value. We restrict the solution set to the one written in the definition as Deligkas et al. [21] proved that the only Purify solutions essential for **PPAD**-completeness are $\{(0, \perp), (\perp, 1), (0, 1)\}$, and the addition of more solutions does not affect its complexity.

not	
0	1
$\perp$	$\perp$
1	0

(a) not gate

and	0	$\perp$	1
0	0	0	0
$\perp$	0	$\perp$	$\perp$
1	0	$\perp$	1

(b) and gate

or	0	$\perp$	1
0	0	$\perp$	1
$\perp$	$\perp$	$\perp$	1
1	1	1	1

(c) or gate

Table 1.2.1: Three-valued logic [37].

Topological problems

This section refers to types of problems that take advantage of some combinatorial property of their topology in order to guarantee a solution.

Sperner Problems Below we will refer to the notion of Sperner problems, which are types of problems that are based around *Sperner's Lemma* [53]. We are given a triangle that has subdivided into smaller sub-triangles, and we define the index of each point using the following set:

$$\forall n \in \mathbb{N} : \mathcal{T}_n \triangleq \{(x, y, z) \in [n] \mid x + y + z = n\}$$

We say $\lambda : \mathcal{T}_n \to [3]$ is a valid colouring if and only if it satisfies the following conditions:

$$\forall (x_1, x_2, x_3) \in \mathcal{T}_n : \lambda(x) \in \begin{cases} \{2,3\} & \text{if } x_1 = 0 \\ \{1,3\} & \text{if } x_2 = 0 \\ \{1,2\} & \text{if } x_3 = 0 \end{cases}$$

Given a valid colouring λ, we can always find a small triangle τ where all 3 colours are assigned to one of its vertices. A visual example is provided in figure 1.2.4, where we can observe that any colouring that we choose will always result in some panchromatic simplex.

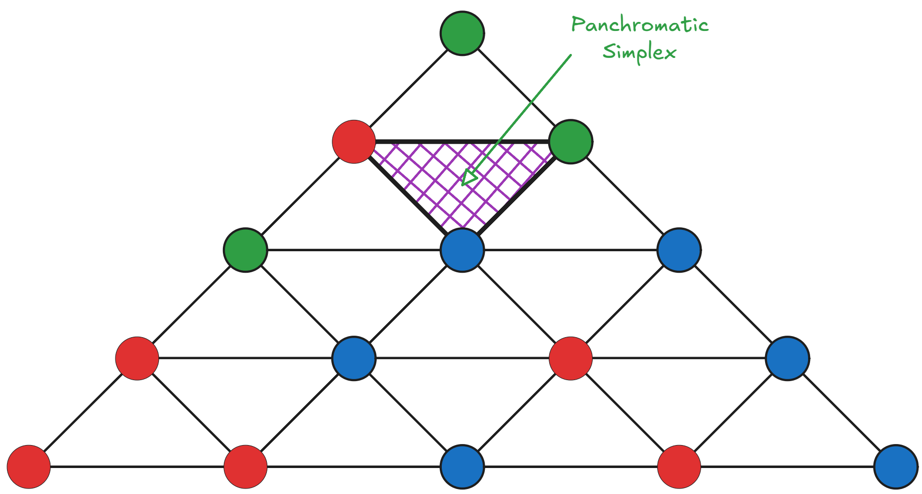


Figure 1.2.4: A colouring example of a subdivided simplex.

This type colouring will be referred to as the **linear** colouring. There are several variants of the problem, where we can either work in a triangle with an exponential amount of subdivisions or with polynomial subdivisions, but on higher dimensional simplexes. Both of these problems are **PPAD**-complete [45, 12, 11].

We introduce an alternative variant of the Sperner problems, introduced by Daskalakis et al. [18], known as *StrongSperner* or *BiSperner*. In the traditional problem, we assign 1 colour to each vertex in order to find a panchromatic simplex with $d + 1$ colours. The *StrongSperner* problem uses *bipolar colouring* in a hypercube space 1.2.10. We refer to the figure 1.2.5 for a visual interpretation of the colouring. For each point in some hypercube $[N]^d$, we assign d colours, where for colour $i \in [d]$ we can choose either $\{i^+, i^-\}$. The aim is to find a cube where in every dimension there exists at least two points that cover both colours. This has been used in many problems throughout literature as seen in the papers [12, 21, 18] to demonstrate **PPAD**-hardness of many problems.

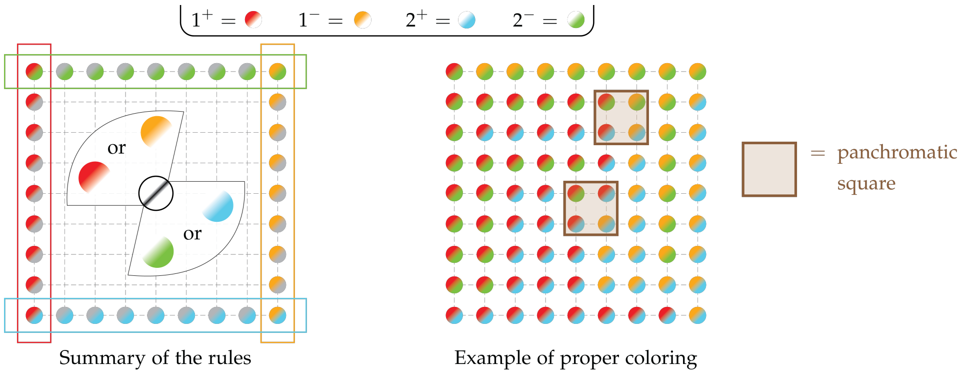


Figure 1.2.5: Visual interpretation of the *StrongSperner* problem. Figure is derived from paper [17]

Definition 1.2.10: Bipolar colouring

Given a set S^d, where S is some arbitrary set, we refer to bipolar colouring $C : S^d \to \{-1, +1\}^n$ as:

$$\forall v \in S^d, j \in [d] : [C(v)]_j \in \{-1, +1\}$$

We say that a set of points $A \subseteq S^d$ **cover all the labels** if:

$$\forall i \in [d], \ell \in \{-1, +1\}, \exists x \in A : [\lambda(x)]_i = \ell$$

Compared to the linear colouring variant, Deligkas et al. [21, 20] showed that $\forall A \subseteq S^d : |A| \geqslant d + 1$ and satisfy the bipolar colouring condition, $\exists D \subseteq A : |D| \leqslant d$ where D also satisfies the colouring condition, which implies that we need at most d separate points to achieve a colouring.

Definition 1.2.11: *StrongSperner* problem [21]

Input: A boolean circuit that computes a bipolar labelling $\lambda : [M]^N \to \{-1, 1\}^N$ 1.2.10, such that $M \in n^{O(1)}$, which satisfies the following boundary conditions for every $i \in [N]$:

- if $x_i = 1 \implies [\lambda(x)]_i = +1$
- if $x_i = M \implies [\lambda(x)]_i = -1$

Output: A set of points $\{x^{(i)}\}_{i \in [N]} \subseteq [M]^N$, such that:

- *Closeness condition*: $\forall i, j \in [N] : \|x^{(i)} - x^{(j)}\|_\infty \leqslant 1$
- *Covers all labels* as defined in 1.2.10

Relevant literature defines the same variants of the problem or specifications have been defined using the names Sperner or discrete Brouwer [12, 11, 15, 21]. This is very similar to a different type of problem known as *StrongTucker*, which can be thought of as the **PPA** analogue of *StrongSperner* [45, 2]. Chen et al. [12] demonstrated that for a fixed dimensionality with an exponentially large width the problem remains **PPAD**-complete, as defined in 1.2.12.

Definition 1.2.12: *nD-StrongSperner* problem [12, 11]

Input: A tuple $(\lambda, 0^k)$ such that $\lambda : (\mathbb{B}^k)^n \to \{-1, +1\}^n$, and λ abides the *boundary conditions* of the *StrongSperner* problem.
Output: A point $\alpha = (a_1, \ldots, a_n) \in (\mathbb{B}^k \backslash \{1^k\})^n$ such that

$$\{\alpha + x \mid x \in \mathbb{B}^n\} \text{ covers all the labels } 1.2.10$$

We assume dimensionality $n \geqslant 2$.

Both colouring schemes are interchangeably when discussing their reductions, so we can assume that all reductions (not necessarily counting reductions) work similarly for either colouring [12, 21, 15, 11].

Complementary to topological colouring, we will also introduce a tiling problem known as mutilated chessboards. We refer to the problem introduced by Blank [5] as follows: assume we have a $2n \times 2n$ chessboard. If we are provided with 1×2 pieces, we can trivially tile the board by filling each row with n horizontal pieces. The idea is when we remove the bottom left corner, the board becomes impossible to tile. The argument is provided as follows: each tile piece covers an even square and an odd square, which implies that it has to be the case that one of the squares has to remain untiled. Hollender et al. [29] introduced a **PPAD**-complete variant of the problem, known as *1-MC*. An example can be seen in the figure 1.2.6 where we can observe that there will always be an untiled square.

Definition 1.2.13: *1-MC* [29]

Input: Given a circuit $C : \{0,1\}^{2n} \to \{0,1\}^{2n}$, such that $C(0,0) = (0,0)$, such that tiling between two squares is indicated such that:

$$\forall x \in \{0,1\}^{2n} : C(C(x)) = x \wedge x - C(x) \in \{(0,1),(1,0),(0,-1),(-1,0)\}$$

Output: We want to find a point $x \in \{0,1\}^{2n} \backslash \{(0,0)\}$ such that we have no tiling.

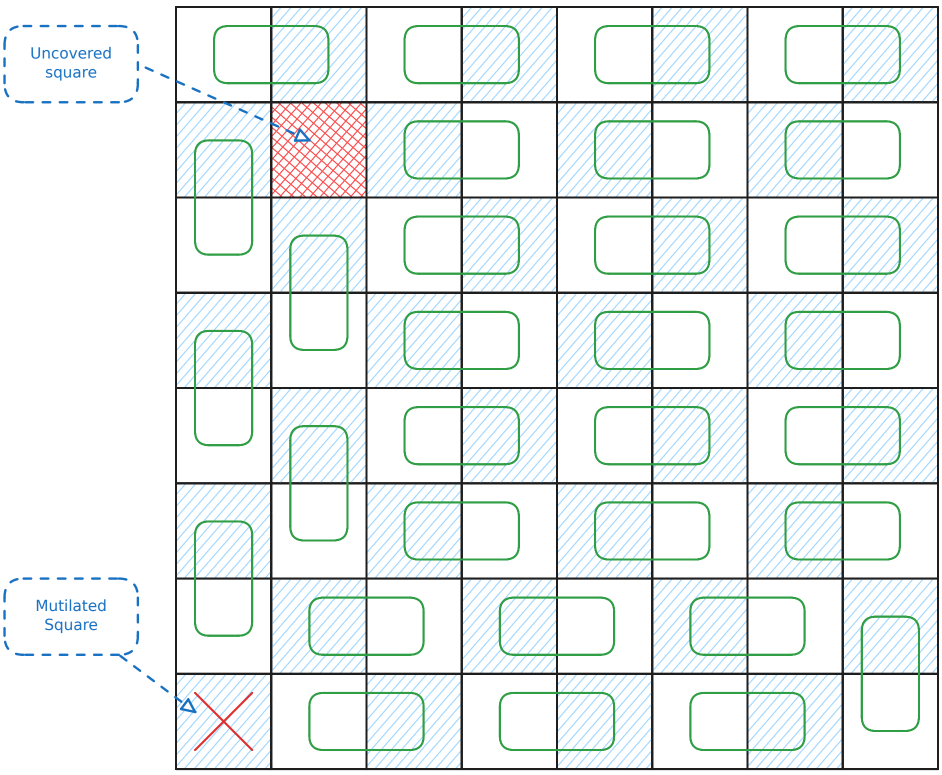


Figure 1.2.6: Example on a 8×8 chessboard with a missing square. As we can observe, there will always be one square we cannot tile.

Lastly, we will introduce *Tarski* 1.2.15 which uses the Knaster-Tarski fixed point theorem 1.2.1 and belongs in **CLS**, the class that covers the intersection of parity graphs (**PPAD**) and local search (**PLS**) [45, 35, 23].

Definition 1.2.14: Monotone functions

Given two partially ordered sets $(L_1, \preceq_{L_1})$ and $(L_2, \preceq_{L_2})$, a function $f : L_1 \rightarrow L_2$ is **monotone** if and only if:

$$\forall x, y \in L_1 : x \preceq_{L_1} y \implies f(x) \preceq_{L_2} f(y)$$

Theorem 1.2.1: Knaster Tarski Fixed point theorem[7, 25]

Given a *monotone* function $f : L \rightarrow L$ 1.2.14 of a complete lattice $(L, \wedge, \vee)$, $\exists c \in L : f(c) = c$.

Definition 1.2.15: *Tarski* problem definition [25]

Given a *monotone* function $f : L \rightarrow L$ 1.2.14 of a lattice $(L, \wedge, \vee)$ we define solutions to the problem as:

1. Find $x \in L : f(x) = x$
2. Find $x, y \in L$ such that $x \preceq y$ and $f(x) \npreceq f(y)$

We assume that f is represented by boolean circuits.

The above argues the existence of either finding points that are maximum in the partial order chain, or points that violate the monotonicity constraint.

1.2.2 Counting Complexity and Combinatorial Interpretations

Search problems as previously observed, explore what is a valid solution given the current input instance like such as a valid colouring of a graph or a satisfying assignment to a 3-SAT problem. Counting complexity, or counting problems extend this notion by asking how many valid solutions does a problem have. Valiant formalised this idea with the help of **#P** complexity class 1.2.16, in which he demonstrated that even if a problem is polynomially solvable in its decision/search variant, counting can be much harder [51]. A much simpler example can be found in the book by Barak et al. [3] where they provide a reduction from the **NP**-complete problem of hamiltonian cycles to the cycle counting problem.

Definition 1.2.16: #P Complexity Class [51]

A function $f : \mathbb{B}^* \rightarrow \mathbb{N} \in$ **#P**, if there exists a poly-function $p : \mathbb{N} \rightarrow \mathbb{N}$ and a poly-time TM M such that:

$$f(w) = \left|\left\{v \in \mathbb{B}^{p(|w|)} \mid M(w, v) = 1\right\}\right|$$

Alternative but equivalent definition is as follows: A function $f \in$ **#P**, if there exists a non-deterministic poly-time TM N such that:

$$f(w) = \mathbf{\#acc}_N(w)$$

Where $\mathbf{\#acc}_N(w)$ denotes the number of accepting paths of N on w.

In hindsight, we can observe that functions in **#P** express functions of the form $\forall w \in \{0, 1\}^* : f(w) = |A_w|$, where A_w is a set of verifiers or accepting paths of size polynomial to the input. Given the above definition, Pak et al. [43, 34, 33] used **#P** to decide whether a class of numbers or objects have or do not have a combinatorial interpretation. Moreover, they argue that the usage of **#P** has several benefits:

1. By polynomially bounding objects, we avoid cases such as: $f(w) = |\{1, \ldots, f(w)\}|$. In other words, we create a set of combinatorial objects.

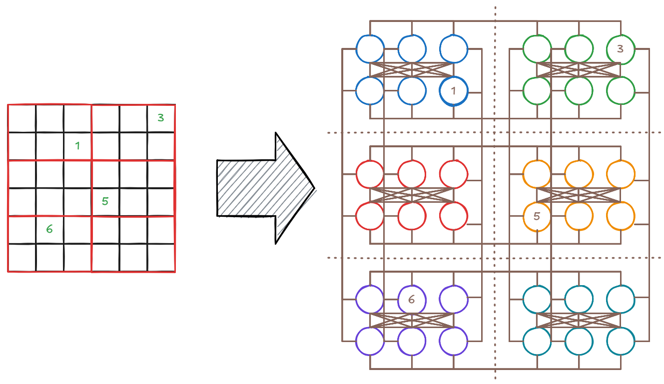


Figure 1.2.7: Example of reducing a 6x6 sudoku puzzle into a graph colouring problem. Each colouring corresponds to a sudoku solution of the original instance.

2. We can work with $f(\cdot)$ even if its direct computation is hard. For example, counting the number of cycles in a graph is believed to be an **NP**-complete problem but verifying individual cycles is trivial.

3. **#P** is a formal system that is highly flexible. Specifically, we are able to use results in complexity theory and NTMs to find interpretations or prove their non-existence.

We introduce the $\#L$ notation to denote the counting version of a search problem L, formally denoted in definition 1.2.17. Sometimes we use notations $\#L$ or $\#K(L)$ interchangeably to highlight that L is in complexity class K.

Definition 1.2.17: $\#L$ Notation

Given a search problem L with relation R, we can define its counting version as the function $\#L$ where:

$$\forall w \in \{0,1\}^* : \#L(w) = |R(w)|$$

Where for relation R we say $R(x) \triangleq \{y \in \{0,1\}^* \mid xRy\}$.

Given the above, we introduce the notion of correlating two counting problems with the help of *parsimonious reductions* which we define in 1.2.18. An example of a parsimonious reduction can be seen in figure 1.2.7, where we relate the difficulty of sudoku solving to a graph colouring problem.

Definition 1.2.18: Parsimonious reductions [3]

Let L, L' be counting problems. We say L is parsimoniously reducible to L', if there exists a polynomial computable function $f : \{0,1\}^* \to \{0,1\}^*$ such that

$$\forall x \in \{0,1\}^* : L(x) = L'(f(x))$$

We denote the reduction above as $L \subseteq L'$.

Since counting problems are regarded as functions to natural numbers, that means that we can create new counting problems through composition. For example, given $h : \{0,1\}^* \to \mathbb{N}$, and $g : \mathbb{N} \to \mathbb{N}$, we would like to express $g \circ h$. We use the definition 1.2.19 to represent the aforementioned notion.

Definition 1.2.19: Function composition in counting problems

Given counting problem L and a function $f : \mathbb{N} \to \mathbb{N}$, we can define a new counting problem $f(L)$ such that:

$$\forall w \in \{0,1\}^* \to \mathbb{N} : f(L)(w) = f(L(w))$$

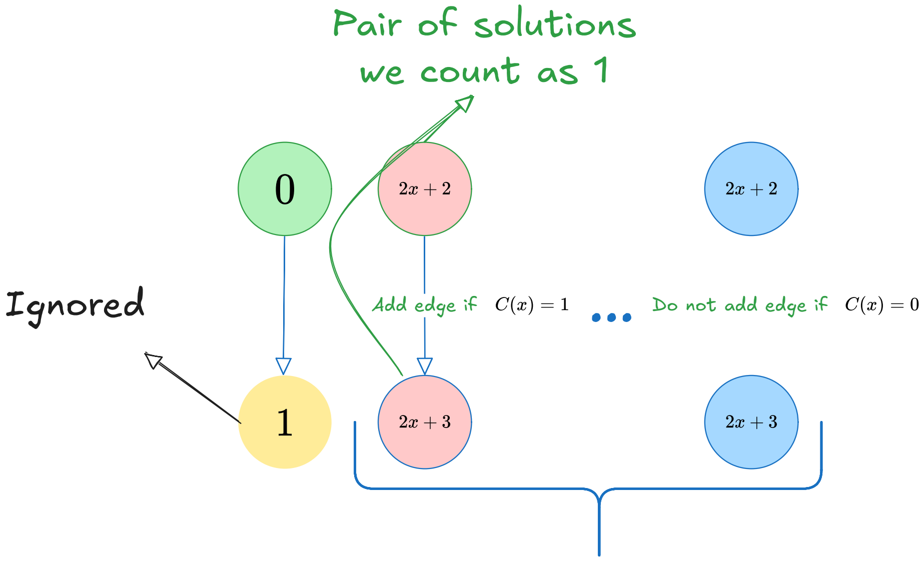


Figure 1.2.8: *#CircuitSAT* to $((\#EndOfLine) - 1)/2$.

We provide an example of the above notation using a finding from the paper by Ikenmeyer et al. [33] where they demonstrate *#CircuitSAT* $\subseteq ((\#EndOfLine) - 1)/2$ where *CircuitSAT* is the circuit variant of *SAT*. Given instance $C : \{0,1\}^n \to \{0,1\}$, using figure 1.2.8, we create pairs of nodes $(2x + 2, 2x + 3)$ for all $x \in \{0,1\}^n$. If we have a satisfying assignment, we create a source/sink pair. Excluding the sink that is always paired with the 0^n sink, we observe that the reduction is parsimonious.

1.2.3 Kleene Logic and Hazard-Free Circuits

Kleene logic uses the boolean system with an additional *unstable* value [37], which we denote as $\perp$. The study of Kleene logic assisted in the development of robust physical systems [26] and aiding in the development of lower bounds for monotone circuits [22, 30, 31, 8]. The current section offers a brief summary to relevant notions and concepts in Kleene logic.

Definition 1.2.20: Kleene value ordering [42, 30]

The *instability ordering* or *information ordering* for $\preceq$ is defined as: $\perp \preceq 0, 1$. The n-dimensional extension of the ordering, we say that:

$$\forall x, y \in \mathbb{T}^n : x \preceq y \implies \forall j \in [n] : (x_j \in \mathbb{B} \implies x_i = y_i)$$

We can also refer to an alternative type of ordering, known as *voltage* ordering, such that:

$$0 \leqslant^v \perp \leqslant^v 1$$

Definition 1.2.21: Kleene Resolutions [42, 30]

Given $x \in \mathbb{T}^n$, we define the *resolutions* of x, using the following notation:

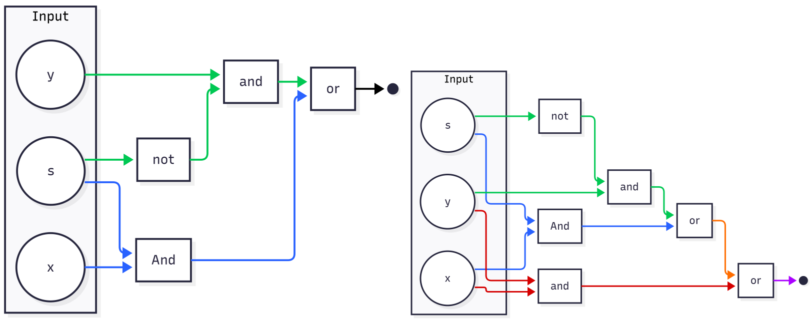
$$\textsc{Res}(x) \triangleq \{y \in \mathbb{B}^n \mid x \preceq y\}$$

Definition 1.2.22: Natural functions [30]

A function $F : \mathbb{T}^n \to \mathbb{T}^m$ for some $n, m \in \mathbb{N}$ is *natural* if and only if it satisfies the following properties:

1. *Preserves stable values*: $\forall x \in \mathbb{B}^n : F(x) \in \mathbb{B}^m$
2. *Preserves monotonicity*: $\forall x, y \in \mathbb{T}^n : x \preceq y \implies F(x) \preceq F(y)$

Unless otherwise specified, we will assume that all Kleene functions are *natural* 1.2.22, since they can be represented by circuits 1.2.1. Natural functions essentially indicate that a point with its resolution should respect the information order, or more simply if $f(00) = 1$ then $f(0\perp) \neq 0$ as it will not preserve monotonicity. Moreover, we can think resolutions of a point x as all the values it "could be". For example $0\perp$ could be either 00 or 01. Naturally, if $f(00) = f(01) = 1$ then we should also expect that $f(0\perp) = 1$. This yields the concept of hazards on circuits (formal definition 1.2.23) where given $x \in \mathbb{T}^n$, its resolutions have the same value but the point itself does not. Detecting hazards is a concept that is analysed heavily when discussing about Kleene logic, and it is an **NP**-hard problem [30, 8, 22]. An example of hazard values can be seen in figure 1.2.9.



(a) Multiplexer with hazard at $(s = \perp, x = 1, y = 1)$

(b) Hazard free multiplexer

Figure 1.2.9: Hazard circuit and hazard-free circuit. Main idea is for $MUX(s, x, y)$, $MUX(\perp, 1, 1)$ should be 1 by our hazard definition. We can see that when we replace $s = \perp, x = 1, y = 1$ to our circuits, one of them presents a hazard. Figure recreated from [30].

Proposition 1.2.1: Natural functions and Circuits [42, 30]

A function $F : \mathbb{T}^n \to \mathbb{T}^m$ can be computed by a circuit *iff* F is *natural* 1.2.22

Definition 1.2.23: Hazard [30, 22]

A *circuit* C, on n inputs has **hazard** at $x \in \mathbb{T}^n \iff C(x) = \perp$ and $\exists b \in \mathbb{B}, \forall r \in \textsc{Res}(x) : C(r) = b$. If such value does not exist then we say that C is hazard-free.

Definition 1.2.24: K-bit Hazard [30]

For $k \in \mathbb{N}$ a circuit $C : \mathbb{T}^n \to \mathbb{T}$ has a *k-bit hazard* at $x \in \mathbb{T}^n \iff C$ has a hazard at x and $\perp$ appears at most k times.

We also wish to introduce the following corollary by Ikenmeyer et al. [30] which allows us to construct k-bit hazard free circuits.

Corollary 1.1: K-bit Hazard Construction [30]

Given circuit $C : \mathbb{T}^n \to \mathbb{T}$, one can construct a k-bit hazard free circuit of C denoted as C' such that:

$$\begin{aligned} size(C') &= \left(\frac{ne}{k}\right)^k (size(C) + 6) + O(n^{2.71k}) \\ depth(C') &= depth(C) + 8k + O(k \log n) \end{aligned}$$

Where $size(P)$ denotes the number of gates of P and $depth(P)$ denotes the longest path from the input bit to the output bit

Kleene logic and Complexity Theory

Lastly we want to introduce the notion of *Alternating Turing Machines* (ATMs), which are meant to represent a generalisation of non-deterministic Turing machines. These have been introduced by Chandra et al. [10] and using universal and existential quantifiers we can alternate the course of computation. The informal presentation can be considered as follows: In a non-deterministic TM, given its current state q or configuration $\mathcal{C}$, we spawn $\beta_1, ..., \beta_k$ separate processes. We say that the current state will accept if at least one of the processes $\{\beta_i\}_{i\in[k]}$ accepts. Alternating Turing machines introduce additional conditions such as all subprocess must accept. We provide a formal definition in 1.2.25.

Definition 1.2.25: Alternating Turing Machines(ATM) [38, 10]

An alternating TM is a seven tuple

$$M = (k, Q, \Sigma, \Gamma, \delta, q_0, g)$$

Where each term denotes:

1. $k \in \mathbb{N}$: The number of work tapes
2. Q: Finite set of states
3. Σ: Finite input alphabet. Moreover, $\star \notin \Sigma$ denotes the end of the input
4. Γ: Finite work tape alphabet. Moreover, $\# \in \Gamma$ to denote blank symbol
5. δ transition relation which is defined as:

   $$\delta \subseteq \left(Q \times \Gamma^k \times (\Sigma \cup \{\star\})\right) \times \left(Q \times (\Gamma \setminus \{\#\})^k \times \{L, R\}^{k+1}\right)$$

   Important to observe that for a single state, input symbol and work tape symbols, we can take multiple types of transitions
6. $q_0 \in Q$: Is the initial state
7. $g : Q \to \{\vee, \wedge, \neg, \mathbf{1}, \mathbf{0}\}$: Where g denotes the type of state and $\mathbf{1}, \mathbf{0}$ denote whether it is accepting or not.

The machine essentially contains a read-only input tape with end-markers and k work tapes which are initially blank. A *step* of M consists of reading a symbol from the input tape, writing a symbol to each work tape, and then moving each head left or right while transitioning into a new state.

Definition 1.2.26: ATM configurations [38, 10]

A configuration $\mathcal{C}$ of an ATM M is a element of the set $\mathcal{C}_M$

$$\mathcal{C}_M \triangleq Q \times \Sigma^* \times (\Gamma \setminus \{\#\})^k \times \mathbb{N}^{k+1}$$

Which represent, the current state, the input, the current contents of each work tape, and the $k+1$ head positions.

We say that $\mathcal{C}$ that *configuration* is universal (existential, negating, accepting or rejecting) if q is also universal (existential, negating, accepting or rejecting). The initial configuration of M on input x is

$$\sigma_M(x) = (q_0, x, \lambda^k, 0^{k+1})$$

Where λ is a null string.

Definition 1.2.27: Successor of configurations [10]

Given ATM M, we write $\mathcal{C} \vdash \mathcal{C}'$ to say that $\mathcal{C}'$ is the successor of $\mathcal{C}$, if we can go from one configuration to another with a single δ rule. We require for δ that, for accepting or rejecting configurations to have no successors, for universal or existential to have at least one successor configuration, and for negating configurations to have only one. We can define the **computation path** on input x as a sequence $a_0 \vdash a_1 \vdash ... \vdash a_n$ where $a_0 = \sigma_M(x)$

Denoting Accepting or Rejecting computations As previously mentioned, we begin from the original configuration. If a process is in an existential configuration $\mathcal{C}$ and $\beta_1, ..., \beta_m$ are all possible successors, then we spawn a process for each configuration and run each one concurrently. If at least one process is accepting we report back to the parent and terminate the rest. Same line of reasoning can be applied with the universal quantifiers. The goal is to define a labelling function f such that:

$$f(\mathcal{C}) = \begin{cases} \bigwedge_{\mathcal{C} \vdash \beta} f(\beta) & \text{if } \mathcal{C} \text{ is universal} \\ \bigvee_{\mathcal{C} \vdash \beta} f(\beta) & \text{if } \mathcal{C} \text{ is existential} \\ \neg f(\beta) & \text{where } \mathcal{C} \vdash \beta \text{ if } \mathcal{C} \text{ is negating} \\ \mathbf{1} & \text{if } \mathcal{C} \text{ is accepting} \\ \mathbf{0} & \text{if } \mathcal{C} \text{ is rejecting} \end{cases}$$

However some processes may never halt or require a much greater computation than necessary. We therefore use Kleene logic $\mathbb{T}$ where $\perp$ denotes configurations that we do not know if they are accepting or not. The quantifiers use the same logic as described in the tables 1.2.1. We present the formal labelling function $\ell : \mathcal{C}_M \to \{0, \perp, 1\}$ and define an operator τ to be an operator of mappings such that:

$$\tau(\ell)(\mathcal{C}) = \begin{cases} \bigwedge_{\mathcal{C} \vdash \beta} \ell(\beta) & \text{if } \mathcal{C} \text{ is universal} \\ \bigvee_{\mathcal{C} \vdash \beta} \ell(\beta) & \text{if } \mathcal{C} \text{ is existential} \\ \ell(\beta) & \text{where } \mathcal{C} \vdash \beta \text{ if } \mathcal{C} \text{ is negating} \\ \mathbf{1} & \text{if } \mathcal{C} \text{ is accepting} \\ \mathbf{0} & \text{if } \mathcal{C} \text{ is rejecting} \end{cases}$$

We can check that τ is monotone with respect to $\preceq$ since if $l \preceq l'$, then $\tau(l) \preceq \tau(l')$. By the *Tarski theorem* 1.2.1 there must exist maximum labelling such that

$$\ell^* = \sup_{m \in \mathbb{N}} \tau^m(\Omega)$$

where we define τ^m as:

$$\tau^0(\ell) = \ell$$
$$\tau^{m+1}(\ell) = \tau(\tau^m(\ell))$$

where Ω is denoted as the minimal labelling function $\lambda x.\bot$. The above procedure describes the "propagation" process from the leaf nodes up to the root of the tree where the leaf nodes are assigned a label of $\bot$ or accepting. Given the above, we can formally define a halting state:

Definition 1.2.28: Halting states[10]

We say that M halts on x iff $\ell^*(\sigma_M(x)) \in \{0,1\}$ and it accepts or rejects respectively iff $\ell^*(\sigma_M(x)) = 1$ or $\ell^*(\sigma_M(x)) = 0$

We aim for this ATM to work for only recursively enumerable sets and therefore we aim to limit to the depth of the recursion. Therefore, we define operator $\tau_{\hat{C}}$ where $\hat{C} \subseteq \mathcal{C}_M$ such that

$$\tau_{\hat{C}}(\ell)(a) \triangleq \begin{cases} \tau(\ell)(a) & \text{if } a \in \hat{C} \\ \bot & \text{otherwise} \end{cases}$$

Where we can say that $\hat{C}$ is the set of configuration that are reachable within some $f(|x|)$ steps or require space of at most $f(|x|)$. An example of a computation tree can be observed in figure 1.2.10, where we can observe the computation paths and the labelling propagation from the leaf nodes up to the root.

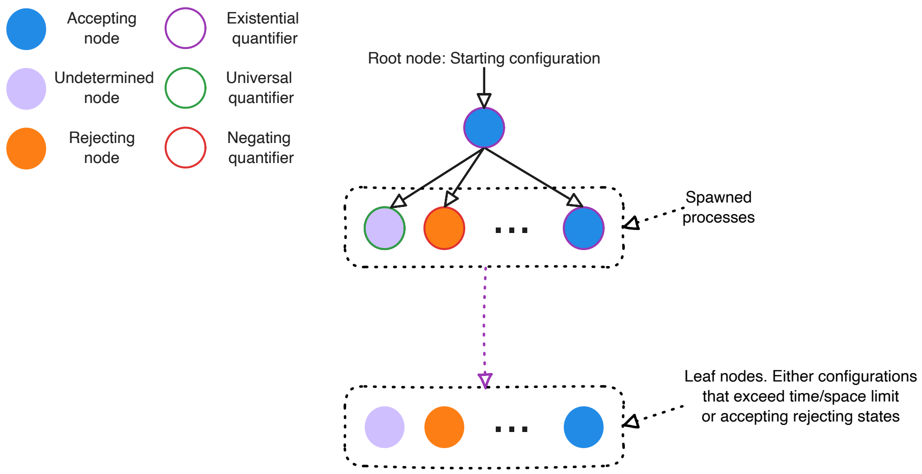


Figure 1.2.10: Computation tree of an ATM with labellings.

1.2.4 Bridging Counting Complexity and Search problems

Ikenmeyer et al. [33], developed tools to demonstrate how one can show whether a problem is in **#P** or not, with the help of relativising techniques. Relativisation is the technique of equipping circuits or TMs with calls to arbitrary oracles languages [4]. They demonstrated that for several problems in **TFNP**, some can be parsimoniously reduced to each other under all oracles, whereas for some directions this is not the case (also known as *oracle separations*). From the figure 1.2.11, we can observe that for several **PPAD**, **CLS** and **PPADS** problems one can indeed find parsimonious reductions. For other problems, even **PPAD**-complete such as **#PPAD**($SourceOrExcess(k,1)$) $- 1$, this does not seem to be the case.

Relativisation techniques are beyond the scope of the current project. All reductions that we prove in the upcoming sections are implicitly relativising. For the purposes of this project, we

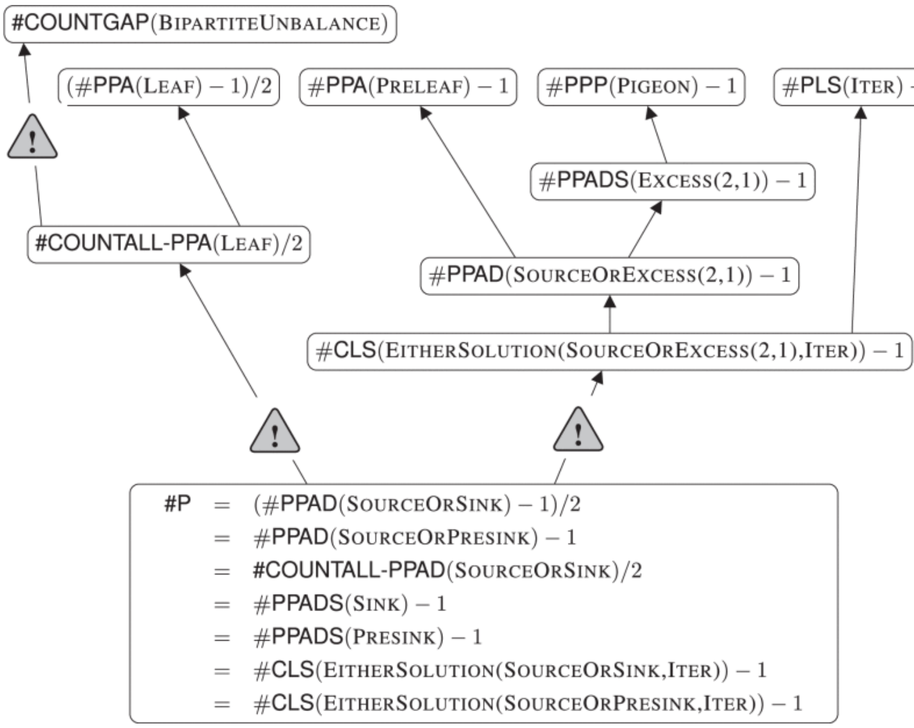


Figure 1.2.11: Total Search Problem Counting Hierarchy. Bold lines imply relativising parsimonious reductions. Lines with warning signs imply there is an oracle separation in the inverse direction. Figure by Ikenmeyer et al. [33]

argue that if a problem A presents an oracle separation from problem B, we interpret that as problem A is "combinatorially harder" than B. In our case, oracle separations have been used to argue that problems such as **#PPAD**$(SourceOrExcess(k,1)) - 1$ lack a combinatorial interpretation due to their separation from **#P**.

1.3 Project Question and Objectives

As mentioned in section 1.2.4, we observed how different **PPAD** problems can behave differently under decrementation. The main motivation comes from the observation of how *Kleene logic* can be used to simulate the computation paths of NTMs. We can parallelise this to the boolean problem of *SAT* which given by the *Cook-Levin* theorem [14] one can convert every **NP** problem into a *SAT* problem. The theorem invloves the creation of a boolean expression for each snapshot of the Turing Machine such that a satisfying assignment corresponds to an accepting path. Their theorem also demonstrated that every **NP** problem can be parsimoniously reduced to the *SAT* problem, meaning:

$$\forall L \in \mathbf{NP} : \#L \subseteq \#SAT$$

A reasonable conjecture would also be if one can exploit Kleene-circuit nature of *PureCircuit* to create a *Cook-Levin* type equivalent reduction for **PPAD** problems, denoted as

$$\forall L \in \mathbf{PPAD} : \mathbf{\#PPAD}(L) \subseteq \mathbf{\#PPAD}(PureCircuit)$$

If the above statement were indeed true, then it would give us insights as to how counting problems behave in **#PPAD** $- 1$. Unfortunately, there is limited research with respect to the combinatorial nature of the problem. We wish to offer a new perspective of *PureCircuit* and establish its *parsimonious relations* with other problems. To break down our conjecture we offer the following set of objectives:

1. Establish a chain of parsimonious reductions from the *EndOfLine* to the *PureCircuit* problem.

2. Establish a chain of parsimonious reductions from the *SourceOrExcess*$(2, 1)$ to the *PureCircuit* problem. Ideally, we would like to extend this to the entire hierarchy of *SourceOrExcess*$(k, 1)$ or more formally

$$\forall k \in \mathbb{N} : \textbf{\#PPAD}(\textit{SourceOrExcess}(k, 1)) \subseteq \textbf{\#PPAD}(\textit{PureCircuit})$$

3. Establish a *Cook-Levin* type theorem for the all **PPAD** problems, formally denoted as:

$$\forall L \in \textbf{PPAD} : \textbf{\#PPAD}(L) \subseteq \textbf{\#PPAD}(\textit{PureCircuit})$$

The above milestones were created to tackle the main conjecture in incremental steps. Demonstrating a parsimonious reduction from the *EndOfLine* is an important baseline as every **PPAD** problem reduces to the *EndOfLine*, even if it is not parsimonious. The second milestone is meant to demonstrate that we can use *PureCircuit* for problems with no combinatorial interpretations, as we can see from the figure 1.2.11. Our last objective is the overall main conjecture that we are trying to prove.

In chapter 2, we explain our methodology and provide several tools, new problems and conjectures that will aid in the future resolution or our objectives. To assist with the theory development, we set out to develop a visualisation tool that will be able to create and validate instances of the problem. We hope that this will aid with theory development, and therefore we provide the following core set of requirements:

1. Establish a program in which the user is able to construct a *PureCircuit* instance. The tool should be capable of verifying instance or highlighting unsatisfied nodes.
2. Develop algorithms to find or at least approximate solutions.
3. Construct algorithms to enumerate all solutions for small graphs.

We will a give a greater in-depth overview of the tool in the section 3 and the algorithms used to fulfil the aforementioned requirements.

Chapter 2

Theory Analysis

This section explores the approaches and findings uncovered while tackling each of our theory objectives. More specifically, we highlight several theorems that may assist in their full resolution. We state the following observation: when working with *PureCircuit*, one cannot fully work with binary logic, meaning there is no trivial method for detecting $\perp$ values and removing them, as made clear by Marino's continuity argument [40]. We therefore make use of topological problems such that resolutions of our kleene input corresponds to some set of nearby points in a space. If all points abide some property then we can use that to detect solutions. This is an extension of the non-parsimonious reduction used by Deligkas et al. [21] to show *PureCircuit* is **PPAD**-hard.

With respect to the first objective, we provide the following list of theorems.

1. *#nD-StrongSperner* $\subseteq$ *#HF-nD-StrongSperner* 2.1.1
2. *#HF-DiscreteStrongSperner* $\subseteq$ *#PureCircuit* 2.1.2
3. *#EndOfLine* $\subseteq$ *#AntipodalStrongSperner* 2.1.3

The problems mentioned are defined in the section 2.1, where we also discuss their implications with respect to the current objective and with **PPAD** as a whole.

Following the same process for our second objective, we demonstrated how to reduce *SourceOrExcess* to a topological problem. Specifically in section 2.2 we argue the following result

$$\#\textit{SourceOrExcess}(2,1) \subseteq \#\textit{3D-CountingStrongSperner} \text{ 2.2.1}$$

Where we define a different combinatorial version of the *3D-StrongSperner* and demonstrate how it can be used for parsimonious reductions between the *SourceOrExcess*$(2,1)$ problem.

In the third section, we offer a combinatorial extension to the **PPAD** class regarding directed graphs and the number of known sources. Specifically given Hollender et al.'s work [29] on directed graphs with multiple known sources, we make observations on the combinatorial nature of the problems that can be created as well as important problems to analyse for a full *PureCircuit* Cook-Levin type theorem. Lastly we demonstrate how we can relate to *Tarski* with *Kleene theory* and subsequently how it can be used for a parsimonious reduction to *PureCircuit*.

2.1 PureCircuit and EndOfLine

We built upon the *StrongSperner* reduction that Deligkas et al. [21] derived by expressing how we can redefine *StrongSperner* instances under *Kleene logic*. We subsequently demonstrate how we can use that to relate *PureCircuit* and the *EndOfLine* problem with each other as well as highlighting the current limitations of these approaches.

Hazard Free StrongSperner

First we observe that solutions of *StrongSperner* problems revolve around the notion of panchromatic hypercubes. We begin by associating points with cubes and vice versa using the definitions in 2.1.1.

Definition 2.1.1: *StrongSperner* anchor points

Given a hypercube $H \subset \mathbb{Z}^n$ of side lengths 1, we denote the $\mathbf{Anc}(H)$ as

$$\mathbf{Anc}(H) = \min_{p \in H} \|p\|_1$$

Alternatively, we will refer to these hypercubes using the notation $\mathbf{Cube}(\cdot)$ such that

$$\mathbf{Cube}(p) \triangleq \left\{(p+b) \mid b \in \{0,1\}^n\right\}$$

We would like to express a cube as a coordinate of Kleene numbers, implying that the resolutions of the point will result in a hypercube, where each point is within $\|\cdot\|_\infty \leqslant 1$ of each other. This leads to a natural interpretation of the *StrongSperner* problem in *Kleene logic*. We provide two definitions, one for *nD-StrongSperner* and the other for *StrongSperner*. We emphasize that the main difference of the two problems is that the former has fixed dimensions n of exponentially large cubes, whereas the latter can have varying dimensions of polynomially large widths.

Definition 2.1.2: *HF-nD-StrongSperner* problem

Input: A circuit $\lambda : (\{0,1\}^k)^n \to \{-1,+1\}^n$, that describes a *nD-StrongSperner* labelling, as explained in 1.2.11, such that it is hazard free on inputs $(p_i\bot)_{i\in[n]}$ where $p_i \in \{0,1\}^{k-1}$
Output: A point $p \in \bigtimes_{i\in[n]} \left(\{0,1\}^{k-1} \times \{0,1,\bot\}\right)$ if and only if: $\lambda(p) = \bot^n$.

We argue that the above problem is **PPAD**-complete for all *HF-nD-StrongSperner* problems. Firstly, we can observe that all *HF-nD-SS* are reducible to their *nD-SS* counterparts since hazard-free transformations have no effect on binary inputs and therefore our circuits act like any other boolean circuit. To show the hardness, we claim that

Theorem 2.1.1: *HF-nD-SS* hardness

$$\mathbf{\#PPAD}(\mathit{nD\text{-}SS}) \subseteq \mathbf{\#PPAD}(\mathit{HF\text{-}nD\text{-}SS})$$

Proof. Given circuit $\lambda : (\{0,1\}^k)^n \to \{-1,+1\}^n$ from *nD-SS*, assume that S is the set of anchors points of a panchromatic cube. We create $\Lambda : (\{0,1\}^{k+1})^n \to \{-1,+1\}^n$ such that:

$$\Lambda((x_i)_{i\in[n]}) = \lambda\left(\left(\left\lfloor \frac{x_i+1}{2} \right\rfloor\right)_{i\in[n]}\right) \tag{2.1.1}$$

We prove the following claim:

Claim 2.1.1: Transformation claim

Given S_{even} the set of even solutions of the transformed space or more formally:

$$S_{\text{even}} \triangleq \left\{(p_i0)_{i\in[n]} \mid p \in (\mathbb{B}^{(k)})^n : (p_i0)_{i\in[n]} \text{ is an anchor of a panchromatic cube}\right\}$$

We claim $|S| = |S_{\text{even}}|$

Proof. We argue that we can bijectively map between $S \leftrightarrow S_{\text{even}}$. To show the left to right direction, assume $p \in S$. Using figure 2.1.1 as reference, we can observe that each point becomes an n-dimensional hypercube. The hypercube that was anchored at $(p_1, ..., p_n)$ is mapped to the hypercube anchored at $(2p_1, ..., 2p_n)$. To prove that, we can observe that:

$$\begin{aligned}\forall (2p_i)_{i\in[n]} \in (\{0,1\}^{k+1})^n, b \in \{0,1\}^n : \Lambda((2p_i + b_i)_{i\in[n]}) \\ = \lambda\left(\left(\left\lfloor \frac{2p_i + b_i + 1}{2} \right\rfloor\right)_{i\in[n]}\right) \\ = \lambda\left(\left(\left\lfloor p_i + \frac{b_i + 1}{2} \right\rfloor\right)_{i\in[n]}\right) \\ = \lambda\left((p_i + b_i)_{i\in[n]}\right)\end{aligned}$$

Which implies that the cube anchored at $2p$ must also be panchromatic. We can use a similar argument to prove the inverse direction. $\square$

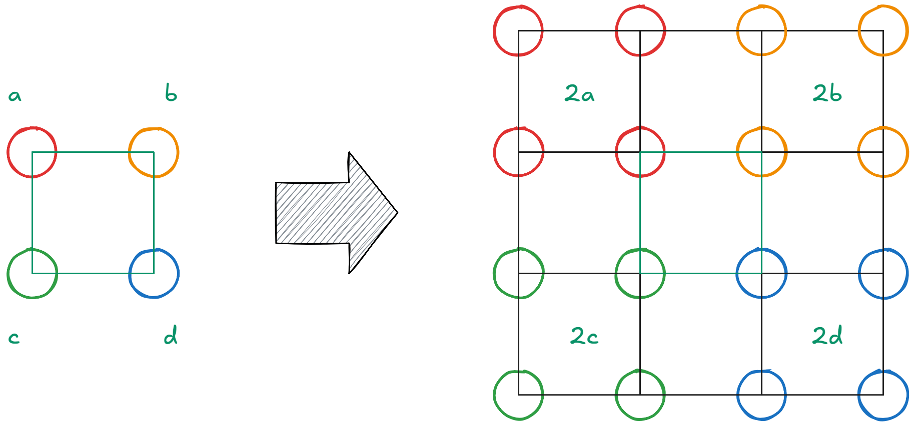


Figure 2.1.1: Dimension doubling, demonstrated for two dimensions.

To ensure that Λ is hazard free we can use the corollary 1.1, where we can create an n-bit hazard-free circuit. It can be observed that since the number of hazards increase the circuit size exponentially, higher dimensional problems have larger circuit sizes. However, since all the input instances scale with respect to the width of the hypercube space, it implies that we are able to keep the size of the circuit polynomial with respect to the input. Therefore, solutions of the form $(p_i\perp)_{i\in[n]}$ correspond to panchromatic cubes in the original space and therefore our reduction is polynomial and parsimonious. $\square$

To introduce *HF-StrongSperner*, we first focus on embedding Kleene logic into unary numbers 2.1.3. We will then demonstrate how to extend this notation to define the full *HF-StrongSperner* problem.

Definition 2.1.3: Kleene Unary Numbers

In a binary setting, we can represent a unary number $M \in \mathbb{N}$ as 1^M. Under Kleene algebra, we refer to notations M_k^n where $k, n, M \in \mathbb{N}_0$ such that:

$$M_k^n \triangleq \bigtimes_{i\in[n]} \{1^j \perp^k \mid j \in \mathbb{N}_0 : j + k \leqslant M\}$$

For a cleaner notation, given $1^j\perp^k \in M_k$ we denote it as $j\perp^k$.

For examples of the above notation we refer to the table 2.1.1.

Kleene Unary Sets	Member examples
3^2_1	$(11\bot, 1\bot0) = (2\bot, 1\bot)$
5^3_0	$(11000, 11110, 11111) = (2, 4, 5)$
7^1_2	$11\bot\bot000 = 2\bot^2$

Table 2.1.1: Kleene Unary Examples

Definition 2.1.4: *HF-StrongSperner* problem

Input: A hazard-free circuit $\lambda : [M]^n \to \{-1, +1\}^n$, where $M \in n^{O(1)}$ that describes a *StrongSperner* labelling, as explained in 1.2.11, where the inputs are represented using Kleene Unary numbers 2.1.3.
Output: A point $p \in M^n_{\leqslant 1}$ if and only if: $\lambda(p) = \bot^n$.

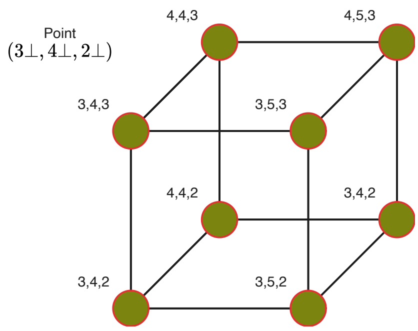


Figure 2.1.2: Kleene Hypercube using unary notation

We can use figure 2.1.2 as reference to demonstrate how the Kleene resolutions of a point relate to a cube, and therefore argue that the above is a natural extension of the *StrongSperner*. We will primarily focus on sub-problems of the *HF-StrongSperner* where two cubes do not overlap with each other. Using figure 2.1.3 as reference, if two cubes overlap, that means a subspace of the cube (face or edge) is panchromatic. This implies that for the same cube we can have multiple solutions. For example if an edge is panchromatic then the cube, the face that contains the panchromatic edge and the edge itself are all valid solutions. To simplify the counting arguments, we define the problem *HF-DiscreteStrongSperner* 2.1.5, such that all solutions are *discrete*, meaning there are no overlapping panchromatic cubes.

Definition 2.1.5: *HF-DiscreteStrongSperner* problem

An *HF-StrongSperner* instance λ such that $\forall p \in M^n_1$, if $\lambda(p) = \bot^n$, then

$$\forall y \in M^n_{\leqslant 1} : p \preceq y \implies \lambda(y) \neq \bot^n$$

Essentially, no subspace of a panchromatic cube should cover the labels.

Lastly we introduce a subset of the above instances where we enforce antipodal panchromatic cubes. In the specific type of solutions, we enforce that every two opposite corners of a cube contain opposite colourings.

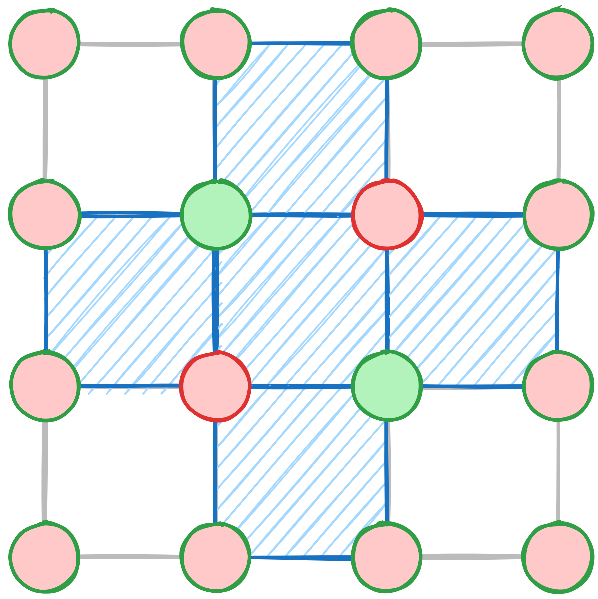


Figure 2.1.3: Overlapping Sperner Solutions

Definition 2.1.6: MinRes min and max resolution functions

We use the **MinRes**, to denote the smallest resolution of a Kleene string. More formally we can use the following equivalent definitions:

$$\forall x \in \mathbb{T}^*, j \in [|x|] : [\mathbf{MinRes}(x)]_j \triangleq \begin{cases} x_j & \text{if } x_j \in \{0,1\} \\ 0 & \text{otherwise} \end{cases}$$

For the maximum resolution we will use the **MaxRes** function where we instead replace all $\perp$ with 1 instead.

Definition 2.1.7: *HF-AntipodalStrongSperner* problem

An *HF-DiscreteStrongSperner* instance λ such that $\forall p \in M_1^n$, if $\lambda(p) = \perp^n$, and $\mathbf{MinRes}(x) = \hat{x}$, then:

$$\forall b \in \{0,1\}^n, j \in [n] : [\lambda(\hat{x} + b)]_j = -[\lambda(\hat{x} + 1^n \oplus b)]_j$$

We emphasise that the above class considers the subset of the *Hf-DiscreteStrongSperner* as the antipodality condition cannot ensure discreteness. We can also define its non-HF variant of the problem which we can refer to *AntipodalStrongSperner* where for each cube, we only need to check its two opposite ends to verify if it is a valid solution. For the discrete and the antipodal properties we emphasise that these are *promise problems*, meaning we assume that all solutions abide the properties of the description of the definition. Moreover, we claim 2.1.2 that every antipodal discrete square must contain all 2^d colours. We provide its proof in the appendix 5.6.

Claim 2.1.2: Antipodal discrete squares

Given a panchromatic solution of a *HF-AntipodalStrongSperner* p, the following holds true:

$$\forall j \in \{-1,1\}^n, \exists! x \in \mathbf{Res}(p) : \lambda(x) = j$$

All the problems above are clearly in **PPAD** as they all reduce to the *StrongSperner* problem, since we assume by construction that they *natural*. For the purposes of our current research, we demonstrate a parsimonious reduction from *HF-DiscreteStrongSperner* to *PureCircuit*.

Theorem 2.1.2

$$\textbf{\#PPAD}(\textit{HF-DiscreteStrongSperner}) \subseteq \textbf{\#PPAD}(\textit{PureCircuit})$$

Proof. Given an input instance Λ of a *HF-DiscreteStrongSperner*, we create a *PureCircuit* instance which we break down into the following parts: *Input nodes*, *Output nodes*, *Bit generator gadget* and *Circuit application gadget*. To visualise our construction we will refer to the figure in 2.1.4. Each of these parts describe the following:

1. **Input nodes**: Vector of n value nodes which will be represented as s_i for element i.
2. **Bit generator gadget**: Given a value node s_i, apply the *Bit generator* gadget C_i such that we create Kleene unary number as defined in 2.1.3. We will refer to the Kleene unary numbers as (u^i_j) for $i \in [n]$ and $j \in [M]$.
3. **Circuit application gadget**: Since the above numbers form coordinates in a $M^n_{\leqslant 1}$ space, we apply the Λ function to extract a set of output nodes $(o^i)_{i\in[n]}$.
4. **Output nodes**: We use the Copy gate to copy $o^i \to s_i$, for all $i \in [n]$.

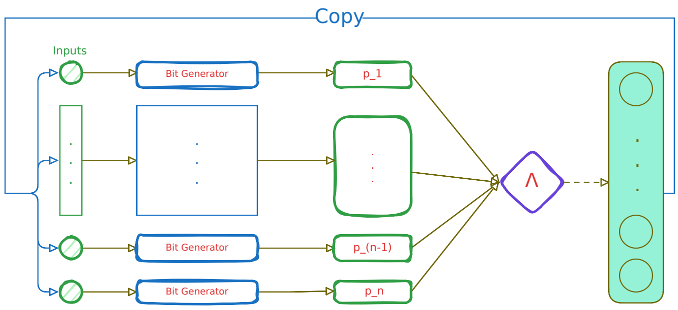


Figure 2.1.4: *PureCircuit* construction visualisation

Bit generator gadget In order to create Kleene unary numbers we create a binary tree of Purify gates as seen in the figure 2.1.5. We now claim that the current construction has certain desirable properties.

Claim 2.1.3: Bit Generator Claim

Given the description of the bit generator gadget, we make the following two claims:

1. $b \in \{0,1\} : C_i(b) = b^M$
2. $C_i(\perp) \in M^n_{\leqslant 1}$

Proof. From the definition of the Purify gate we know that $\forall b \in \{0,1\} : \text{Purify}(b) = (b,b)$, and therefore any subtree that takes a pure input will copy its value to all its leafs, which proves our

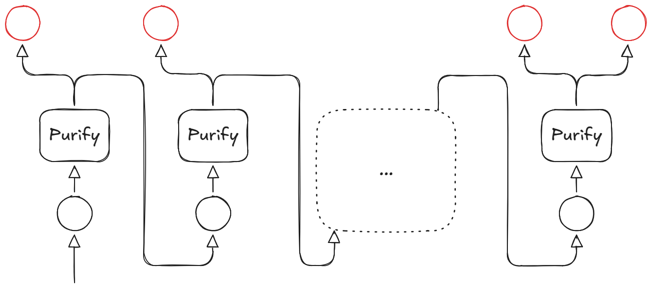


Figure 2.1.5: Bit Generator gadget. The red nodes denote the outputs of the purification gadget.

first claim. For the second part of our lemma, we can observe that the only valid outputs are $(0, \perp), (\perp, 1), (0, 1)$. Based on our previous observation, if a PURIFY gate outputs a $(0, \perp)$ value, it will propagate $\perp$ to the rest of the tree and return 0 in the current level. If the value is $(\perp/0, 1)$, the tree will return $\perp/0$ at the specific level, and then propagate 1 to the rest of the leaves. This shows that all possible strings generated are of the form $0^* \perp^{0|1} 1^k \in M_{\leqslant 1}$. □

In appendix 5.6, we can observe that our visualisation application validates our *Bit generator* claim.

Correctness and parsimonious argument One can observe that we have a valid construction since the instance is of polynomial size, all gates have the correct arity and every value node has in-degree of at most 1. It remains to demonstrate that every assignment corresponds to a solution of *HF-DiscreteStrongSperner* using lemma 2.1.1.

Lemma 2.1.1: Correctness Lemma

A correct assignment corresponds to panchromatic cube in the original problem instance.

It suffices to show that every vector assignment is of the form $\forall i \in [n] : \mathbf{x}[o^i] = \perp$. Assume that we have a correct assignment, and WLOG $\mathbf{x}[o^i] = 0$ for some $i \in [n]$. By our COPY gate, we know that $\mathbf{x}[o^i] = \mathbf{x}[s_i]$. By our bit generator lemma 2.1.3 we know that $\mathbf{x}[u^i] = 0$. Since our circuit is hazard-free and applies the boundary conditions of the Sperner problem, we know that $[\Lambda(u)]_i = 1 \neq \mathbf{x}[o^i]$. We can make a similar argument for $\mathbf{x}[o^i] = 1$. Therefore, a satisfying assignment corresponds to some panchromatic solution. Given that our instance is in *HF-DiscreteStrongSperner*, it has to be the case that all $u^i \in M_1$, otherwise that would imply that there exists a $(n-1)$-subspace that also covers all the labels. This also shows that there exists a 1-to-1 mapping between a panchromatic cube and a satisfying assignment. □

Although the above reduction also works for the general *HF-StrongSperner*, it will not be parsimonious since the *PureCircuit* will accept solutions for all $(n-1)$ subspaces that contain a covering set. Since all *AntipodalStrongSperner* instances are also discrete, it directly follows the corollary 2.1:

Corollary 2.1

$$\textbf{\#PPAD}(\textit{HF-AntipodalStrongSperner}) \subseteq \textbf{\#PPAD}(\textit{PureCircuit})$$

EndOfLine to AntipodalStrongSperner

Proving hardness for *HF-StrongSperner* is a much harder task. Chen et al. [12] showed that every Sperner problem of space $[2^{f(n)}]^d$ where $d = \left\lceil \frac{n}{f(n)} \right\rceil$ and $f(n) \leqslant n/2$ for large enough n, is **PPAD**-complete and parsimonious from the *EndOfLine* problem, using *Snake Embedding* technique. The technique works by starting from a *2D-Sperner* instance and repeatedly fold and pad the dimensions until they reach the desired space.

We would like to use the same method to go from *2D-HF-DiscreteSS* to *HF-DiscreteSS* as that would demonstrate a full parsimonious reduction chain from the *EndOfLine* since it can be parsimoniously reduced to *2D-HF-DiscreteSS*. However we cannot guarantee the hazard-freeness property after every fold. If we try to use the corollary 10 by 1.1, we observe that the circuit size will increase exponentially. This implies the best we can hope is the following space:

$$f(n) = \log_2(n)^k \implies M = n^k, d = \frac{n}{log(n)^k}$$

However that would imply that the size of our circuit is $\Omega\left(2^{n/(log(n))^k}\right)$. We propose a chain of parsimonious reductions from the *EndOfLine* to *AntipodalStrongSperner*. We emphasise that this method will not be hazard-free, but we hope that the antipodal properties of the resulting problem may enable for a full reduction in the future since by corollary 2.1, we know that its hazard-free variant can be parsimoniously reduced to *PureCircuit*. We begin with a formal description of the planar embedding technique by Chen et al. [11] in order to reduce to *2D-EoL* instances.

Planar Graph Embedding We define the graph embedding of a graph $G = (V, E)$ as $G_n = (V_n, E_n)$ where $|V| = n$ and V_n is defined as:

$$\forall n \in \mathbb{N} : V_n \triangleq \left\{\mathbf{u} \in \mathbb{N}_0^2 \mid \mathbf{u}_1 \leqslant 3(n^2 - 1), \mathbf{u}_2 \leqslant 6(n-1) + 3\right\}$$

G can have any in- and out-degree. For a vertex $i \in V$, we correspond it to the node $(0, 6i)$ in the embedded version. To define the edge set, we introduce the following notations:

Definition 2.1.8: $E(\mathbf{p}^1\mathbf{p}^2)$

Given two points $\mathbf{p}^1, \mathbf{p}^2 \in V_n$ we define the set of edges $E(\mathbf{p}^1\mathbf{p}^2)$ as:

$$E(\mathbf{p}^1\mathbf{p}^2) \triangleq \begin{cases} \{(\alpha, k) \mid k \in \mathbb{N} : \min\{\mathbf{p}_2^1, \mathbf{p}_2^2\} \leqslant k \leqslant \max\{\mathbf{p}_2^1, \mathbf{p}_2^2\}\} & \text{if } \mathbf{p}_1^1 = \mathbf{p}_1^2 = \alpha \\ \{(k, \alpha) \mid k \in \mathbb{N} : \min\{\mathbf{p}_1^1, \mathbf{p}_1^2\} \leqslant k \leqslant \max\{\mathbf{p}_1^1, \mathbf{p}_1^2\}\} & \text{if } \mathbf{p}_2^1 = \mathbf{p}_2^2 = \alpha \\ \varnothing & \text{otherwise} \end{cases}$$

Definition 2.1.9: E_{ij}

Given an edge of the original graph $(i, j) \in E$, we define the set of edges E_{ij} as:

$$\begin{aligned} \mathbf{p}^1 &= (0, 6i) \\ \mathbf{p}^2 &= (3(ni + j), 6i) \\ \mathbf{p}^3 &= (3(ni + j), 6j + 3) \\ \mathbf{p}^4 &= (0, 6j + 3) \\ \mathbf{p}^4 &= (0, 6j) \\ E_{ij} &\triangleq E(\mathbf{p}^1\mathbf{p}^2) \cup \cdots \cup E(\mathbf{p}^4\mathbf{p}^5) \end{aligned}$$

To formally define E_n, we first split E_n into two disjoint sets E_n^1 and E_n^2. We define E_n^1 as $\bigcup_{ij \in E} E_{ij}$. The graph generated by (V_n, E_n) has some special properties. For instance, $\forall v \in V_n : \text{in-deg}(v) + \text{out-deg}(v) \leqslant 4$ and $\forall v \in V_n : \text{in-deg}(v) + \text{out-deg}(v) = 4 \implies \text{in-deg}(v) = 2 = \text{out-deg}(v)$. Moreover, for every 4-degree node, every incoming edge has an opposite outgoing edge. We use this fact to add 4 additional edges denoted in blue, based on the cases denoted in figure 2.1.6.

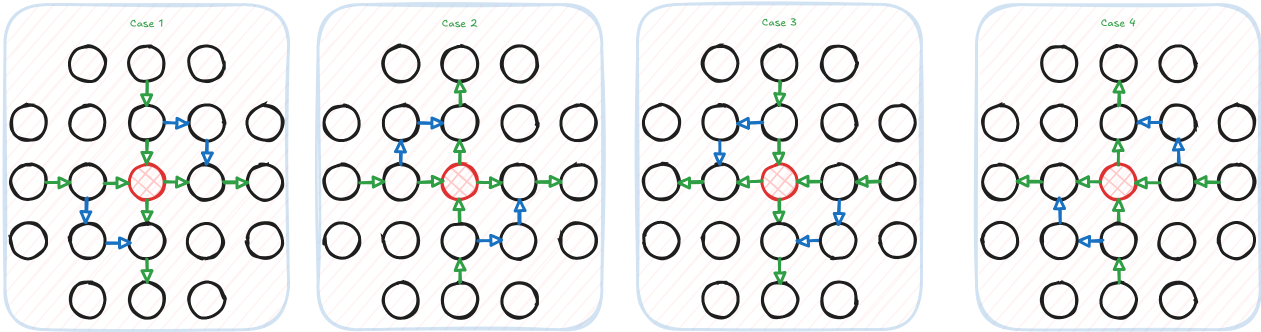


Figure 2.1.6: Edges to add for each 4 degree vertex. Figure redrawn from [11]

Given the aforementioned, we have a full description of G_n. A visual explanation can be seen in figure 2.1.7 and an example of the K_3 complete graph in figure 2.1.8.

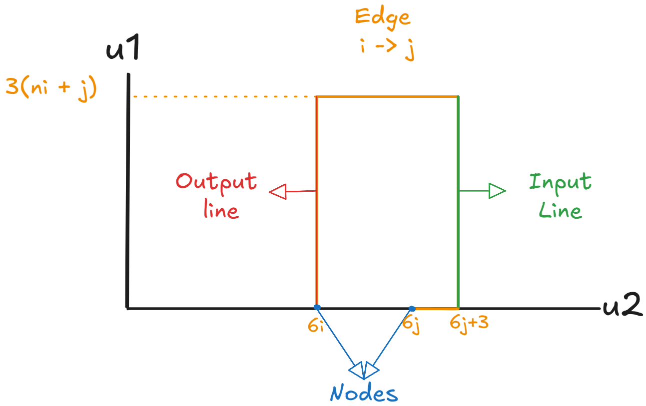


Figure 2.1.7: Edge between points i, j of the original graph which illustrates that each point comes with a pair of input output lines.

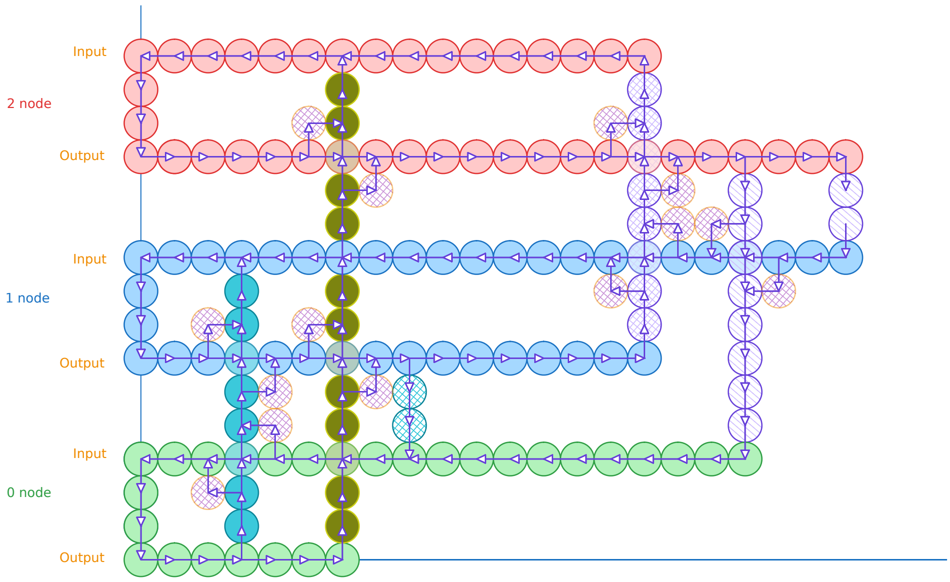


Figure 2.1.8: K_3 graph construction. Combinations of colours indicate an edge between the two nodes. Solid dots indicate edges from 0, crossed lines indicate edges from 1 and stripped lines indicate edges from 2. We add two coloured stripped circles on the 4 degree nodes to indicate the E_n^2 edges. Image reconstructed from [11].

Embedding of *EndOfLine* Chen et al. [11] demonstrated that we can apply a planar embedding of a *EndOfLine* graph with vertex set $V = \{0,1\}^k$ such that we get graph $G_{2^k} = (V_{2^k}, E_{2^k})$. Moreover they demonstrated the following property:

$$\forall v \in V_{2^k} : deg(v) \in \{(0,0),(1,1),(1,0),(0,1),(2,2)\}$$

Using figure 2.1.9 as reference, we remove all the edges of nodes with degree of $(2,2)$.

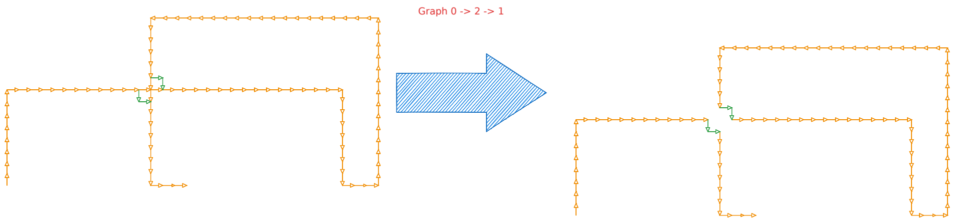


Figure 2.1.9: Depiction of the edge removal. Main idea is that this we keep the number of sources or sinks by redirecting the lines.

Assume that $C_n \subset G_n$ are the sets of all planar graphs such that

$$\forall v \in V_n : deg(v) \in \{(1,0),(0,0),(0,1),(1,1)\}$$

The key insight is that we get a *2D-EOL* instance where the number of sources or sinks is preserved. and therefore the reduction is parsimonious and polynomial.

Bipolar colouring of *2D-EoL* We extend Chen et al. [11] colouring technique for *bipolar colouring*. Given an *2D-EoL* we construct space :

$$B_n \triangleq [n]_0 \times [n]_0$$

We will use points $\mathbf{u}, \mathbf{v}, \mathbf{w} \in V_{2^k}$ and $\mathbf{p}, \mathbf{q}, \mathbf{r} \in B_{2^{4k+10}}$. Below we will denote our colouring scheme for clarity:

$$\begin{aligned} + &= (+1,+1) \\ \pm &= (+1,-1) \\ \mp &= (-1,+1) \\ - &= (-1,-1) \end{aligned}$$

First, we define mapping $\mathcal{F} : V_{2^k} \to B_{2^{4k+10}}$ such that

$$\mathcal{F}(\mathbf{u}) = \begin{pmatrix} 6\mathbf{u}_1 + 6 \\ 6\mathbf{u}_2 + 6 \end{pmatrix} \in B_{2^{4k+10}}$$

Since $G \in C_{2^k}$, its edge-set can be decomposed into a collection of paths and cycles $(P_i)_{i \in [m]}$. By using $\mathcal{F}$, every P_i is mapped to a set $I(P_i) \subset B_{2^{4k+10}}$.

$$\begin{gathered} P = \mathbf{u}^1, \ldots, \mathbf{u}^t, \text{if P is a cycle } \mathbf{u}^1 = \mathbf{u}^t \\ I(P) \triangleq \bigcup_{i \in [t-1]} E(\mathcal{F}(\mathbf{u}^i)\mathcal{F}(\mathbf{u}^{i+1})) \end{gathered}$$

This method of shielding, is commonly used when dealing with *Line reductions*, as seen in [11, 20, 2, 12, 15]. We embed our lines and cycles in a grid such that, points that are not on a line have a negative charge $-$. All lines are denoted as positive charge $+$. To protect our lines from creating solutions, we cover them with $\pm, \mp$ lines. A visualisation can be seen in figure 2.1.10. We denote the set of shielding points around our paths as:

$$O(P) = \{\mathbf{p} \in B_{2^{4k+10}} \backslash I(P) :|: \exists \mathbf{p}' \in I(P), \|p - p'\|_\infty = 1\}$$

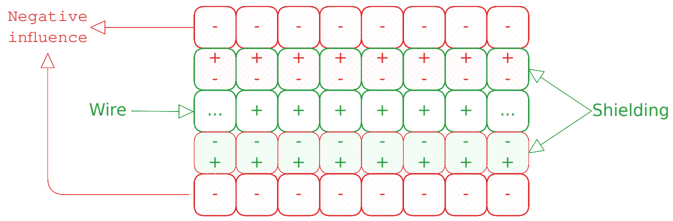


Figure 2.1.10: Shielding method on 2D-EoL embeddings. We can observe that each square of our figure does not yield a panchromatic solution.

If P is a simple path we can decompose $\{\mathbf{s}_p, \mathbf{e}_p\} \cup L(P) \cup R(P)$ such that: $\mathbf{s}_p = \mathcal{F}(\mathbf{u}^1) + \Delta + \Delta^\perp$, where $\Delta = (\mathbf{u^1} - \mathbf{u}^2)$, $\mathbf{e}_p = \mathcal{F}(\mathbf{u}^t) + \bar{\Delta} + \bar{\Delta}^\perp$, where $\bar{\Delta} = (\mathbf{u}^t - \mathbf{u}^{t-1})$ and $x^\perp$, denotes the vector perpendicular to x. Choice of which perpendicular vector we consider is arbitrary. Starting from $\mathbf{s}_p$, enumerate vertices in $O(P)$ clockwise as $\mathbf{s}_p, \mathbf{q}^1, \ldots, \mathbf{q}^{n_1}, \mathbf{e}_p, \mathbf{r}^1, \ldots, \mathbf{r}^{n_2}$ such that:

$$L(P) = \{\mathbf{q}^i\}_{i \in [n_1]} ,$$
$$R(P) = \{\mathbf{r}^i\}_{i \in [n_2]}$$

If P is a simple cycle, then we can decompose $L(P) \cup R(P)$, where $L(P)$ contains all the vertices on the left side of the cycle and $R(P)$ all the vertices on the right side. If G is specified by $(S, P, 0^k) \subset C_{2^k}$, we can uniquely decompose the edge set into $P_1, \ldots P_m$. Every path is either a maximal path, or a cycle in G. We can use the following to describe our colouring algorithm:

Algorithm 2.1.1 Turing machine F with input $(p_1, p_2) \in B_{2^{4k+10}}$

if $p_1 = 0$ **then**
    if $p_2 \leqslant 6$ **then**
        return $+$
    else
        return $\pm$
else if $p_1 < 6$ **thenk**
    if $p_2 = 6$ **then**
        return $+$
    else if $p_2 < 6$ **then**
        return $\mp$
    else if $p_2 = 7$ **then**
        return $\pm$
    else
        return $-$
else
    return $M_K(\mathbf{p})$

Where $M_k(\mathbf{p})$ is can be described with the following description:

$$M_k(\mathbf{p}) = \begin{cases} + & \text{if } \exists i, \mathbf{p} \in I(P_i) \\ \pm & \text{if } \exists i, \mathbf{p} \in L(P_i) \vee p_1 = 0 \\ \mp & \text{if } \exists i, \mathbf{p} \in R(P_i) \vee p_2 = 0 \\ - & \text{otherwise} \end{cases}$$

We will call the path of nodes $(0,0) \to (0,6) \to (6,6)$ as the tail, which ensures that the 0^n node of our original graph is not a solution. We refer to the figure 2.1.11 for examples of such colouring.

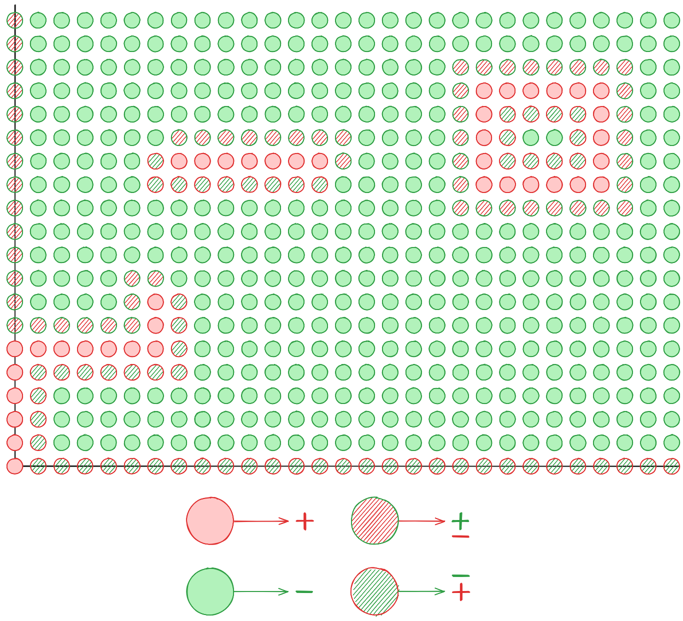


Figure 2.1.11: *2D-EoL* Bipolar Colouring

Lemma 2.1.2: Correctness Lemma

The following statements hold correct:

1. $\forall i, j : \min_{x \in P_i, y \in P_j} \|\mathcal{F}(x) - \mathcal{F}(y)\|_\infty > 4$
2. All panchromatic squares represent solutions

To prove the first statement, assume we have $a, b \in V_{2^k} : \|a - b\|_\infty \geqslant 1$. WLOG assume $a - b = (x, y)$ for some $x, y \in \mathbb{N}$ which implies the following:

$$\mathcal{F}(a) = \mathcal{F}\begin{pmatrix} b_x + x \\ b_y + y \end{pmatrix} = \begin{pmatrix} 6b_x + 6x + 6 \\ 6b_x + 6y + 6 \end{pmatrix}$$

$$\mathcal{F}(a) - \mathcal{F}(b) = \begin{pmatrix} 6x \\ 6y \end{pmatrix}$$

Since $x + y > 0 \implies \|\mathcal{F}(a) - \mathcal{F}(b)\|_\infty \geqslant 6 > 4$. This statement indicates that all lines and their shieldings are far enough from each other such that the colourings will not create additional solutions. Visually, if we had two parallel lines in V_{2^k}, then the mappings with the colourings, can be illustrated as in figure 2.1.12.

To show the second part of our lemma, we can observe that: All nodes that are not boundary and not in $\bigcup_{i \in [m]} I(P_i) \cup L(P_i) \cup R(P_i)$ are coloured $-$. We will use figure 2.1.11 for reference. We

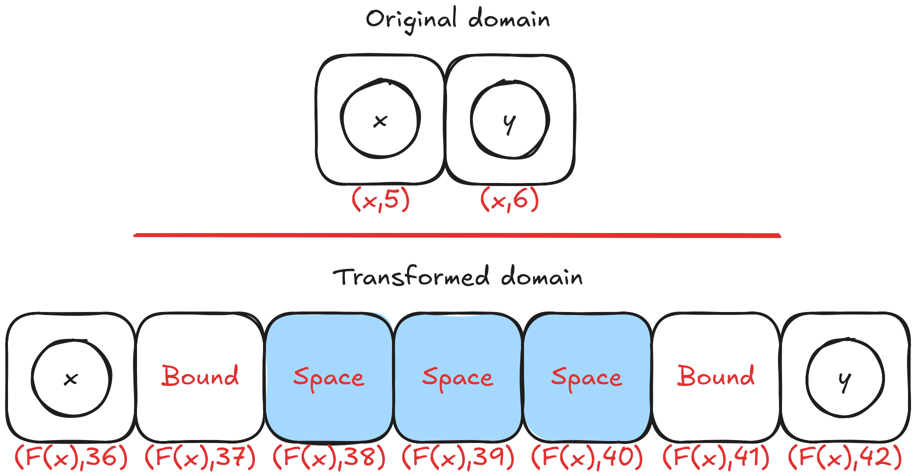


Figure 2.1.12: Transformed domain mapping example.

can observe that all squares on the left or the right of the line do not result in solutions as they contain only labels $\{\pm, +\}$ and $\{\mp, +\}$ respectively, as seen in figure 2.1.11. One can observe that the square that contains $\mathbf{s}_{P_i}$ or $\mathbf{e}_{P_i}$ and a point of the line will be panchromatic as well as discrete and antipodal. Therefore, we have a valid and parsimonious reduction since only the ends of lines can be solutions. We call these coloured instances as *2D-StrongEoL*.

Going from 2D to higher dimensions We now demonstrate our transformation from *2D-StrongEoL* to *AntipodalStrongSperner*. We employ a slightly adjusted method than the one proposed by Chen et al. [12], which closely resembles the *Snake Embedding* technique used by Deligkas et al. [20] on the *StrongTucker* version of the problem. The latter reduced the space from $2^n \times 2^n \to [7]_0^m$, where $m = O(n)$. We argue that such reduction will resolve in an *AntipodalStrongSperner* instance.

Theorem 2.1.3

$$\textbf{\#PPAD}(\textit{2D-StrongEoL}) \subseteq \textbf{\#PPAD}(\textit{AntipodalStrongSperner})$$

Proof. We are given as an input a $2^n \times 2^n$ grid representing any *2D-StrongEoL* problem. The procedure is summarised as follows: Given dimension to fold i where width of i is $n > 8$, apply the *padding operation* such that $n' = 3 \cdot s + 1$ for some $s \in \mathbb{N}$. Then apply the *folding operation*, which reduces "width" of the space to $s + 3$ and compresses the space into a higher dimension. We will now go into a greater detail as to how each operation works. To make notation simple and consistent, we define M^t as

$$M^t \triangleq \bigtimes_{i \in [n_t]} [m_i^t]$$

Where m_i^t denotes the width of dimension i at iteration t, and n_t as the number of dimensions.

Padding Dimensions We will use the definition 2.1.10 to define our padding operation.

Definition 2.1.10: Padding dimension $P(T, i, u)$

Given labelling function $\Lambda^t : M^t \to \{-1, +1\}^{n_t}$, we define the padding operation $P(\Lambda^t, i, u)$ for $i \in [n_t]$ and $u \in \mathbb{N}$ where $n_{t+1} = n_t$ and

$$\forall j \in [n_{t+1}] : m_j^{t+1} = \begin{cases} m_i^t & \text{if } i \neq j \\ m_i^t + u & \text{otherwise} \end{cases}$$

The above operation extends dimension i with u additional layers. The operation works by appending a *null* dimension, such that given $\bar{x} = x_{-i}(m_i^t + 1)$ where $x \in M^t$

$$\forall j \in [n_{t+1}] : [\Lambda^{t+1}(\bar{x})]_j = \begin{cases} -1 & \text{if } \bar{x}_i > 0 \\ +1 & \text{otherwise} \end{cases}$$

We can be sure that the above will not result in any new solutions since in $[\Lambda^t(\cdot, m_i^t, \cdot)]_i = -1$, and therefore any solution between $(\cdot, m_i^t, \cdot), (\cdot, m_i^t + 1, \cdot)$ will not yield a solution. We can repeat the above procedure u times, if we need to pad the dimension by u.

Snake Folding We will provide an overview of the snake folding operation. We first give an informal overview of the technique based on the figure 2.1.13. The y-axis denotes the new dimension, denoted as n_{t+1} and the x-axis denotes the folded dimension as $\bar{i}$ with width $m_{\bar{i}}^{t+1} = s + 3 = m_{\bar{i}}^{t'}$, given that the original width was $3s + 1$ for some $s \in \mathbb{N}$. Moreover, we will use notation (a, b) to refer to a as the labels in the n_t subspace and b as the label of the $n_t + 1$ dimension. From the figure we observe that we have orange, red, green and blue lines. Blue and orange lines denote $(\omega, -)$ and $(\omega, +)$ where ω is the n_t-dimensional *null* space, where for each point > 0, we use label $-$ and for 0 we use label $+$. These lines are meant to satisfy the boundary conditions. For the red and green lines, these are denoted as $(\star, -), (\star, +)$ where $\star$ is the n_t-dimensional copy of the lower space with some additional points. All solutions must belong in between the red and green lines, such that if we have a solution in the n_t-space, we should also have a solution in the $n_t + 1$ space.

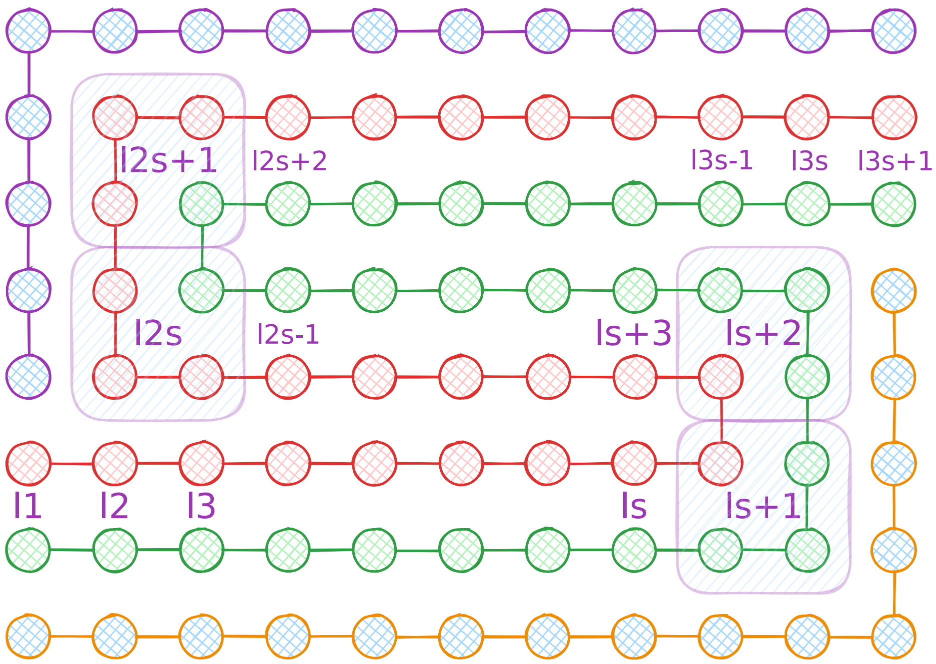


Figure 2.1.13: Snake Embedding technique view between the folded dimension j and the new dimension. The x-axis denotes the folded dimension and the y-axis denotes the new dimension. The point ℓ_i denote the indexes that correspond the j dimension of the pre-folded space. Figure inspired by [21].

To formally define the aforementioned, we define sets $B^L, B^U \subseteq M^{t+1}$ which denote the set of boundary indices of the orange and blue lines. In addition, we define sets $\ell^P, \ell^N \subseteq M^{t+1}$ to denote the points on the green and red lines with functions ϕ^P, ϕ^N that are the inverse maps to the original space. Given our new circuit Λ^{t+1}, we define the following:

$$\forall x \in M^{t+1}, j \in [n_t] : [\Lambda^{t+1}(x)]_j = \begin{cases} -1 & \text{if } x \in B^U \cup B^O \wedge x_j > 0 \\ +1 & \text{if } x \in B^U \cup B^O \wedge x_j = 0 \\ [\Lambda^t(\phi^P(x))]_j & \text{if } x \in \ell^P \\ [\Lambda^t(\phi^N(x))]_j & \text{if } x \in \ell^N \end{cases}$$

For the new dimension, we apply the labelling rules as described previously:

$$\forall x \in M^{t+1} : [\Lambda^{t+1}(x)]_{n_t+1} = \begin{cases} -1 & \text{if } x \in B^U \cup \ell^N \\ +1 & \text{if } x \in B^L \cup \ell^P \end{cases}$$

Using the above, we formally define the snake embedding operation $S(T, i)$.

Definition 2.1.11: Snake Embedding operation $S(\Lambda^t, i)$

Given labelling function $\Lambda : M^t \to \{-1, +1\}^{n_t}$, apply the snake embedding operation on dimension $i \in [n]$, where $m_i \equiv 1 \mod 3$, to a get a new labelling function $\Lambda^{t+1} : M^{t+1} \to \{-1, +1\}^{n_{t+1}}$ such that $m_i^{t+1} = s + 3, n_{t+1} = n_t + 1, m_{n_{t+1}}^{t+1} = 8$, where Λ^{t+1} is defined based on the above description.

Claim 2.1.4: Antipodality Solution preservation

Given operation $S(\Lambda^t, i)$, we argue the following hold true:

1. The operation will preserve the discrete antipodal properties of the function.
2. The number of solutions between the two spaces will be preserved.

Proof. Assume we apply the snake folding operation on dimension j. We can observe that all solutions in the embedded space will be anchored between the red or green lines (using figure 2.1.13), otherwise the new dimension will not be covered. Moreover, one can observe that a panchromatic hypercube can never be anchored in any of the turning points since our labelling is discrete.

Assume we have a point in $p \in M^{t+1}$, which is an anchor of some panchromatic solution for some $p_j \in [m_j^{t+1}]$ and $p_x \in [m_{n_{t+1}}^{t+1}]$. We can observe from the diagram 2.1.13 that the coordinates $(\cdot, p_j, p_x)$ correspond to some point $(\cdot, j)$ for some point in the pre-folded space. We can also observe that the point $(\cdot, j+1)$ may correspond to the point $(\cdot, p_j + 1, p_x)$ or $(\cdot, p_j, p_x + 1)$, depending on whether $j \in \{2s, 2s+1\}$, as it implies whether its on the vertical or horizontal portion portion of the space. We denote this observation as the effective direction, meaning if $(\cdot, j) \to (\cdot, j+1) \implies (\cdot, p_j, p_x) \to (\cdot, p_j + 1, p_x)$, then we say that the jth dimension is the effective direction, otherwise the n_{t+1}th will be the effective dimension.

WLOG we assume the effective dimension is j. Since $(\cdot, p_j, p_x)$ is the anchor of a panchromatic cube, it implies that the points $(\cdot, p_j, p_x), (\cdot, p_j + 1, p_x)$ cover all the n_t labels, and $p_x, p_x + 1$ will cover the n_{t+1}th dimension. This shows that we can preserve the cardinality of the solution sets.

We can also observe that our solutions remain antipodal. WLOG assume that the effective dimension is j. We know that if p is a solution of the pre-folded, then $\forall b : \Lambda^t(p + b) = -\Lambda^t(p + \bar{b})$. This implies that if $\hat{p}$ is the relevant corresponding solution in the folded space then:

$$\forall b \in \{0,1\}^{n_{t+1}} : \Lambda^{t+1}(\hat{p} + b_{-n_{t+1}}(0)) = -\Lambda^{t+1}(\hat{p} + \bar{b}_{-n_{t+1}}(1))$$

since the n_{t+1}th dimension will not affect the labelling of the other dimensions. We can use a similar argument if the effective dimension is n_{t+1}, since it will take the role of the jth dimension.

□

We observed that our operations do not affect the cardinality of solutions. We now outline our construction of our new circuit

1. Check if there is a dimension j with $m^t_j > 8$. If yes, first pad the dimension such that $m^t_j \equiv 1 \mod 3$. Afterwards fold the dimension on j to get new folded circuit
2. If the folding results in a width of $m^t_j < 8$, then we can pad the dimension to have a width of 8
3. If there are no operations that can be applied, we can terminate

We can observe that in each step of the algorithm we preserve our solutions. Moreover, each fold divides the dimension by some factor of 3. Since we start with space of $2^n \times 2^n$, we can conclude that the number of operations needed are $O(n \log_3 2)$, concluding our proof. □

This is the closest that we manage to bring the reduction chain across *EndOfLine* and *PureCircuit*. We can observe two interesting directions: if one can create a bit generator for Kleene binary numbers as presented previously, then we can use a similar construction to the proof we did in 2.1.2 for a full parsimonious reduction between *nD-SS* to *PureCircuit*. Alternatively one can attempt to reduce *AntipodalStrongSperner* to *HF-AntipodalStrongSperner* or *PureCircuit* parsimoniously. We argue that the antipodality property allows us to check for solutions more efficiently than trying to find $d + 1$ points in some hypercube. In hindsight, we argue that if one could construct a Kleene gadget to generate the maximum and minimum resolution of a *Kleene unary*, it would allow us to create *PureCircuit* instances that take advantage of the antipodal property.

2.2 SourceOrExcess analysis

The current section focuses on how we can extend our reductions to *SourceOrExcess* using topological interpretations. By providing a topological interpretation of the *SourceOrExcess* problem, we hope to re-use the construction for future reductions with the *PureCircuit* problem.

We first demonstrate how to topologically reduce the *SourceOrExcess*$(2, 1)$ problem to a *2D* space. We will demonstrate some limitations with *2D* topological problems, and then show how using a *3D* counting version of the *StrongSperner* problem will allow for a full parsimonious reduction.

2.2.1 Topological representations

Given a *SourceOrExcess*$(2, 1)$ input we first eliminate all excess nodes, meaning nodes with degree $(2, 1)$ by separating one of the lines as shown in figure 2.2.1. This simplifies our counting argument to only 3 scenarios. Furthermore, for double sinks, we enforce that the MSB value of the bit-string for one of the two values is one. We can observe that these modifications can be done in polynomial time since they only require to analyse the local neighbouring of nodes, and they preserve the number of solutions.

Planar Transformation We reuse the planar embedding method 2.1 with slight deviations. First we double V_n as such:

$$\forall n \in \mathbb{N} : V_n \triangleq \left\{ \mathbf{u} \in \mathbb{N}_0^2 \mid \mathbf{u}_1 \leqslant 6(n^2 - 1), \mathbf{u}_2 \leqslant 12(n - 1) + 6 \right\}$$

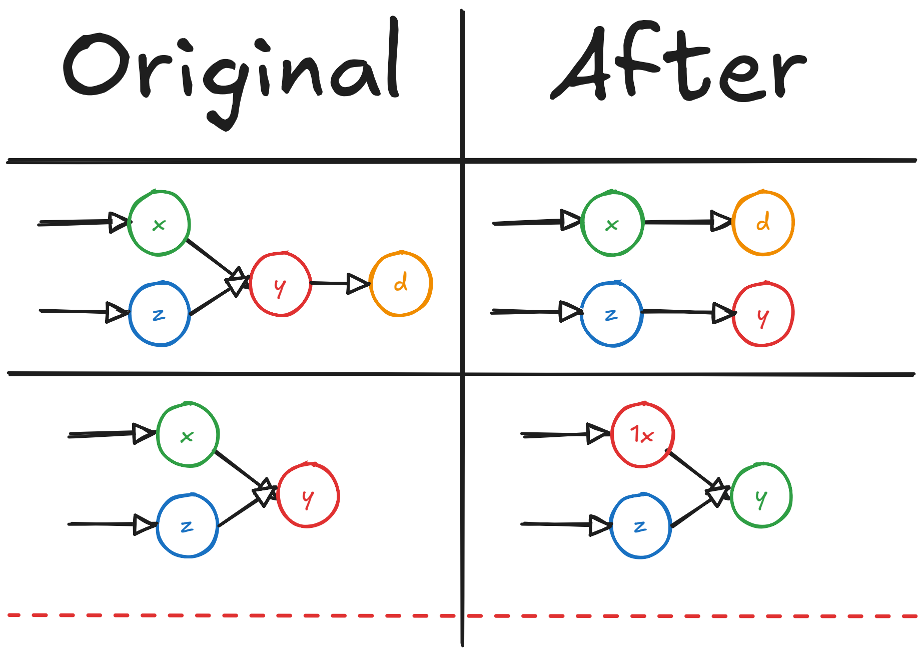


Figure 2.2.1: The current table shows how we deal with excess nodes. All other nodes such as sources, sinks, lines or isolated vertices remain the same.

Every node is now represented as $(0, 12i)$. Moreover given an edge in $(i, j) \in E_n$ where the MSB of i is 0, we define path:

$$\begin{aligned}
\mathbf{p}^1 &= (0, 12i) \\
\mathbf{p}^2 &= (6(ni + j), 12i) \\
\mathbf{p}^3 &= (6(ni + j), 12j + 6) \\
\mathbf{p}^4 &= (0, 12j + 6) \\
\mathbf{p}^5 &= (0, 12j) \\
E_{ij} &= E(\mathbf{p}^1\mathbf{p}^2) \cup \cdots \cup E(\mathbf{p}^4\mathbf{p}^5)
\end{aligned}$$

If j is a double sink, attach a small tail:

$$\begin{aligned}
\mathbf{p}^6 &= (3, 12j) \\
E'_{ij} &= E_{ij} \cup E(\mathbf{p}^5\mathbf{p}^6)
\end{aligned}$$

If the MSB value of i is 1, then we create path

$$\begin{aligned}
\mathbf{p}^1 &= (0, 12i) \\
\mathbf{p}^2 &= (6(ni + j), 12i) \\
\mathbf{p}^3 &= (6(ni + j), 12j) \\
\mathbf{p}^4 &= (3, 12j) \\
E_{ij} &= E(\mathbf{p}^1\mathbf{p}^2) \cup E(\mathbf{p}^2\mathbf{p}^3) \cup E(\mathbf{p}^3\mathbf{p}^4)
\end{aligned}$$

In the original problem, we argued that for every node, we have I/O lines that denote the successor and predecessors of the current node, which can be exponentially large. This would make it computationally difficult to differentiate between double sinks and excess nodes, and we would end up double counting solutions. By removing excess nodes, we only need to focus on double sinks. Since there is no output line in a double sink, we send one of the predecessors from the input path, and the other from the output path.

We use similar arguments as in the previous planar graph construction 2.1 to argue that: $\forall v \in V : degree(v) \in \{(0,1),(1,0),(1,1),(2,2),(2,0)\}$. We can observe that if we do not have any double sinks, our graph follows the original transformation where only 4 out of the 5 potential degrees are possible. The only difference occurs on double sinks. We can observe that if the MSB of a node is 1, it can only ever be incident to double sink, meaning $(3, 12i)$ can never have an outwards edge. We know that for every node in its path, besides $(3, 12i)$, its degrees can only ever be one of the 4 cases outlined. Similarly, for the other node with LSB 0, its main path stays the same except if its incident to a double sink, where we add 3 additional edges. For all edges, we know that the height of the path $E(\mathbf{p}^2, \mathbf{p}^3)$ can only be at height greater or equal to 6, and therefore no path other than the two incoming edges will ever intersect $(3, 12i)$. The above paths denote the set E_n^1.

We follow the same construction for the E_n^2 and apply the same removal transformation. The resulting graph is the same as in the traditional *2D-EoL* problem but with the addition of double sinks, and we call this *2D-SourceOrExcess*$(2,1)$. Additionally, we can observe that *2D-SourceOrExcess*$(2,1)$ is parsimonious to the *SourceOrExcess*$(2,1)$ problem as sources and sinks are preserved.

Topological Difficulty We observe that in the *2D-StrongEoL* bipolar colouring, we use the left-handed rule to colour the shielding of our lines in a specific way, such that we always end up with panchromatic solutions at the ends of a line. The issue is that when we try to extend the same logic to the *2D-SourceOrExcess*$(2,1)$ problem, we end up with the problem shown in figure 2.2.2. Any permutation of colours we choose will always result in 2+ solutions. Even if we relax the problem to allow squares incident to panchromatic edges to count as a single solution, we still obtain multiple solutions. The problem occurs due to the chirality of the colouring since we are connecting a left-handed line with another left-handed line at opposite ends.

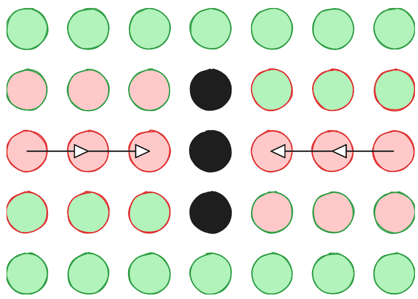


Figure 2.2.2: *2D-StrongSperner* with *2D-SourceOrExcess*$(2,1)$

The chirality problem extends *StrongSperner* instances. Hollender et al. [29], demonstrated how one can reduce parsimoniously the *2D-EoL* problem to *1-MC*. In summary, given an edge between two nodes, we can add 3 domino tiles to represent the edge. When two opposite paths meet, we always end up with two squares that cannot be covered, as seen in the figure 2.2.3. We can observe that due to chirality, we can never find appropriate tiling.

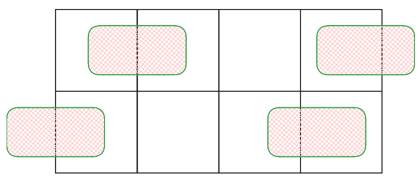


Figure 2.2.3: *1-MC* with *2D-SourceOrExcess*$(2,1)$ at the double sink points

2.2.2 Topological colouring

The key insight of our objective is showing how expand the dimension space to $3D$ and allowing panchromatic edges, allows us to achieve parsimonious reductions with *StrongSperner*. We first define the problem *3D-CountingSS*, which is a combinatorial friendly version of the *3D-StrongSperner*.

Definition 2.2.1: *3D-CountingSS* problem definition

Input: A colouring circuit $\lambda : (\{0,1\}^k)^3 \to \{-1,+1\}^3$ which follows the boundary conditions of the *StrongSperner* problem as explained in 1.2.11.
Output: A coordinate $p = (i,j,k) \in (\{0,1\}^k)^3$ such that the points in $\mathbf{Cube}(p)$ cover all labels. To reduce duplicated solutions, assume there exists $H \subset \mathbf{Cube}(p)$ which also covers the labels. Assume $\mathcal{D} = \{A \subseteq H \mid A \text{ covers all labels}\}$ such that $A = \arg\min_{A\in\mathcal{D}} |A|$. Accept only if $p = \mathbf{Anc}(A)$.

The above is a natural extension of the *StrongSperner* interpretation. Using figure 2.2.4, in the traditional Sperner problem, we would count two solutions. In the current variant we argue that it suffices to only count the edge, as it is the set that covers our labels. We abuse this notion to count only the anchor of the cube which is incident to the smallest panchromatic subspace. An example of the counting argument is seen in figure 2.2.4.

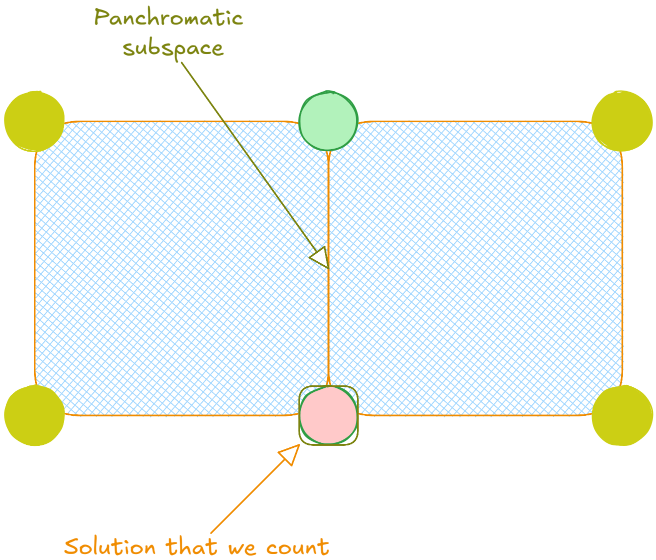


Figure 2.2.4: Panchromatic edge counting example

Trivially *3D-CountingSS* can be reduced to *3D-StrongSperner* even if we are over-counting the cubes. To show hardness, we make the following theorem:

Theorem 2.2.1

$$\mathbf{\#PPAD}(\textit{2D-SourceOrExcess}(2,1)) \subseteq \mathbf{\#PPAD}(\textit{3D-CountingSS})$$

Proof of the theorem Our first reduction is transforming our $2D$ grid, into a $3D$ with grid points:

$$B_n^3 = \left\{\mathbf{p} \in \mathbb{N}_0^3 \mid \mathbf{p}_3 \leqslant 6, \mathbf{p}_{-3} \in [n]_0 \times [n]_0\right\}$$

We denote the transformation $\mathcal{F} : V_{2^k} \to B^3_{2^{4k+10}}$ such that

$$\forall \mathbf{p} \in B_n : \mathcal{F}(\begin{pmatrix} \mathbf{p}_1 \\ \mathbf{p}_2 \end{pmatrix}) \to \left\{ \begin{pmatrix} 6\mathbf{p}_1 + 6 \\ 6\mathbf{p}_2 + 6 \\ 0 \end{pmatrix}, \begin{pmatrix} 6\mathbf{p}_1 + 6 \\ 6\mathbf{p}_2 + 6 \\ 1 \end{pmatrix} \right\}$$

We can observe that the above is our *2D-SourceOrExcess*$(2, 1)$ but copied twice in the 3D space. We note of the terms $I^b(P_i), L^b(P_i), R^b(P_i), \mathbf{s}^b_{P_i}, \mathbf{e}^b_{P_i}$ that we defined in paragraph 2.1 and index b denotes the first and second copy. Lastly, we introduce m^b_{ij} to denote the "midpoint", or the point with degree $(2, 0)$ that meets path i and path j with b to denote the upper and lower face. If these paths P_i, P_j meet at m_{ij}, then we place their ends $\mathbf{e}_i, \mathbf{e}_j$ as seen in the figure 2.2.5.

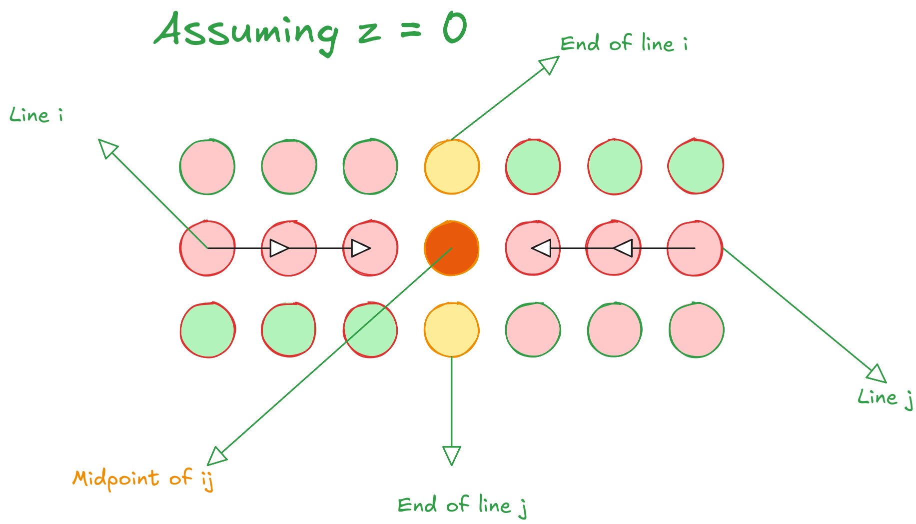


Figure 2.2.5: Double sink depiction from a 2D point of view

We now propose our colouring circuit $\mathcal{M}$:

$$\mathcal{M}(\mathbf{p}) \triangleq \begin{cases} (+, +, (-1)^b) & \text{if } \exists i, b : \mathbf{p} \in I^b(P_i) \\ (-, +, (-1)^b) & \text{if } \exists i, b : \mathbf{p} \in R^b(P_i) \\ (+, -, (-1)^b) & \text{if } \exists i, b : \mathbf{p} \in L^b(P_i) \\ (-, -, (-1)^b) & \text{if } \exists i, b : \mathbf{p} \in \{\mathbf{s}^b_{P_i}, \mathbf{e}^b_{P_j}\} \\ (-1)^b * (-, -, +) & \text{if } \exists i, j, b : \mathbf{p} = \mathbf{m}^b_{ij} \\ ((-1)^{\mathbb{1}(\mathbf{p}_1 > 0)}, (-1)^{\mathbb{1}(\mathbf{p}_2 > 0)}, (-1)^{\mathbb{1}(\mathbf{p}_3 > 0)}) & \text{otherwise} \end{cases}$$

An example of the above colouring can be seen in figure 2.2.6.

Correctness argument Using figure 2.2.7, assume that we have an edge $\mathbf{p}^t, \mathbf{p}^{t+1}$ of some path. We use red to represent the points on the left of the edge, blue for the points on the right, and brown for the surrounding. The red cube will not be panchromatic as x-coordinate is $+$ on all corners and similarly for the blue cube on the y-coordinate. In addition, we can observe that the brown cubes surrounding each face will also not result in a panchromatic cube as for the brown cube on the left of red will be $-$ on all y coordinates, with the same argument for blue. Moreover for the brown cubes on top, we can observe that z-coordinate will be $-$ on all points. Therefore, points in between lines or cycles will not result in solutions. As we can observe from figure 2.2.6, our construction results to discrete antipodal solutions.

For the double sink case, we utilise figure 2.2.8, to observe what happens around the double

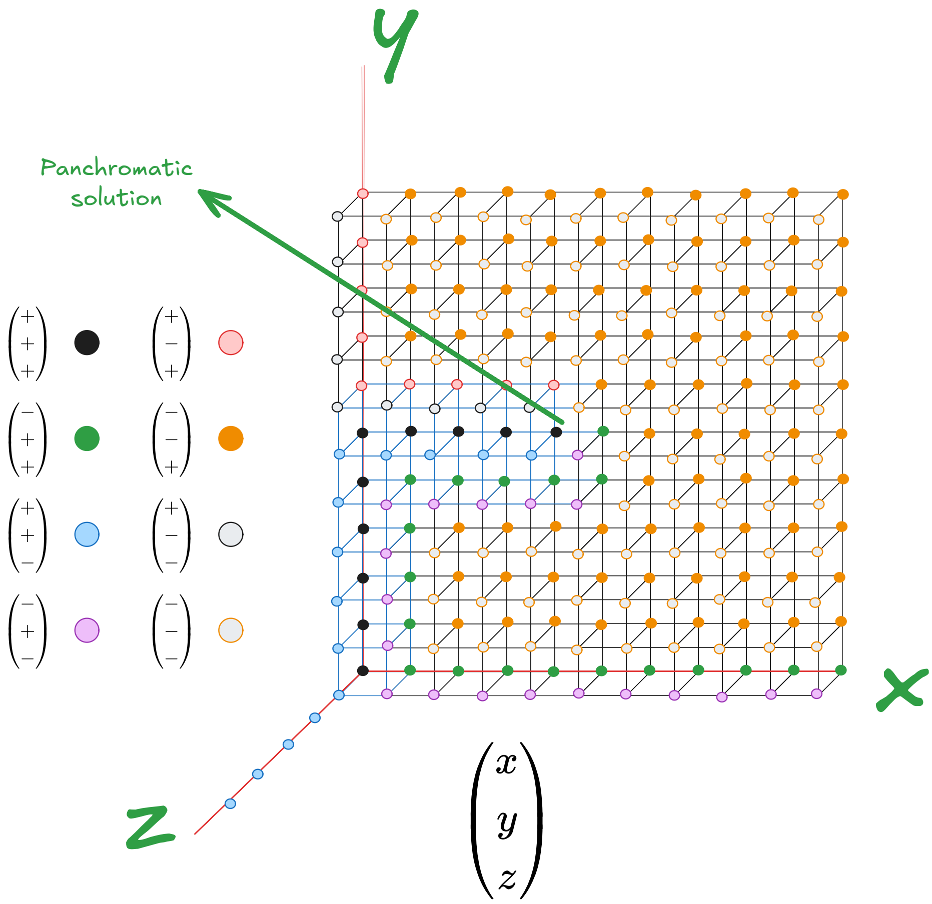


Figure 2.2.6: Colouring example based on the proposed circuit

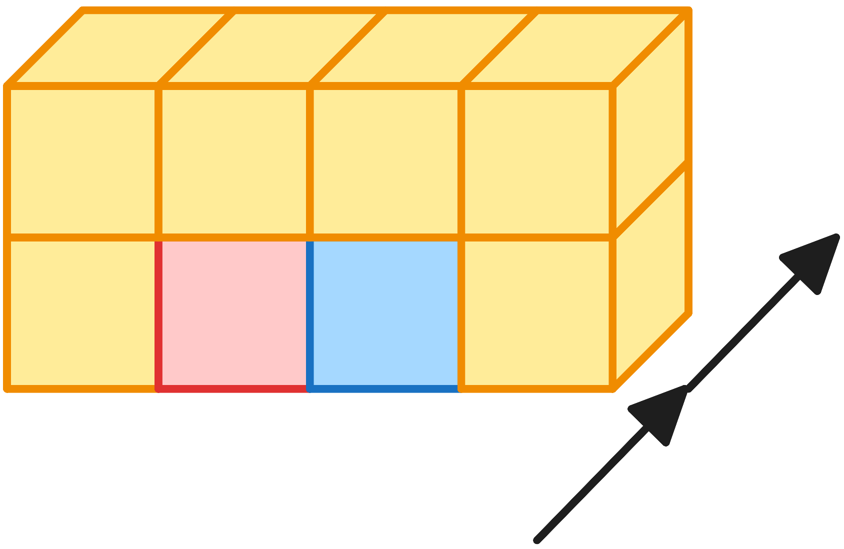


Figure 2.2.7: Example colouring where the arrow denotes the direction. In between the red and blue cubes in the bottom corners we have the positive line. The brown cubes around denote the negative labels

sink area. For points m_{ij}^0, m_{ij}^1, we can observe that their labelling is antipodal, meaning given our counting definition, all surrounding cubes which contain the side (m_{ij}^0, m_{ij}^1) count as one solution. We will refer to all the cubes incident to the edge as the combined panchromatic cube. We can manually observe that for each outer face of the combined panchromatic cube there will always exist a dimension whose colouring is not covered. For example, we can see that for all faces on top of the combined cube can never cover the z dimension. We can make similar arguments for the left and right outer faces. Since the outer faces do not yield a solution, that means we only have one solution, and therefore we can observe that the set of panchromatic solutions correspond to a 1-to-1 mapping with the original problem.

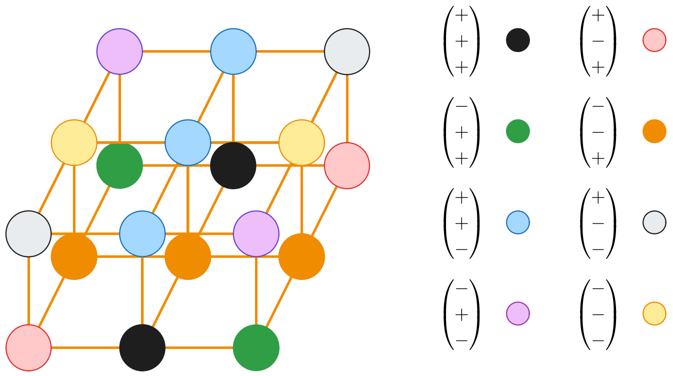


Figure 2.2.8: Colouring around the double sink points

2.2.3 Analysis of our results

We observed that in order to make the current reduction work, we had to remove excess nodes and focus only on double sinks. Moreover, we had to introduce antipodal edges in order to group solutions as one. As a result, the current method cannot be generalised directly to other *SourceOrExcess*$(k, 1)$ problems, unless we generalise the planar embedding technique, since we cannot reduce our transformations to only paths and cycles. In addition, given our snake embedding method, the cardinality of solution is not guaranteed to be preserved, since we dropped the discreteness requirement and the turning points may increase the number of solutions. On the contrary, our result raises questions as to the relation between the degree of a graph and its the dimension of its topological embeddings. We discuss this in greater detail in the future works section 4.2.

2.3 PPAD Counting Hierarchies

We want to extend the work done by Ikenmeyer et al. [33], in order to analyse the combinatorial nature of **PPAD** as a whole. Although the project did not manage to achieve the desired theorem, we hope to demonstrate relevant findings that can allow us to reveal more intricate properties of **#PPAD**. Our first step is to extend the hierarchy between *EndOfLine* and *SourceOrExcess*$(2, 1)$. We can observe that

$$\forall a, b \in \mathbb{N} : a \leqslant b \implies \textbf{\#PPAD}(\textit{SourceOrExcess})(a, 1) \subseteq \textbf{\#PPAD}(\textit{SourceOrExcess})(b, 1)$$

In fact, we can extend the degree of graphs to polynomial degrees, which we define as the *Unbalanced* problem 2.3.1.

Definition 2.3.1: *Unbalanced* problem definition

Input: We are given tuple $(\mathbf{S}, \mathbf{P}, f, g)$, such that $\mathbf{S} = \{S_i\}_{i \in [f(n)]}$ and $\mathbf{P} = \{P_i\}_{i \in [g(n)]}$ and $f, g : \mathbb{N} \to \mathbb{N}$ are poly-time computable polynomial functions. This describes a graph $G = (V, E)$ such that $V = \{0,1\}^n$ and

$$\forall (x,y) \in E, \exists i \in [f(n)], j \in [g(n)] : S_i(x) = P_j(y)$$

We enforce the condition that $\forall P_i : P_i(0^n) = 0^n$ and $\exists S_j : S_j(0^n) \neq 0^n$
Output: A node $v \in \{0,1\}^n \setminus \{0^n\}$ such that $\textit{in-deg}(v) \neq \textit{out-deg}(v)$.
If $f(\cdot) = p$ or $g(\cdot) = k$ for some $p, k \in \mathbb{N}$, we will denote the problem as $\textit{Unbalanced}(p,k)$

Lemma 2.3.1

Unbalanced is **PPAD**-complete [29]

We know by Hollender et al. [29] that *Unbalanced* is **PPAD**-complete. As mentioned in section 1.2.1, Hollender et al. [29] demonstrated how extending the number of known sources of the *EndOfLine* problem does not affect its **PPAD**-completeness. Moreover, combinatorially we can observe that if an *EndOfLine* problem has k known sources, it implies that there must exist at least k solutions. This raises relevant questions as to the relation of multi-source problems with higher degree graphs. We can observe that by lemma 2.3.2, the *Unbalanced* problem is not affected. The proof of our lemma is provided in appendix 5.6.

Lemma 2.3.2

$$\#\mathbf{PPAD}(\textit{PolyS-Unbalanced}) \subseteq \#\mathbf{PPAD}(\textit{Unbalanced})$$

For simpler instances, such as the *EndOfLine*, the reduction demonstrated by Hollender is non parsimonious, implying that there is a possibility of an oracle separation between *EndOfLine* and *2S-EoL*. In fact one can observe that we can create a 3D-matrix of problems such that

$$\forall i, j, k \in \mathbb{N} \cup \{\textit{Poly}\} : M_{i,j,k} = \textit{kS-Unbalanced(i,j)}$$

All the above problems are **PPAD**-complete since by lemma 2.3.2 and 2.3.1:

$$\forall i, j, k \in \mathbb{N} \cup \{\textit{Poly}\} : \textit{PolyS-EoL} \leqslant_L M_{i,j,k} \leqslant_L \textit{Unbalanced}$$

Moreover, these hierarchies seem to reveal some hidden combinatorial structures in **PPAD** and in **TFNP** as a whole. For example, one can demonstrate the following claim 2.3.1

Claim 2.3.1: SourceOrExcess with fixed degrees and sources

Given $\alpha, \beta \in \mathbb{N}$, the problem $\alpha\textit{S-SourceOrExcess}(\beta, 1)$ will have at least $\mu = \lceil \alpha/\beta \rceil$ solutions.

The proof our statement will be provided in appendix 5.6. This implies that there exists subclasses in **PPAD** which we can denote as $\mathbf{PPAD}(k)$ where we have **PPAD** problems with at least k solutions. Moreover we can generalise our questions to ask about combinatorial interpretations of $\mathbf{PPAD}(\alpha) - \beta$ for $\alpha \geqslant \beta$. We elaborate regarding potential directions in the future works section 4.2.

2.4 Supplementary Findings

In this section we would like to present an additional finding with respect to the parsimonious nature *PureCircuit* and **CLS** problems. We define the following subclass of *Tarski* 1.2.15:

Definition 2.4.1: *KleeneTarski* problem definition

Given $F : \mathbb{T}^n \to \mathbb{T}^n$, where F is a *natural* function 1.2.22, find $x \in \mathbb{T}^n$ such that $F(x) = x$. We assume F will be represented as a circuit that uses set of gates $\{\wedge, \vee, \neg, \mathbf{0}, \mathbf{1}\}$.

The above problem exploits the fact that F is *natural* since that implies that F is monotone with respect to the partially ordered set $(\mathbb{T}^n, \preceq)$. We illustrate how this results in a parsimonious reduction between *PureCircuit* and *KleeneTarski*.

Proposition 2.4.1: Parsimonious Reduction between *KleeneTarski* and *PureCircuit*

$$\#KleeneTarski \subseteq \#PureCircuit$$

Proof. Given an input circuit $C : \mathbb{T}^n \to \mathbb{T}^n$, construct *PureCircuit* instance:

1. Initiate a vector of n nodes, which we denote as s.
2. Construct the C using the standard set of gates and apply $C(x)$ to create output vector o.
3. Copy the results back into s.

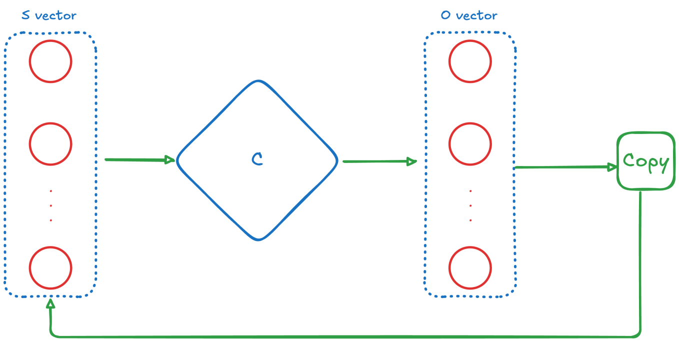


Figure 2.4.1: KleeneTarski construction. Figure from interim report.

We can visualise the above construction in figure 2.4.1. We can observe that, for any valid assignment $\mathbf{x} \implies \mathbf{x}[s] = \mathbf{x}[o]$, which implies $\mathbf{x}[o] = F(x) = \mathbf{x}[s]$. □

The above problem demonstrates how we can use *PureCircuit* to find fixed points in *Tarski*. We expand how we can relate *PureCircuit* with ATMs in our future works section 4.2.1[1].

[1]The above proposition and proof has been re-used from the interim report.

Chapter 3

Software Deliverable

In the current section, we give an extensive overview of the software tool we developed to help with our theory crafting. Our main requirements consist of: the ability to construct *PureCircuit* gadgets, verify correctness and find solutions. We split this chapter giving an overview on the tooling we used, the architecture of the software and the algorithms we deployed for solutions finding.

3.1 Tooling Selection

3.1.1 Rust Programming Language

`Rust`[1] is a systems-level compiled programming language that can achieve high speeds due to its low-level compilation, while maintaining memory safety. Moreover, it offers high level features such as its strong algebraic and trait type system which allows us to represent complex states.

3.1.2 Bevy and the ECS architecture

`Bevy` [2] is a `Rust` library, primarily used for game development. It uses the Entity-Component-System architecture where entities with varying components exist in world state. Each entity interacts with each other based on systems. A visual reference is provided in the figure 3.1.1.

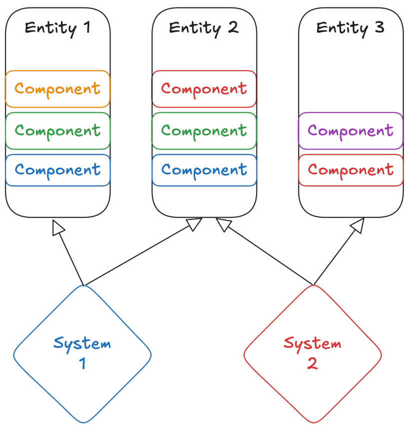


Figure 3.1.1: ECS visualisation implementation. Figure from interim report.

The current tool comes with several benefits. It offers flexibility when it comes to drawing and interacting with shapes on a screen. We wanted to give our tool and editor-like aesthetic

[1]https://www.rust-lang.org/
[2]https://bevy.org/

where the user can navigate and interact with a *PureCircuit* environment. Moreover, we are able to decouple the logic from entities to their individual components and express more intricate relations. Regarding the current application, we chose to represent the graph nodes as separate entities which interact with each other based on their edges. The above choice allow us to build upon a software which is prone to changes.

3.1.3 Software Testing methodology

The core philosophy of our testing is Test-Driven-Development. This methodology was used to develop and validate our critical parts of our code, such as graph validation or solution enumeration. Its main philosophy is to first write the unit tests, and then develop the code to resolve them. This allows us for the code to be self-documenting as well as ensuring that we are acting within the expected scope. We provide a sample of our unit test results in figure 3.1.2. A full log of our test results will be provided in our supplementary files.

Figure 3.1.2: Sample Unit Testcases.

Another testing technique that we heavily took advantage of is *Property-based Testing*, which we implement using the `Proptest` library [3]. This library was inspired by the `Haskell` library `QuickCheck` [13], where we test the properties of a pure function across a range of inputs. The notion is described as follows, the more complex the input of the function, the greater is the input space and therefore harder and cumbersome to check all possible combinations or edge-cases. With `Proptest`, we can stochastically and strategically check whether a property of our function is maintained across most inputs, such as a sorted output, or a validating assignment.

For our purposes, we use value-based property testing in which: We define a set of types where in each one we define a *generator* and a *shrinking strategy*. The *generator* is responsible for generating significant values that represent the specific type. The *shrinking strategy* is responsible for finding the smallest case in which our case fails. We then define test cases that automatically run the generator and shrinker procedures until we have proved that our function passes all the generated cases, or finds the smallest test case it fails on.

3.2 Program Visualisation Summary

We use the figure 3.2.1 as reference of our application UI. Our application contains a UI legend and a drawing board. The UI legend provides the user with the information about the current state of the app. Moreover, we provide the user information with the key bindings that we assigned, as well as parameters and buttons to execute our solution finding algorithms. We will dive deeper into the functionalities in the upcoming sections.

[3]https://github.com/proptest-rs/proptest

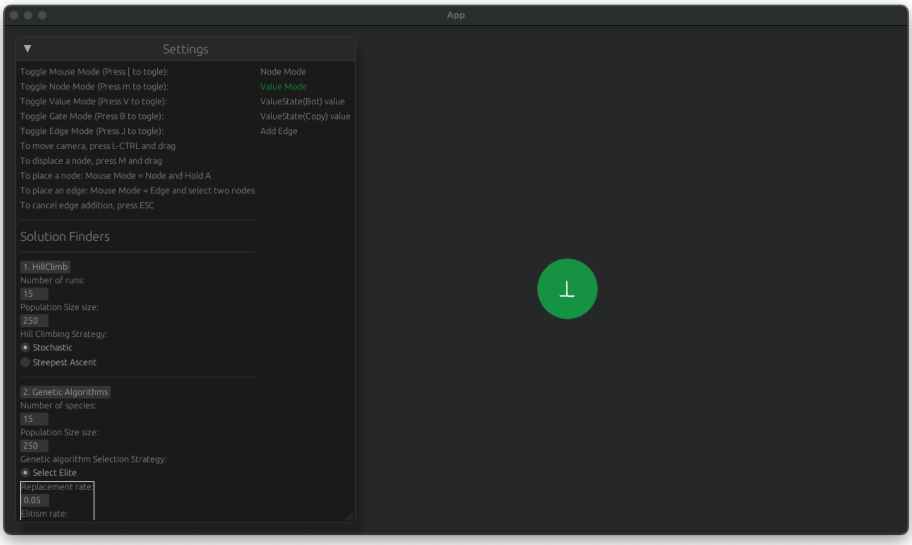


Figure 3.2.1: Application Layout.

Our application is composed of 5 different states which are summarised in the table 3.2.1. Using the aforementioned states, we are able to add/remove nodes/edges as well as move them around or change their values. We added visual indicators so if a gate has an incorrect arity or an incorrect assignment, it will be highlighted. Moreover, we attached numbers to our edges so the user we will be able to differentiate between inputs and outputs. This is mostly useful for the PURIFY gate as each output behaves differently. In our appendix 5.6 we attach user stories that explain in greater detail how one can perform the above operations.

State	Functionality
Mouse Mode	Determines whether we want to work with edges or nodes
Node Mode	Determines whether we want to add a value node or a gate node
Value Mode	The type of value node we want to add
Gate Mode	The type of gate node we want to add
Edge Mode	Determines whether we want to add an edge or delete an edge

Table 3.2.1: Application states.

3.2.1 Solution Selection

For the solution selection we opted with meta-heuristic algorithms. Since these problems are conventionally hard to find solutions especially as the number of nodes increase, we focused on the notion of approximation algorithms in order to find mostly satisfying assignments in large graphs or gadgets. With the help of the `genetic_algorithm`[4] library, we use the *Hill Climbing algorithm* and *Genetic algorithm* in order to find solutions. We will give a brief explanation on how they work in the upcoming sections but in general we express our graph as a chromosome of value nodes. We then formulate our optimisation function as the number of unsatisfied gates given an assignment. The goal of either algorithm is to minimise the number of errors. Once an assignment is found, we pass the solutions back into our graph and display them.

Hill Climbing Algorithm

The hill climbing algorithm can be described by starting from some base solution and repeatedly improving our assignment. We allow for 2 strategies, one of them strictly always chooses the best

[4] https://crates.io/crates/genetic_algorithm

neighbour whereas one allows for stochastic choice. The above operation is repeated until we either find a satisfying assignment or the maximum threshold of simulations has been reached.

Overall this algorithm, one of the simplest and faster algorithms to run and can work well for convex problems as explained in [47]. It may significantly struggle against large graphs or instances that multiple changes need to occur in order to reach an optimal point. For the purposes of the current project, we allow for multiple re-runs in order to maximise our odds of finding a solution.

We provide a list of parameters as shown in figure 3.2.2. The number of runs determines how many times we can re-run the algorithm until an optimal solution is found. Population size determines the number of assignments to work with in parallel. The strategies are the ones that we described previously.

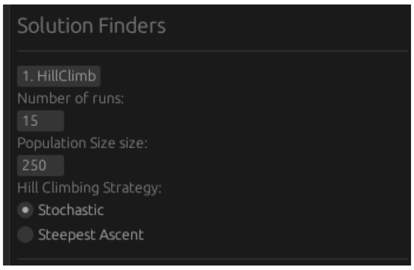


Figure 3.2.2: Hill Climbing parameters.

Genetic Algorithm

Genetic algorithms are based of the idea of evolution. They were introduced by Holland [28], and has found many usages throughout computer science, particularly in optimisation problems such as *TSP* problems [41] or *Neural Networks* [50, 49].

Using figure 3.2.4 as reference, we start with a population of chromosomes. We define a function that evaluates how good a specific chromosome is, also known as its *fitness function*. We then apply the *selection method* which picks the parents for the next generation based on the performance of the chromosomes. The *crossover method* determines how to combine the genes of two parents in order to create the offspring. And lastly we have the *mutation step*, where we randomly mutate the genes of the new population. We can repeat the above series of steps until we reach the target value or until we pass the maximum time-step limit. In our application, using figure 3.2.3, we use refer to the following list as a summary of parameters:

1. **Number of species**: We allow for our algorithm to evolve several populations independently and then merge each population at the end to find the optimal solution. Increase the odds of avoiding local minima and ensures diversity.
2. **Population size**: Number of chromosomes per species.
3. **Selection step**: We allow for two selection methods: *Tournament based* and *Elitism selection*. *Elitism selection* selects the best performing chromosomes whereas *Tournament based* selection creates random sets tournaments across our chromosomes and picks the best ones from each. The tournament method is repeated until we recreate the population size. Out of the two methods, elitism is the faster method but can result in less diversity.
4. **Crossover step**: We allow for two main types of crossover methods:
   (a) *Crossover Uniform*: Each gene of the child is randomly selected from either parent. This allows for diverse solutions to be explored. Can reduce diversity but performs well on large population sizes [36].

(b) *Crossover Multi-point*: Using figure 3.2.5 as reference, we select crossover points that determine cut-off points from each parent. For each even sequence we use the sequence of the other parent. This method can be useful if consecutive pairs of genes are needed to improve the fitness.

5. **Mutation step**: Given the offspring of the **Crossover step**, we stochastically modify each gene in order to introduce diversity, and escape local optima [36].

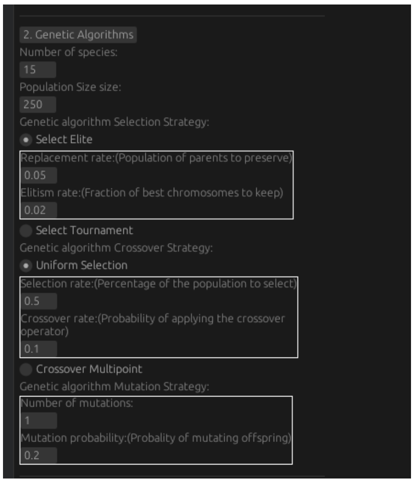


Figure 3.2.3: Genetic Algorithm parameters.

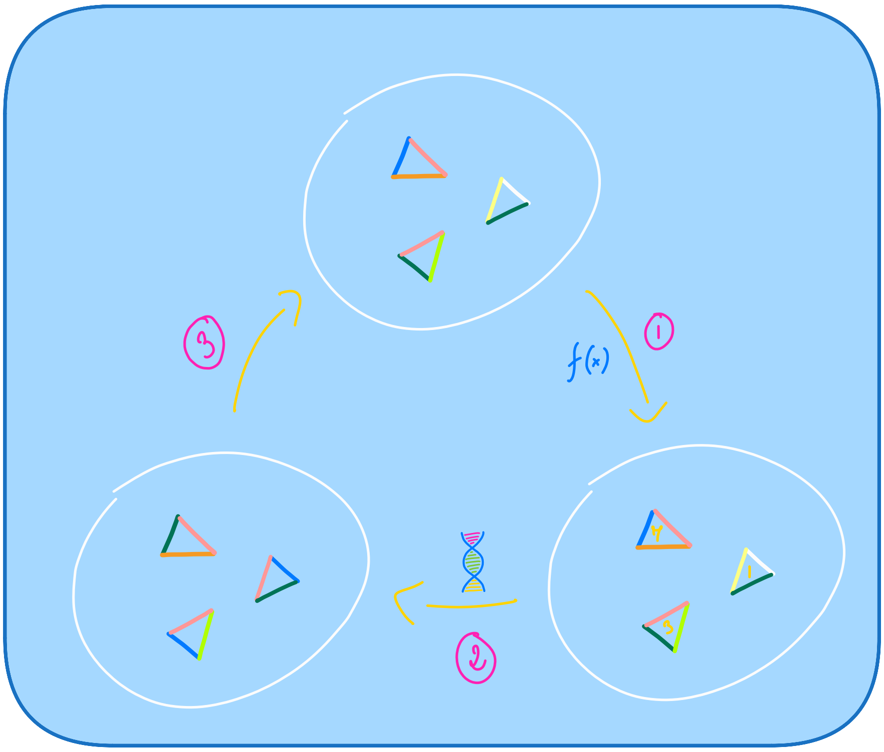


Figure 3.2.4: Visual representation of the genetic algorithm cycle. Image source [52].

Algorithm comparison

Both algorithms are capable of finding an assignment. The *Hill Climbing* is much cheaper to run but more prone to be stuck at local minima [6]. The *Genetic algorithm* is much more

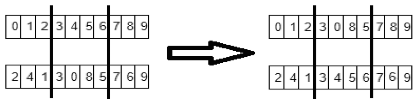


Figure 3.2.5: Crossover with multiple cut-off points. Image source [36].

computationally expensive out of the two but can be fine-tuned sufficiently to almost always find a solution. Further analysis and deeper comparison of the two algorithms is beyond the scope of the current project.

3.2.2 Solution Enumeration

To enumerate through solutions, we employed a traditional backtracking algorithm. Our algorithm is similar to the DPLL algorithm [19] used for *SAT* solving, where we recursively assign values to our graph. If all the nodes have been assigned a value, then we return the current assignment. If at a specific node, there are no more values that we can assign then we reject the current solution. To reduce the computational burden, we employ greedy techniques by eliminating invalid assignments. We summarise our algorithm in 3.2.1 and break it down into the following components: **Backtrack Initialisation**, **Unit Propagation operation**, **Set Simplification operation**, and **Backtrack Operation**.

Algorithm 3.2.1 PureCircuit Backtracking Algorithm Enumerator

```
function BacktrackInstatiation(pc : PureCircuitInstance)
    V ← PCExtract(pc)                         ▷ Extract value nodes from the PureCircuit instance
    S ← Arr(|V|, KleeneSetInit({0, ⊥, 1}))    ▷ Create a state vector that contains the KleeneSet data-structure.
    Q ← PriorityQueueInit(∅)
    for el ∈ V do
        pq_key ← KeyStruct(3, el.is_source(), el.num_neigh())
        PushPQ(Q, pq_key, el.index)
    return S, Q
function BacktrackPureCircuit(pc :PureCircuitInstance)
    S, Q ← BacktrackInstatiation(pc)
    A ← Arr(|V|, ∅)
    r ← BacktrackIteration(pc, A, S, Q)
    return r
function BacktrackIteration(
     pc :PureCircuitInstance,
     A :Arr[KleeneValue],
     S :Arr[KleeneSet],
     Q :PriorityQueue
)
    if IsEmpty(Q) then ▷ If all nodes have been assigned a value, that means we found a solution and therefore we return the current assignment.
        return [A]
    min_graph_ind ← ExtractMin(Q)
    r ← []
    for v ∈ S[m] do                           ▷ Select each value in the set of possible values of the selected node
        S′ ← Clone(S)
        Q′ ← Clone(Q)
        A′ ← Clone(A)
        result ←UnitPropagate(pc, min_graph_ind, v, A′, S′, Q′) ▷ For each assignment in each solution set, clone our states and assign the value selected
```

```
            if result = ⊥ then ▷ If our algorithm return false, it means that the current assignment is invalid and we ignore it
                Continue
            r′ ←BacktrackIteration(pc, A′, S′, Q′)
            r ← ConcatArray(r, r′)
        return r
function UnitPropagation(
    pc :PureCircuitInstance,
    nod_ind :GraphIndex,
    nod_val :KleeneValue,
    A :Arr[KleeneValue],
    S :Arr[KleeneSet],
    Q :PriorityQueue
)
    AssignNode(nod_ind, nod_val, A, S, Q) ▷ Assign value the selected node. Propagate the changes to the solution and assignment set
    PropagateNode(pc, nod_ind, nod_val, A, S, Q)
    while ¬IsEmpty(Q)∧ PeekKey(Q).set_length = 1 do ▷ While the queue contains an element whose value can take only one element, assign its value
        ind ← ExtractMin(Q)
        val ← S[ind]
        AssignNode(ind, val, A, S, Q)
        PropagateNode(pc, nod_ind, nod_val, A, S, Q)
    if IsEmpty(Q) then                                          ▷ If every element is assigned, accept
        return ⊤
    if PeekKey(Q).set_length = 0 then ▷ If the queue contains an element which does not have a satisfying assignment, discard the current assignment
        return ⊥
    return ⊤
function PropagateNode(
    pc :PureCircuitInstance,
    nod_ind :GraphIndex,
    A :Arr[Kleene],
    S :Arr[KleeneSet],
    Q :PriorityQueue
)
    for g ∈ GetAllGateNeighbours(pc, nod_ind) do
        I, O ← GetValueSets(g, A)        ▷ Get the current assignment of the input/output values of g
        (I′, O′) ← SetSimplification(g, I, O) ▷ Determine the available value sets for each input output element given the elements that have been already assigned
        PropagateSetValues(Q, S, I′, I) ▷ Update the value sets of the unassigned values for both input/output gates
        PropagateSetValues(Q, S, O′, O)
```

Backtrack Initialisation We initialise two main structures: a min-based priority queue Q, and a solution map S for each node. S the solution map contains the *KleeneSet* data structure which is the set of values a node can take. We assume that all nodes can take $\mathbb{T}$ in the beginning.

The priority queue will essentially dictate which nodes to select and evaluate. We want to reduce the recursive calls as much as possible and eliminate any partial assignments that lead to unsatisfying solutions. The queue stores the node references of the value nodes and uses as keys the structure KeyStruct(e). The KeyStruct is a special structure that orders elements lexicographically based on the following order:

1. **In Ascending**: The number of possible value elements the current node can take
2. **In Descending**: Whether the node has an *in-degree* of 0
3. **In Descending**: The total number of neighbours

We argue that the above strategy minimises the breadth of the recursive calls. Moreover, for each node with *in-degree* of 0, we know that for each value of their assignment, a solution should exist. The heuristical argument for this can be observed in Deligkas et al. [21], where their construction for finding solutions allows for these source nodes to be independent. Lastly, prioritising nodes with the highest neighbour count implies that we propagate changes to a greater number gates and therefore maximise the influence of the *Unit Propagation* procedure.

Unit Propagation operation The algorithm assigns the selected value to a node by modifying their solution set and the value set. Then for each gate that is incident to the node apply the *Set simplification operation*. While the top element of the queue has one possible solution, extract the solution and repeat the *unit value propagation operation* until the queue is empty or the cardinality of the value set at the head of the priority queue is different from 1. If it's zero then current assignment is not valid, and we return false, otherwise we return true.

Set Simplification operation Given a gate, we have a set of possible values for each of our nodes, as determined by our value set. We want to reduce the set possible values by following the rules of the gate. For example, assume we have an **AND** gate. If the output value is assigned the value 1, then we know that the inputs can only take values $(1, 1)$, and therefore we restrict the values of the input sets to $\{1\}, \{1\}$. If the current solution set of a node is v_t, then its updated set is $v_{t+1} \leftarrow v_t \cap \omega$, where ω are the set of values denoted by the simplification operation. We provide the rules for **AND**, **OR** and **Purify** in the tables 3.2.2, 3.2.3 and 3.2.4. The rules for the other gates will be provided in the appendix 5.6. If a value is not assigned, we denote it as $\cdot$.

AND input	Output sets
$\forall v \in \mathbb{T} : (\cdot, \cdot, v)$	$(\mathbb{T}_{\geqslant v}, \mathbb{T}_{\geqslant v}, \{v\})$
$\forall v \in \mathbb{T} : (\cdot, v, \cdot)$	$(\mathbb{T}, \{v\}, \mathbb{T}_{\leqslant v})$
$\forall a, b \in \mathbb{T} : a < b \implies (\cdot, a, b)$	Invalid combination
$\forall v \in \mathbb{T} : (\cdot, v, v)$	$(\mathbb{T}_{\geqslant v}, \{v\}, \{v\})$
$\forall a, b \in \mathbb{T} : a > b \implies (\cdot, a, b)$	$(\{b\}, \{a\}, \{b\})$
$\forall a, b \in \mathbb{T} : (a, b, \cdot)$	$(\{a\}, \{b\}, \{\min(a, b)\})$
$\forall a, b, c \in \mathbb{T} : (a, b, c)$	$(\{a\}, \{b\}, \{c\})$, only if the triplet (a, b, c) is a correct **AND** assignment

Table 3.2.2: **AND** value propagation rules.

OR input	Output sets
$\forall v \in \mathbb{T} : (\cdot, \cdot, v)$	$(\mathbb{T}_{\leqslant v}, \mathbb{T}_{\leqslant v}, \{v\})$
$\forall v \in \mathbb{T} : (\cdot, v, \cdot)$	$(\mathbb{T}, \{v\}, \mathbb{T}_{\geqslant v})$
$\forall a, b \in \mathbb{T} : a < b \implies (\cdot, a, b)$	$(\{b\}, \{a\}, \{b\})$
$\forall v \in \mathbb{T} : (\cdot, v, v)$	$(\mathbb{T}_{\leqslant v}, \{v\}, \{v\})$
$\forall a, b \in \mathbb{T} : a > b \implies (\cdot, a, b)$	Invalid combination
$\forall a, b \in \mathbb{T} : (a, b, \cdot)$	$(\{a\}, \{b\}, \{\max(a, b)\})$
$\forall a, b, c \in \mathbb{T} : (a, b, c)$	$(\{a\}, \{b\}, \{c\})$, only if the triplet (a, b, c) is a correct **OR** assignment

Table 3.2.3: **OR** value propagation rules.

Purify input	Output sets
$\forall b \in \mathbb{B} : (b,\cdot,\cdot) \vee (b,\cdot,b) \vee (b,b,\cdot)$	$(\{b\},\{b\},\{b\})$
$(\perp,\cdot,\cdot)$	$(\{\perp\},\mathbb{T}_{\leqslant\perp},\mathbb{T}_{\geqslant\perp})$
$(\cdot,\cdot,0)$	$(\{0\},\{0\},\{0\})$
$(\cdot,\cdot,1)$	$(\mathbb{T}_{\geqslant\perp},\mathbb{T},\{1\})$
$(\cdot,\cdot,\perp)$	$(\{\perp\},\{0\},\{\perp\})$
$(\cdot,0,\cdot)$	$(\mathbb{T}_{\leqslant\perp},\{0\},\mathbb{T})$
$(\cdot,1,\cdot)$	$(\{1\},\{1\},\{1\})$
$(\cdot,\perp,\cdot)$	$(\{\perp\},\{\perp\},\{1\})$
$(\perp,0,\cdot)$	$(\{\perp\},\{0\},\mathbb{T}_{\geqslant\perp})$
$(\perp,\cdot,\perp)$	$(\{\perp\},\{0\},\{\perp\})$
$(\perp,\cdot,1)$	$(\{\perp\},\mathbb{T}_{\leqslant\perp},\{1\})$
$(\perp,\perp,\cdot)$	$(\{\perp\},\{\perp\},\{1\})$
$\forall a,b,c \in \mathbb{T} : (a,b,c)$	$(\{a\},\{b\},\{c\})$, only if the triplet (a,b,c) is a correct **PURIFY** assignment
–	Any unlisted combinations are considered invalid

Table 3.2.4: **Purify** value propagation rules.

Backtrack Operation The *backtrack* operation is summarised in 3 key steps. If the priority queue is empty then we found a satisfying assignment. Otherwise, pick a node from the priority queue. For each value in its set, create a copy of the current queue, current solution assignment and value set state and apply the *Unit value propagation* operation. After that create a recursive call apply the *backtrack operation* again until the current recursion chain terminates with a solution, or it rejects. At the root of every execution call, merge the solution assignments in each level and return the sets. The root call will return a set of every possible valid assignment set.

Correctness and Running Time analysis

For the running time analysis, we can observe that each time we assign a node, the unit propagation operation, restricts the solution set of the value nodes that are incident by one gate. Even in the worst case scenario where no reduction occurs, we can clearly see that each time, our priority queue will select a node and will recurse through each of its possible values, creating a call tree with depth of at most n. Therefore, we can argue that the running time of the algorithm is $O(3^n)$.

As for correctness, the algorithm essentially picks nodes in a certain order and selects a value that we believe that will give a satisfying assignment. We only have to look at what happens at the *unit propagation* stage. We can observe that in each stage, we are essentially limiting the set of solutions to only the ones that will give a satisfying assignment. By taking advantage of the localised changes of each gate, we are fast-forwarding the backtracking calls. Values that we discard are nodes that guaranteed to give unsatisfying assignments, and therefore we are not discarding any solutions. This implies that our resulting array of assignments contains only valid solutions, and this array covers the maximal solution set.

We provide a table in 3.2.5 with respect to the number of backtracking calls. The smaller the number the smaller the recursion tree and therefore fewer values to check. We can observe that the premature cancellations and the value set reductions allow us to keep a low number of calls despite the graph size. We will provide a list of the solutions in the appendix 5.6. The above algorithm does present with several limitations which we highlight as future extension in the section 4.2.

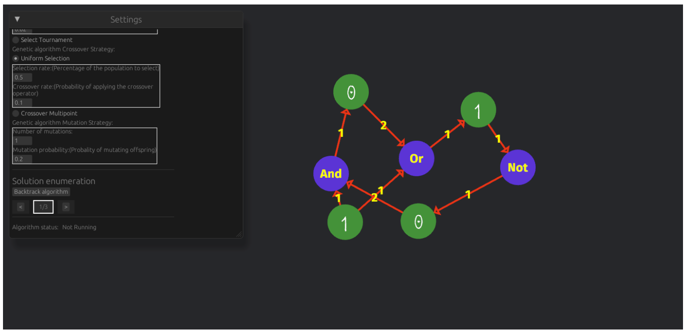


Figure 3.2.6: Small Circuit example.

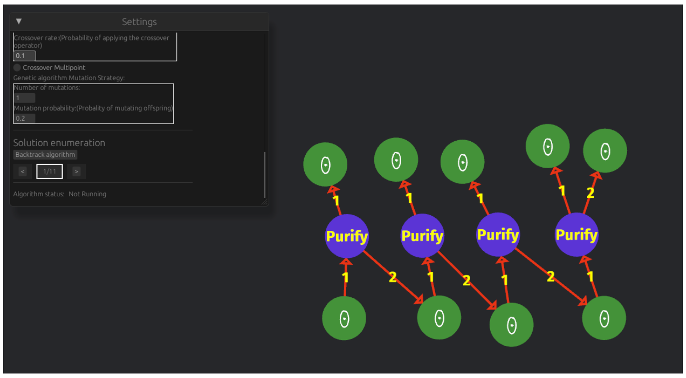


Figure 3.2.7: Bit Generator circuit.

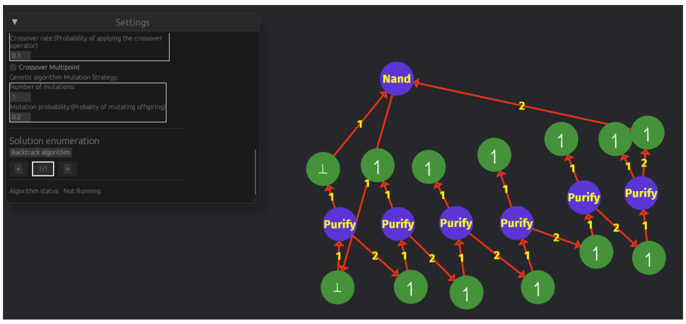


Figure 3.2.8: Complex Circuit.

Circuit types	Number of nodes	Number of backtrack calls
Simple circuit 3.2.6	4	2
Medium circuit 3.2.7	9	9
Complex circuit 3.2.8	13	3

Table 3.2.5: Number of Backtrack calls

Chapter 4

Chapter Conclusions and Future Directions

4.1 Conclusions

In summary, we expanded upon the combinatorial nature of **PPAD**, under the lens of topological representations and the *PureCircuit* problem. First we showed how *Kleene logic* can be embedded into **PPAD** using as a primary example the *StrongSperner* problem and how certain problems can retain their **PPAD**-completeness. Moreover, we demonstrated that under hazard-free conditions and discreteness, how one can parsimoniously reduce *HF-DiscreteStrongSperner* to *PureCricuit*. We were also able to construct a parsimonious reduction from the *EndOfLine* problem to *StrongSperner* instances with discrete and antipodal properties. Given that, we shortened the gap by demonstrating how the parsimonious reduction chain to *PureCircuit* from the *EndOfLine* depends on the hazard free nature of the *AntipodalStrongSperner*.

Extending our work on topological interpretations, we showed first hand how graph problems such as the *SourceOrExcess*$(2, 1)$ can also be encoded topologically and demonstrated how the dimensionality of the space we encode it, can affect the counting hardness of problems. We argued that this finding is significant since we found another **PPAD**-complete problem with no clear combinatorial interpretation. More interestingly, we conjectured the possibility of a connection between the degrees of graph and the dimensionality of its topological embedding.

Moreover, we investigated the different counting classes that can be represented in **PPAD** by expanding on Hollender et al.'s work [29]. Specifically we built upon their combinatorial argument by relating *SourceOrExcess* problems and argued about the potential infinite hierarchy of **PPAD** problems with respect to their minimum number of solutions. In addition, we argued the importance of the *Unbalanced* problem as an important stepping stone to a total *PureCircuit* Cook-Levin theorem, due to its overall relation with all the other unbalanced digraph problems.

Lastly, we showcased our work on our *PureCircuit* visualisation software. We demonstrated its core functionalities such as *PureCircuit* editor tool with assignment verification. Moreover, we implemented several meta-heuristic algorithms for assignment finding assignments in large graphs, such as *Hill Climbing* and *Genetic Algorithms*. Most importantly, we also implemented a specialised backtracking algorithm which takes advantage of the structure of the graph. Despite its high worst-case running time, we manage to use several greedy methods to cut down breadth and depth of the tree making it ideal for verification of small circuits.

4.2 Future Work

4.2.1 Theory Directions

HF-PPAD in PPAD

In our section 2.1, made references to how Kleene logic can be incorporated with **PPAD** problems. For the *nD-StrongSperner* problems we observed that *Kleene logic* provides a natural extension and is equivalent to its non hazard counterpart. For other problems such as *StrongSperner*, we observed that this is not clear by current complexity assumptions and indeed proving **PPAD**-hardness would be essential for establishing the counting hardness of *PureCircuit*. But one could ask further questions such as, could we define complexity classes such as **HF-PPAD** and could we show that **HF-PPAD** = **PPAD**. Along similar line, we can also ask the questions as to existence of problems *HF-nD-Counting-SS* as previously stated and whether there exists a parsimonious reduction to its non-hazard-free variant.

Even more naturally, one can also ask, how does hazard-freeness affect **TFNP** as a whole? We conjecture that one can make similar observations to the *StrongTucker* problem [2], in order to create similar classes such as **HF-PPA** as it also uses a very similar colouring argument as the *StrongSperner* argument. Moreover, we were able to observe how we can use Kleene logic with **PLS** sub-problems like *Tarski*. Naturally, can we define the class of problems **HF-TFNP**, and would it be equivalent to its non-hazard-free variant? We believe that this shift in perspective may also give insights to other areas such as the circuit complexity of *Hazard-Free circuits*.

Parsimonious Topological Representations of PPAD problems

We summarise our conclusion from the section 2.2, by conjecturing the existence of a parsimonious reduction from an *nD-Counting-SS* to a *Counting-SS* problem as it would demonstrate an important stepping stone towards its parsimonious reduction to the *PureCircuit* problem. Moreover, given our **PPAD** hierarchies, we can create the following conjecture which combines the graph representations with the topological ones 4.2.1

Conjecture 4.2.1: *nD-CountingSS* and *Unbalanced*

There exist some polynomial computable function $f : \mathbb{N}^3 \to \mathbb{N}$, such that given $\alpha, \Delta_i, \Delta_o \in \mathbb{N}$ and $m = f(\alpha, \Delta_i, \Delta_o)$

$$\textbf{\#PPAD}(\alpha\text{-}SUnbalanced(\Delta_i, \Delta_o)) \subseteq \textbf{\#PPAD}(mD\text{-}CountingSS)$$

What the above conjecture essentially describes is, given a number of known sources and the maximum number of input/output vertices of graph, can we create some *nD-CountingSS* that is parsimonious to its graphical representation? Since we know that the problem *Unbalanced* can parsimoniously count every graph variant, can we also reduce *Unbalanced* to a topological problem? A reduction like that would be an important milestone for the usage topological *PureCircuit* constructions for enumerating solutions.

PureCircuit Gate Extension

The proof for the **PPAD**-inclusion can be summarised as expressing a *PureCircuit* instance as a *Brouwer* function from $[0,1]^n \to [0,1]^n$, where each input represents a continuous assignment to the circuit nodes, and each output $F_i(x)$ is determined by a continuous function of the gate incident to node i and its neighbouring nodes [21]. Given its fixed point $x^\star$ then we can denote the following assignment: if $x_i^\star \in \{0,1\}$ then $v_i = x_i^\star$, otherwise $v_i = \bot$. For example, the Purify gate can be represented as:

$$\text{Purify}(v) = \begin{pmatrix} \max\{2v_1 - 1, 0\} \\ \min\{2v_2, 1\} \end{pmatrix}$$

Deligkas et al. [21] showed that as long as our gate set consists of the Purify gate and NOR or NAND or any combination of gates that are functionally complete with respect to boolean logic, *PureCircuit* is **PPAD**-complete. It implies that we can extend the gate set of *PureCircuit* to any gate that exhibits continuity behaviour under $[0,1]$ since we could always use the *Brouwer* problem to find a solution. We argue that there may exist gates that may have a more intricate combinatorial nature and therefore assist in the resolution of our conjectures.

PureCircuit with ATMs

In our supplementary finding section 2.4, we showed how to relate variants of the *Tarski* problem with *PureCircuit*. We also know that based on *Tarski*'s theorem, we can always find a labelling function on ATMs which demonstrates whether our starting configuration is accepting, rejecting or non-halting. This interesting relation may provide a pathway towards the full Cook-levin type theorem that we seek. But one also has to acknowledge the difficulty of expressing operator functions such as τ, as it seeks to find the maximal element in the poset of functions of $\mathcal{C} \to \mathbb{T}$, whereas our reduction was based on the poset of the information ordering of Kleene logic. We leave this as integral evidence for the potential relation between *PureCircuit* and ATMs.

4.2.2 Software future Directions

Given the algorithms that we proposed for our software, we argue that there are several improvements that can be made. For all three of the algorithmic methods we mentioned, the most important extension is to treat weakly directly connected components individually. Trivially an assignment in one weakly connected component should not affect the solution set of the other one. We can therefore reduce the search spaces of our problems by finding the solution sets in each graph separately. Moreover, for the *backtracking* algorithm, we can extend the above notion by considering the Cartesian product of the solution sets, meaning:

$$\mathbf{sol}(G) = \bigtimes_{i \in [k]} \mathbf{sol}(G_i)$$

Where G_i is the ith connected component.

We can also optimise the backtracking algorithm even further. First one can observe that if our graph was acyclic, then the permutation of the nodes with no *in-degree* solutions have a significant influence to the downstream nodes. With this in mind, we can apply the following sequence of steps to make the backtracking algorithm simpler:

1. Identify the strongly connected components (SCCs) of the circuit, using polynomial time algorithms such as Kosaraju's Algorithm [48] or Gabow's Algorithm [27].
2. Treating SCCs as an individual node, identify nodes by their distances from base nodes (nodes with no incoming edges)
3. Extend the backtracking algorithm so instead of using the start label, use the distance from the base instead. The greater distance a node has from the base, the less impact it has and therefore should be postponed in later stages of the backtracking calls.

We argue that the above heuristic may result in improved average times but of course further research can be done in order to determine more suitable selection heuristics for our priority queue.

Finally, we highlight potential improvements to our meta-heuristic algorithms. The fitness function could be modified to take advantage of the continuity properties of the *PureCircuit*, and each algorithm can be improved using more sophisticated techniques. More broadly, one could explore additional meta-heuristic approaches beyond those considered here, as shown in [1], though such extensions lie beyond the scope of the project.

Chapter 5

Project Management

5.1 Project management methodology

Due to the hybrid nature of our project, we wanted to ensure that we are able to manage both the software portion and the theory portion of the project effectively. With respect to our software management, we primarily focused on a hybrid approach with the help of a Kanban board, since we expected the core set of requirements to change over time as we developed our theories and ideas. An example of our Kanban board can be seen in figure 5.1.1, and the idea is that we split our requirements into categories. Each category represents the state of a task, and it is a common tool to use for tracking workload or the state of the project.

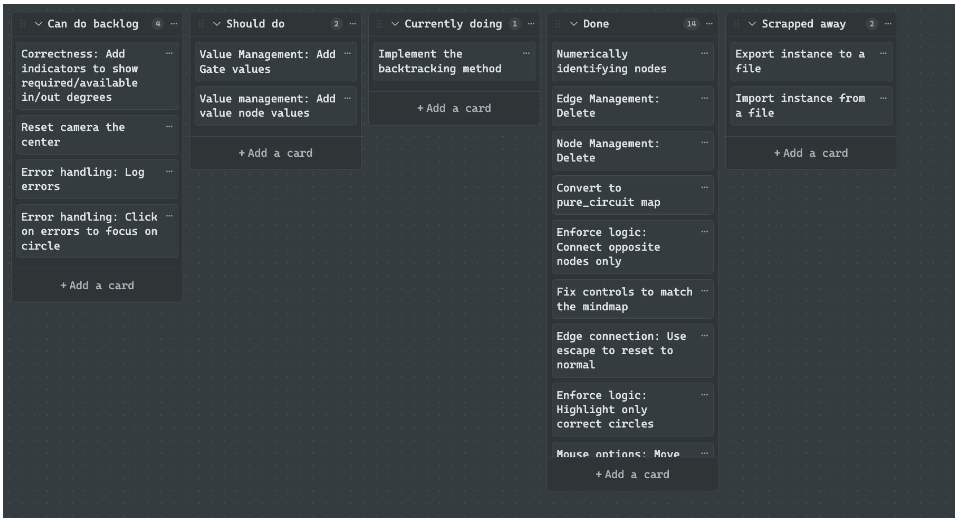


Figure 5.1.1: Kanban board

With regard to the research management, we revolved our methodology around the *Zettelkasten method*. *Niklas Luhmann* developed this method to keep track of notes and information when dealing with a massive wave of publications [9]. The method embodies the following elements:

1. **Literature Notes**: Atomic notes that have been extracted from literature which contain its ideas and techniques.

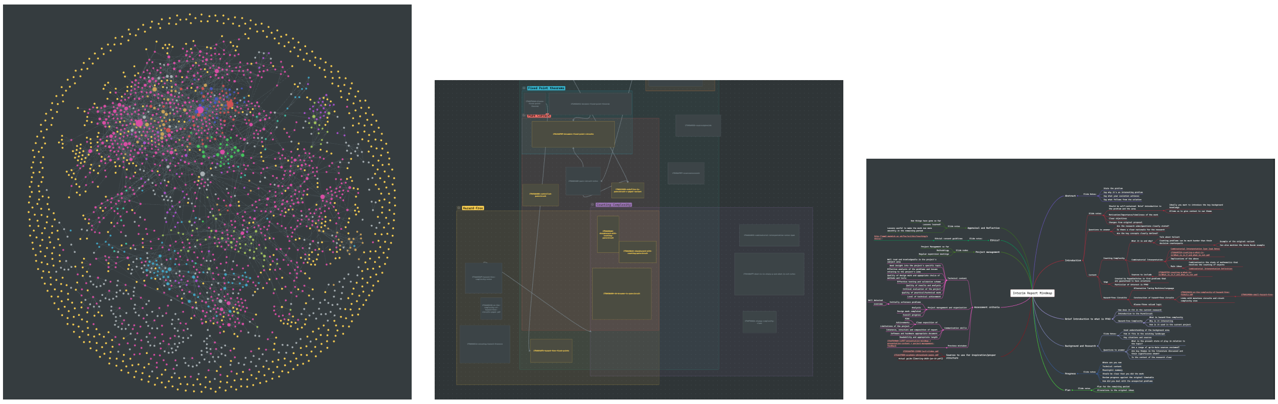
2. **Fleeting notes**: Ideas that pop up randomly but are not linked with any particular source.

3. **Permanent notes**: Reviewed literature notes and fleeting notes which have been promoted to **permanent** ones. These notes maybe linked with other permanent notes to create a web of knowledge.

Each of these notes can be annotated with a tagging systems and relate to each other with the use of references. Using the above method, we were able to extract ideas from different papers and create robust theorems that are based off the current literature.

5.2 Tooling

To aid our research management, we made use of the `Obsidian` [1] tool. `Obsidian`'s graphical representation of the semantical relations between nodes as well as its smart tagging system, made it easier to keep track of our ideas and ongoing theories, as it is based around the philosophy of the *Zettelkasten method*. Moreover, it incorporates additional features as seen in the figure 5.2.1, such as a canvas or a mind-map. All these features allowed us to streamline and refine our theories throughout the project. In addition, to keep version control, we used `Github` [2] as it one of the most well known version control applications in industry. This allowed us to keep constant backups of our code and notes, whilst being able to refer back in case something unexpected occurred.



(a) Obsidian graph. Each dot represents a node and each link corresponds to a reference between two nodes

(b) Obsidian Canvas

(c) Obsidian mind-map

Figure 5.2.1: Usages of Obsidian. Figure from interim report.

5.3 Ethical Considerations

Due to the nature to the theoretical nature of the project, there were no ethical consideration that were required. All the data and diagrams that were referenced, were either made by the author, redrawn by the author or referenced from another paper. The libraries used to build the software were under the MIT licence and no third-party datasets were collected.

5.4 Risk Assessment

We provide a table of our current risk assessment in the table 5.4.1. Overall through our extensive testing, we were able to develop our algorithms fast enough without the worry of logic errors. With respect to the theory crafting we faced several issues, as it was expected due to the nature of the project. Due to the lack of progress, we focused instead on developing theorems for each of our objectives with the hopes that they will aid in their future resolution. Moreover, we had trouble dealing with incorrect proofs as expanded in greater detail in our reflections section 5.6,

[1] https://obsidian.md/
[2] https://github.com/

which shifted our attention on expanding on the theorems and proofs of the already existing literature in order to reduce the margin of error. We give our previous risk assessment table in the appendix 5.6.

Severity	Prob	Description	Mitigation	Resolved
High	Medium	Software may not be feasible within the remaining time frame.	We focus on the validation and creation of instances. The current project is primarily research-based, and therefore we are mainly prioritising the theoretical development aspect.	✓
Low	Medium	Software not identifying a correct solution.	Usage of TDD techniques and property testing to ensure correctness. Comparison with handmade instances.	✓
Medium	High	Difficulty of showing the parsimonious reduction from the *EndOfLine* to *PureCircuit*.	Find parsimonious reductions from both directions. Reduce the overall reduction goal to a simpler one such as *AntipodalSS* with *HF-AntipodalSS*.	✓
High	High	Inability to make significant progress on the *SourceOrExcess* problems or the generalised statement.	Analyse the counting complexity of **PPAD**. Find parsimonious reductions from the *SourceOrExcess* problem.	✓
High	High	Incorrect proofs or reductions.	Analyse the problem under different constraints. Apply the duck method, where we attempt to explain our ideas to an independent third-party. Validate proof with the supervisor or use the tool to investigate sample cases.	-
High	Medium	Develop combinatorial-friendly variants of *PureCircuit*.	Apply robustness on the gate set of PureCircuit. Develop new gates or variants are easier to work with or focus on a subset of solutions as explained in 1.2.1.	✓

Table 5.4.1: Risk management table.

5.5 Timetable

We provide a figure of our final iteration of our Gantt chart in figure 5.5.1 and the Gantt chart from our interim report in the appendix .0.5. Overall there were substantial changes as we pivoted our attention into the completion of the *PureCircuit* tool and the development of the necessary algorithms.

Due to complications that arose shortly after the interim report, as mentioned in our risk assessment section, we shifted focus to the topological interpretations of superpolynomial graphs with polynomial degrees. This is also one of the reasons why the software development was finished this late in project. We expand on this matter in greater detail in our 5.6. Overall, the project was able to finish within the expected timeframe with the full software delivered as anticipated.

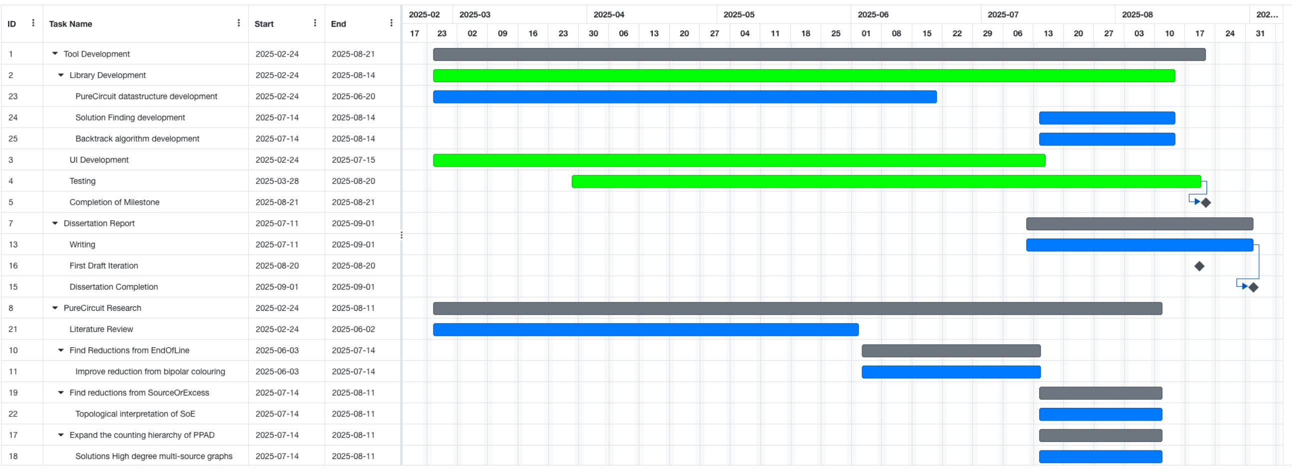


Figure 5.5.1: Final Gantt chart

5.6 Appraisals and Reflections

Like every project, there were things that did and did not go our way. First, a minimum viable version of the software was completed towards a much later stage project. In an ideal scenario, it would have been more valuable for us had we prioritised the development of the visualisation tool as it would have allowed for faster verification of results. Moreover, one of our key results from the interim report ended up having a major issues that halted our progress.

On the bright side, our rigorous testing methodology enabled us to complete the software development without encountering any major issues. Additionally, our systematic note-taking process proved invaluable when handling unforeseen incidents, allowing us to pivot from our research objectives effectively and explore alternative approaches such as topological interpretations and multi-source graph problems. This comprehensive documentation system also enabled us to learn from previous attempts and refine our current results with greater certainty. Many of our initially unsuccessful ideas ultimately contributed to breakthrough results, particularly in the main proofs presented in sections 2.1 and 2.2.

Overall this project was very fun to work with. The opportunity to work in the intersection of many well-studied fields allowed me to explore each one individually as well as experiment with a variety of approaches. Adding on, it offered me valuable experience and skills with more research based projects as well as a deeper understanding of complexity theory. This project truly shaped my view of computer science and my ambitions as a future researcher. In the future, I am happy to make use of all my knowledge and build on the mistakes I made, in order to truly make an impact in computer science.

Bibliography

[1] M. Abdel-Basset, L. Abdel-Fatah, and A. K. Sangaiah. Chapter 10 - Metaheuristic Algorithms: A Comprehensive Review. In A. K. Sangaiah, M. Sheng, and Z. Zhang, editors, *Computational Intelligence for Multimedia Big Data on the Cloud with Engineering Applications*, Intelligent Data-Centric Systems, pages 185–231. Academic Press, Jan. 2018.

[2] J. Aisenberg, M. L. Bonet, and S. Buss. 2-D Tucker is PPA complete. *Journal of Computer and System Sciences*, 108:92–103, Mar. 2020.

[3] S. Arora and B. Barak. *Computational Complexity: A Modern Approach*. Cambridge University Press, Cambridge, 2009.

[4] P. Beame, S. Cook, J. Edmonds, R. Impagliazzo, and T. Pitassi. The Relative Complexity of NP Search Problems. *Journal of Computer and System Sciences*, 57(1):3–19, Aug. 1998.

[5] M. Black. *Critical Thinking: An Introduction to Logic and Scientific Method*. Prentice-Hall Philosophy Series. Prentice-Hall, Incorporated, 1946.

[6] G. Brewka. Artificial intelligence—a modern approach by stuart russell and peter norvig, prentice hall. Series in artificial intelligence, englewood cliffs, NJ. *The Knowledge Engineering Review*, 11(1):122–123, 1996.

[7] K. Bronisław. Un theoreme sur les functions d'ensembles. 6:133–134, 1928.

[8] J. Bund, C. Lenzen, and M. Medina. Small Hazard-Free Transducers. *IEEE Transactions on Computers*, 74(5):1549–1564, May 2025.

[9] A. Cevolini. *Forgetting Machines: Knowledge Management Evolution in Early Modern Europe*. Dec. 2018.

[10] A. K. Chandra, D. C. Kozen, and L. J. Stockmeyer. Alternation. *Journal of the ACM (JACM)*, 28(1):114–133, 1981.

[11] X. Chen and X. Deng. On the complexity of 2D discrete fixed point problem. *Theoretical Computer Science*, 410(44):4448–4456, Oct. 2009.

[12] X. Chen, X. Deng, and S.-H. Teng. Settling the complexity of computing two-player Nash equilibria. *J. ACM*, 56(3):14:1–14:57, May 2009.

[13] K. Claessen and J. Hughes. QuickCheck: A lightweight tool for random testing of Haskell programs. *SIGPLAN Not.*, 35(9):268–279, Sept. 2000.

[14] S. A. Cook. The complexity of theorem-proving procedures. In *Proceedings of the Third Annual ACM Symposium on Theory of Computing*, STOC '71, pages 151–158, New York, NY, USA, May 1971. Association for Computing Machinery.

[15] C. Daskalakis, P. Goldberg, and C. Papadimitriou. *The Complexity of Computing a Nash Equilibrium*, volume 39. Association for Computing Machinery, Jan. 2006.

[16] C. Daskalakis and C. Papadimitriou. Continuous local search. In *Proceedings of the Twenty-Second Annual ACM-SIAM Symposium on Discrete Algorithms*, SODA '11, pages 790–804, USA, Jan. 2011. Society for Industrial and Applied Mathematics.

[17] C. Daskalakis, S. Skoulakis, and M. Zampetakis. The Complexity of Constrained Min-Max Optimization, Sept. 2020.

[18] C. Daskalakis, S. Skoulakis, and M. Zampetakis. The complexity of constrained min-max optimization. In *Proceedings of the 53rd Annual ACM SIGACT Symposium on Theory of Computing*, STOC 2021, pages 1466–1478, New York, NY, USA, June 2021. Association for Computing Machinery.

[19] M. Davis and H. Putnam. A Computing Procedure for Quantification Theory. *J. ACM*, 7(3):201–215, July 1960.

[20] A. Deligkas, J. Fearnley, A. Hollender, and T. Melissourgos. Constant inapproximability for PPA. In *Proceedings of the 54th Annual ACM SIGACT Symposium on Theory of Computing*, STOC 2022, pages 1010–1023, New York, NY, USA, June 2022. Association for Computing Machinery.

[21] A. Deligkas, J. Fearnley, A. Hollender, and T. Melissourgos. Pure-Circuit: Tight Inapproximability for PPAD. *J. ACM*, 71(5):31:1–31:48, Oct. 2024.

[22] E. B. Eichelberger. Hazard Detection in Combinational and Sequential Switching Circuits. *IBM Journal of Research and Development*, 9(2):90–99, Mar. 1965.

[23] J. Fearnley, P. W. Goldberg, A. Hollender, and R. Savani. The complexity of gradient descent: CLS = PPAD and PLS. In *Proceedings of the 53rd Annual ACM SIGACT Symposium on Theory of Computing*, STOC 2021, pages 46–59, New York, NY, USA, June 2021. Association for Computing Machinery.

[24] J. Fearnley, S. Gordon, R. Mehta, and R. Savani. CLS: New Problems and Completeness, Apr. 2017.

[25] J. Fearnley, D. Pálvölgyi, and R. Savani. A Faster Algorithm for Finding Tarski Fixed Points. *ACM Trans. Algorithms*, 18(3):23:1–23:23, Oct. 2022.

[26] S. Friedrichs, M. Függer, and C. Lenzen. Metastability-Containing Circuits. *IEEE Transactions on Computers*, 67(8):1167–1183, Aug. 2018.

[27] H. N. Gabow. Path-based depth-first search for strong and biconnected components. *Information Processing Letters*, 74(3):107–114, May 2000.

[28] J. H. Holland. *Adaptation in Natural and Artificial Systems: An Introductory Analysis with Applications to Biology, Control, and Artificial Intelligence*. The MIT Press, Apr. 1992.

[29] A. Hollender and P. Goldberg. The Complexity of Multi-source Variants of the End-of-Line Problem, and the Concise Mutilated Chessboard, June 2018.

[30] C. Ikenmeyer, B. Komarath, C. Lenzen, V. Lysikov, A. Mokhov, and K. Sreenivasaiah. On the complexity of hazard-free circuits. *Journal of the ACM*, 66(4):1–20, Aug. 2019.

[31] C. Ikenmeyer, B. Komarath, and N. Saurabh. Karchmer-Wigderson Games for Hazard-free Computation, Nov. 2022.

[32] C. Ikenmeyer, K. D. Mulmuley, and M. Walter. On vanishing of Kronecker coefficients. *computational complexity*, 26(4):949–992, Dec. 2017.

[33] C. Ikenmeyer and I. Pak. What is in #P and what is not? In *2022 IEEE 63rd Annual Symposium on Foundations of Computer Science (FOCS)*, pages 860–871, Oct. 2022.

[34] C. Ikenmeyer, I. Pak, and G. Panova. Positivity of the Symmetric Group Characters Is as Hard as the Polynomial Time Hierarchy. *International Mathematics Research Notices*, 2024(10):8442–8458, May 2024.

[35] D. S. Johnson, C. H. Papadimitriou, and M. Yannakakis. How easy is local search? *Journal of Computer and System Sciences*, 37(1):79–100, Aug. 1988.

[36] S. Katoch, S. S. Chauhan, and V. Kumar. A review on genetic algorithm: Past, present, and future. *Multimedia Tools and Applications*, 80(5):8091–8126, Feb. 2021.

[37] S. Kleene and M. Beeson. *Introduction to Metamathematics*. Ishi Press International, 2009.

[38] D. Kozen. *Theory of Computation*. Texts in Computer Science. Springer London, 2006.

[39] R. H. Makar. On the Analysis of the Kronecker Product of Irreducible Representations of the Symmetric Group. *Proceedings of the Edinburgh Mathematical Society*, 8(3):133–137, Dec. 1949.

[40] L. R. Marino. General theory of metastable operation. *IEEE Transactions on Computers*, C-30(2):107–115, Feb. 1981.

[41] C. Moon, J. Kim, G. Choi, and Y. Seo. An efficient genetic algorithm for the traveling salesman problem with precedence constraints. *European Journal of Operational Research*, 140(3):606–617, Aug. 2002.

[42] M. Mukaidono. On the B-ternary logic function. *Trans. IECE*, 55:355–362, Jan. 1972.

[43] I. Pak. What is a combinatorial interpretation?, Sept. 2022.

[44] C. Papadimitriou. On graph-theoretic lemmata and complexity classes. In *Proceedings [1990] 31st Annual Symposium on Foundations of Computer Science*, pages 794–801 vol.2, Oct. 1990.

[45] C. H. Papadimitriou. On the complexity of the parity argument and other inefficient proofs of existence. *Journal of Computer and System Sciences*, 48(3):498–532, June 1994.

[46] A. Rubinstein. Inapproximability of Nash Equilibrium. In *Proceedings of the Forty-Seventh Annual ACM Symposium on Theory of Computing*, STOC '15, pages 409–418, New York, NY, USA, June 2015. Association for Computing Machinery.

[47] H.-P. Schwefel. *Evolution and Optimum Seeking*. Number Book, Whole. Wiley, New York;Chichester;, 1995.

[48] M. Sharir. A strong-connectivity algorithm and its applications in data flow analysis. *Computers & Mathematics with Applications*, 7(1):67–72, Jan. 1981.

[49] K. O. Stanley. A Hypercube-Based Indirect Encoding for Evolving Large-Scale Neural Networks. 2009.

[50] K. O. Stanley and R. Miikkulainen. Evolving Neural Networks through Augmenting Topologies. *Evolutionary Computation*, 10(2):99–127, June 2002.

[51] L. G. Valiant. The complexity of computing the permanent. *Theoretical Computer Science*, 8(2):189–201, Jan. 1979.

[52] P. Wychowaniec. Learning to fly: Let's simulate evolution in rust (pt 3), 2021.

[53] Z. Yang. Sperner Theory. In Z. Yang, editor, *Computing Equilibria and Fixed Points: The Solution of Nonlinear Inequalities*, pages 311–322. Springer US, Boston, MA, 1999.

Appendices

AntipodalStrongSperner Colours

Proof of the claim 2.1.2

Proof. We are going to assume proof by contradiction. Assume there exists colour j that is duplicated on two points $p_1 = p + b_1, p_2 = p + b_2$ such that p is the anchor of a panchromatic solution and $b_1, b_2 \in \mathbb{B}^n$. Since the cube is antipodal, that means that we have to be wary of $\bar{p}_2$ which is at the position $p + \overline{b_2}$. Since the cube in discrete, it implies that $\bar{p}_2$ and p_1 cannot belong in any $(n-1)$-subspace, otherwise our cube won't be discrete. But that would imply that $\forall i \in [n] : |[\bar{p}_2 - p_1]_i| = 1$. We know the only position in the hypercube that satisfies that is $\bar{p}_1$ which implies $p_1 = p_2$. Therefore, every corner of an antipodal discrete cube must contain a different colour. □

PolyS-Unbalanced with Unbalanced

Proof of the lemma 2.3.1

Proof. To show our reduction, assume we have an instance of *PolyS-Unbalanced*, where we have $f(n)$ known sources. Assume that $h(n)$ describes the input degree and $g(n)$ the output degree. We create a new *Unbalanced* instance with in- and out-degree polynomials $p(n) = (\max\{g(n), h(n)\}) \cdot f(n)$. We will use the figure .0.1 to illustrate our reduction. Given s_i is the ith known source of the original problem, we can denote its successor set $\mathbf{S}_i = \{p_j^i\}_{j \in [g(n)]}$. In our new instance, for each group $\mathbf{S}_i$, we create a copy $\mathbf{P}_i$. For each s_i source of our original problem, we connect edges $v_k^i \to s_i$ where $v_k^i \in \mathbf{P}_i$, such that s_i is a balanced node. We also add edges such that $\forall i \in [f(n)], v \in P_i : (0^n, v) \in E$. Since this transformation is local, any other unbalanced node will stay the same and therefore the set of solutions is preserved.

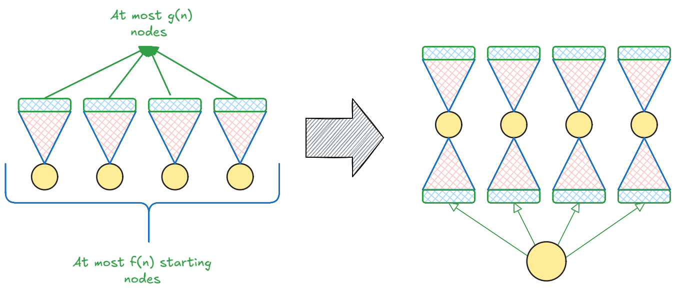


Figure .0.1: *PolyS-Unbalanced* transformation to *Unbalanced*

□

SourceOrExcess with number of known sources

We provide a proof of the claim 2.3.1.

Proof. Given *αS-SourceOrExcess*$(\beta, 1)$, in order to find the minimum number of solutions, we can assume that we there are no other sources other than the first α nodes. We argue that the minimum number of solutions is $\mu = \lceil \alpha/\beta \rceil$ since we can create μ sinks that connect to at most β sources each. We will refer to these graphs as sink graphs.

Furthermore, we can demonstrate that any graph can be reduced to a sink graph. Our first observation is to convert every excess node to a sink node. To do that we can apply two simplification phases:

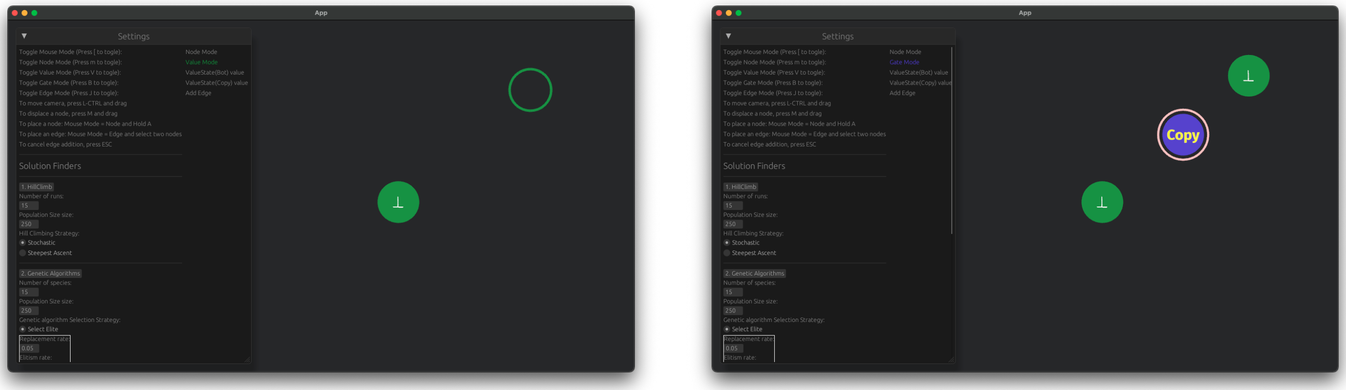
1. Removal phase: Remove all paths between two excess nodes or between excess and sink nodes. We can observe that after this steps our graph only contains sources and sinks.
2. Merging phase: If there are sinks with degrees $(a, 0)$ and $(b, 0)$ such that $a + b \leqslant \beta$ then we can merge the two to remove a solution. Keep repeating this process until no two sinks can be merged

We can observe that any graph will simplify to a sink graph, and therefore we will always have at least μ solutions, since each step will never increase the solution set. □

Visualisation Program Demonstration

User stories

Node addition We demonstrate how we can add nodes. Essentially, ensure the user is in mouse mode, hold A and press left click to place the node, as seen in the figures .0.2. We observe in figure .0.2b that the Copy gate is circled in pink to denote invalid arity, meaning its missing its input and output edge. In addition, hovering over a node will highlight the node in yellow as seen in figure .0.6.



(a) Holding A gives an indicator of where the node will be placed (b) Final graph after addition of values and gate nodes

Figure .0.2: Node addition

Edge addition To add an edge between two nodes, we select the source of the edge. We can observe from the figure .0.3, that we highlight the source node with light blue and the heterogeneous nodes with orange. If we select a gate, we highlight only nodes have an in-degree of 0. Pressing ESC or changing mode can cancel the selection operation. Selecting on any of the orange nodes completes the edge addition operation as seen in figure .0.4. We can also observe that since the Copy has no error circles since it has the correct arity and the input/output values satisfy the gate as seen in .0.5.

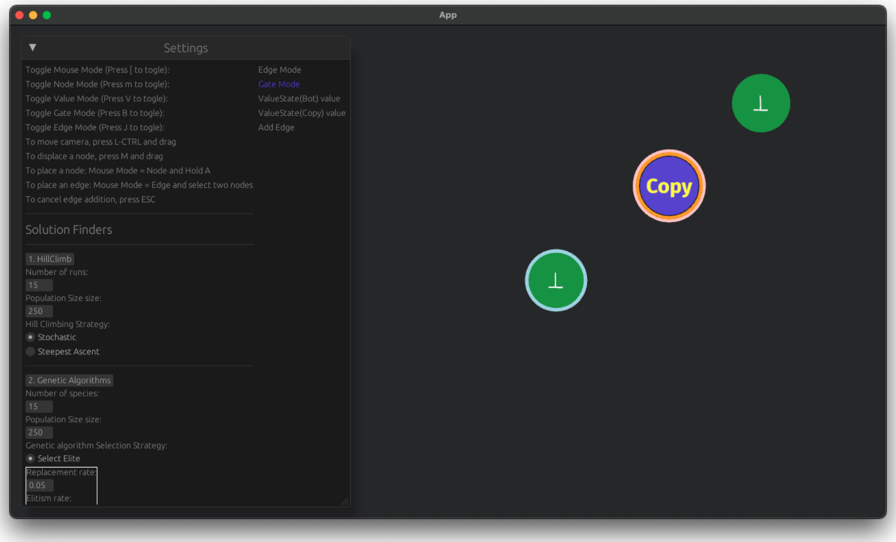


Figure .0.3: Edge addition: Select source node

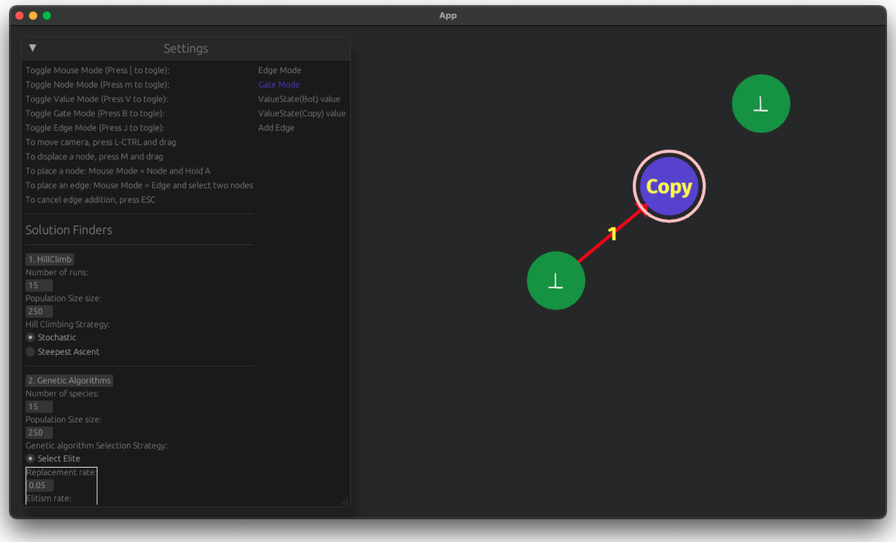


Figure .0.4: Edge addition: Select destination node

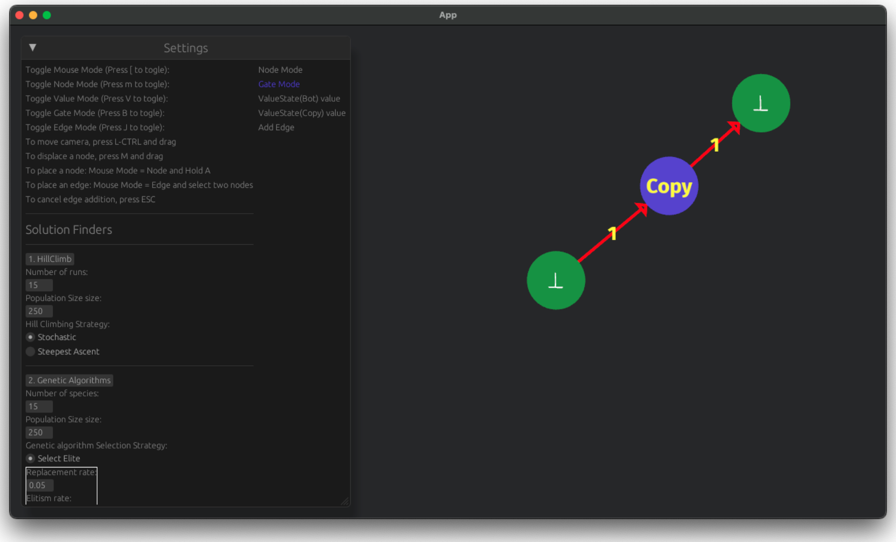


Figure .0.5: Full graph with edges

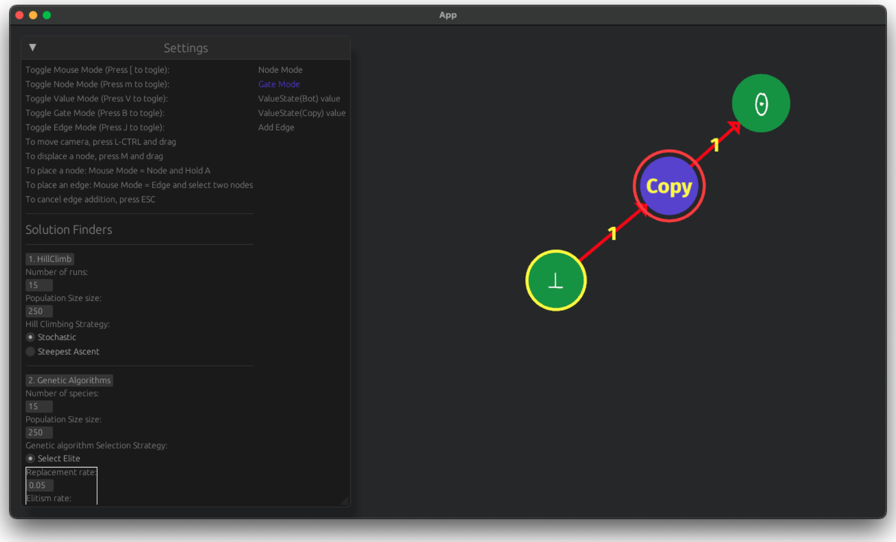
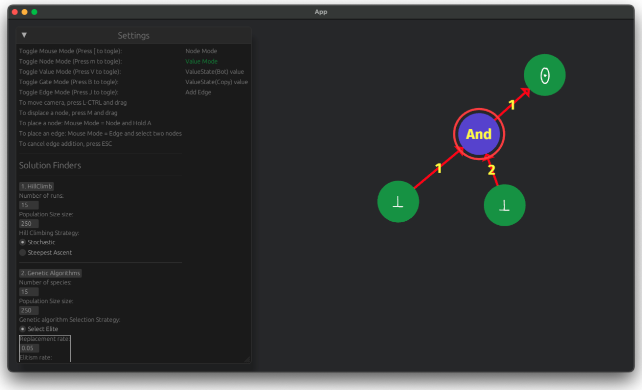
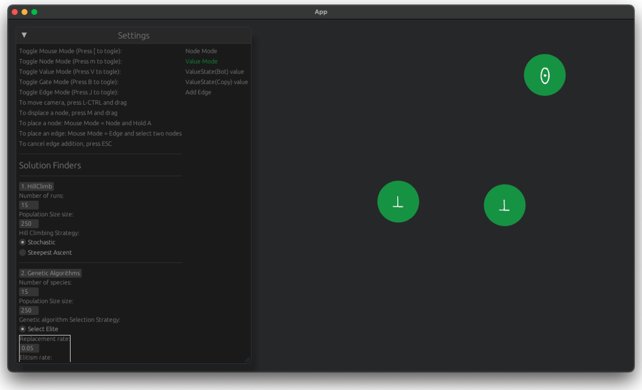


Figure .0.6: Value node toggle and node hover

Node/Edge deletion To delete nodes we can simplify hover over a node and press D. For edge deletion the procedure is the same as the edge addition, but we can only select nodes that are incident to our current node. We provide examples in .0.7 and .0.8

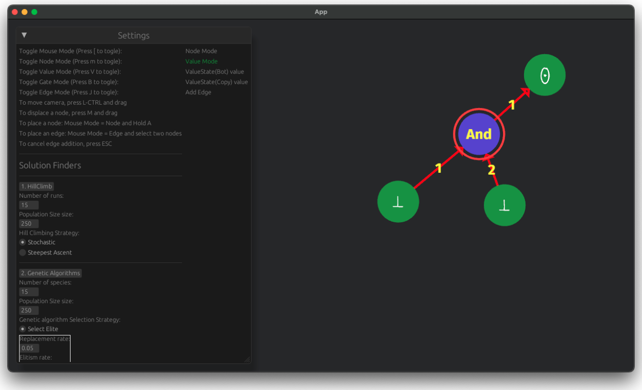


(a) Node removal: Before removal

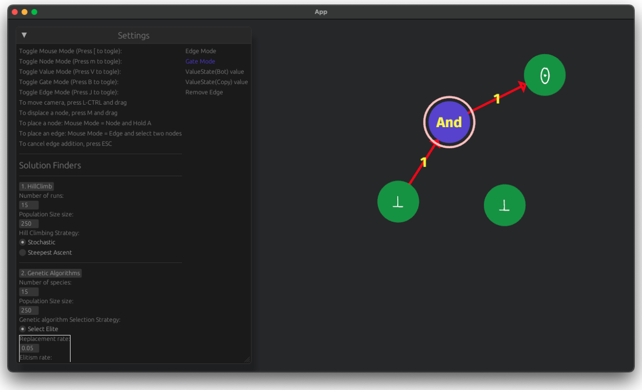


(b) Node removal: After removal

Figure .0.7: Node removal



(a) Edge removal: Before removal



(b) Edge removal: After removal

Figure .0.8: Edge removal

Gate rules

NOR input	Output sets
$\forall v \in \mathbb{T} : (\cdot, \cdot, v)$	$(\mathbb{T}_{\leqslant \neg v}, \mathbb{T}_{\leqslant \neg v}, \{v\})$
$\forall v \in \mathbb{T} : (\cdot, v, \cdot)$	$(\mathbb{T}, \{v\}, \mathbb{T}_{\leqslant \neg v})$
$\forall a, b \in \mathbb{T} : a > b \implies (\cdot, a, b)$	$(\{\neg b\}, \{a\}, \{b\})$
$\forall v \in \mathbb{T} : (\cdot, v, \neg v)$	$(\mathbb{T}_{\leqslant v}, \{v\}, \{\neg v\})$
$\forall a, b \in \mathbb{T} : a < b \implies (\cdot, a, b)$	Invalid combination
$\forall a, b \in \mathbb{T} : (a, b, \cdot)$	$(\{a\}, \{b\}, \{\neg \max(a, b)\})$
$\forall a, b, c \in \mathbb{T} : (a, b, c)$	$(\{a\}, \{b\}, \{c\})$, only if the triplet (a, b, c) is a correct **NOR** assignment

Table .0.1: **NOR** value propagation rules

NAND input	Output sets
$\forall v \in \mathbb{T} : (\cdot, \cdot, v)$	$(\mathbb{T}_{\geqslant \neg v}, \mathbb{T}_{\geqslant \neg v}, \{v\})$
$\forall v \in \mathbb{T} : (\cdot, v, \cdot)$	$(\mathbb{T}, \{v\}, \mathbb{T}_{\geqslant \neg v})$
$\forall a, b \in \mathbb{T} : a < b \implies (\cdot, a, b)$	$(\{\neg b\}, \{a\}, \{b\})$
$\forall v \in \mathbb{T} : (\cdot, v, \neg v)$	$(\mathbb{T}_{\geqslant v}, \{v\}, \{\neg v\})$
$\forall a, b \in \mathbb{T} : a > b \implies (\cdot, a, b)$	Invalid combination
$\forall a, b \in \mathbb{T} : (a, b, \cdot)$	$(\{a\}, \{b\}, \{\neg \min(a, b)\})$
$\forall a, b, c \in \mathbb{T} : (a, b, c)$	$(\{a\}, \{b\}, \{c\})$, only if the triplet (a, b, c) is a correct **NAND** assignment

Table .0.2: **NAND** value propagation rules

COPY input	Output sets
$\forall v \in \mathbb{T} : (\cdot, v) \vee (\cdot, v)$	$(\{v\}, \{v\})$
$\forall a, b \in \mathbb{T} : (a, b)$	$(\{a\}, \{b\})$, only if the tuple (a, b) is a correct **COPY** assignment

Table .0.3: **COPY** value propagation rules

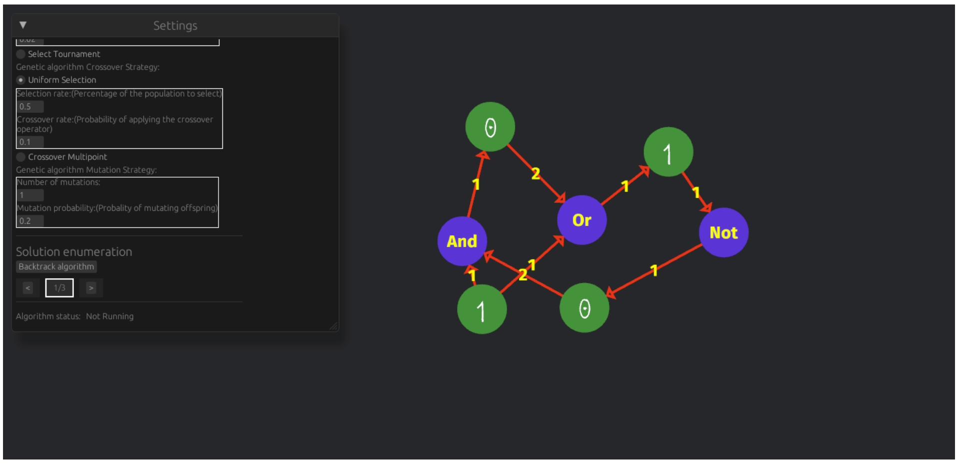
NOT input	Output sets
$\forall v \in \mathbb{T} : (\cdot, v)$	$(\{\neg v\}, \{v\})$
$\forall v \in \mathbb{T} : (v, \cdot)$	$(\{v\}, \{\neg v\})$
$\forall a, b \in \mathbb{T} : (a, b)$	$(\{a\}, \{b\})$, only if the tuple (a, b) is a correct **NOT** assignment

Table .0.4: **NOT** value propagation rules

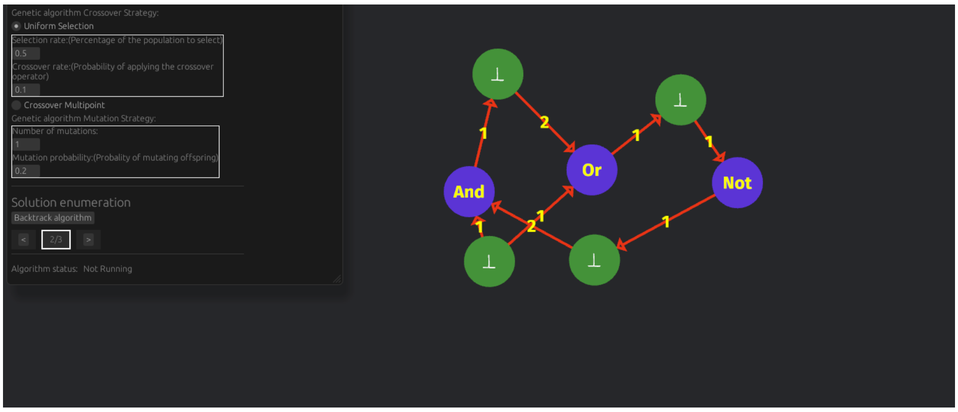
Software enumerators

Small circuit Example

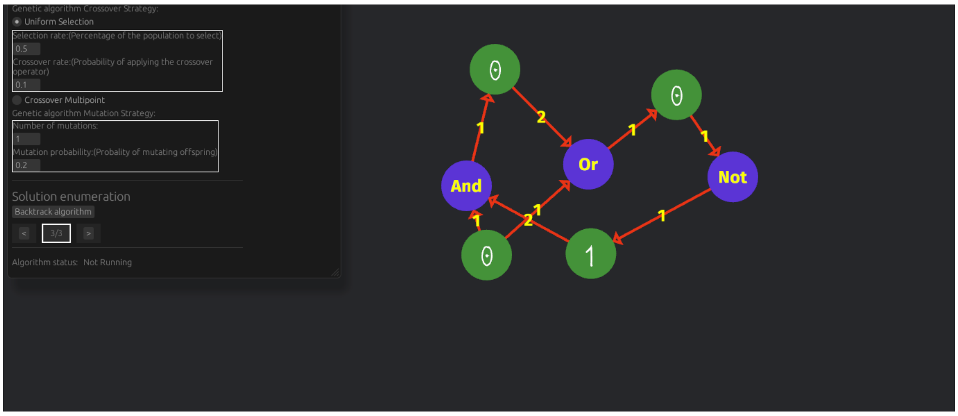
We provide the solution set for the small circuit from section 3.2.2.



Solution 1



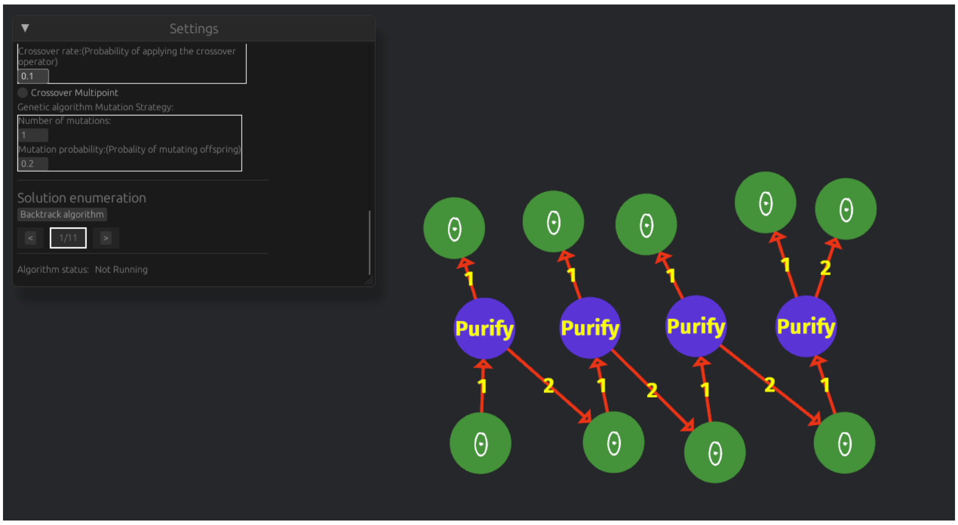
Solution 2



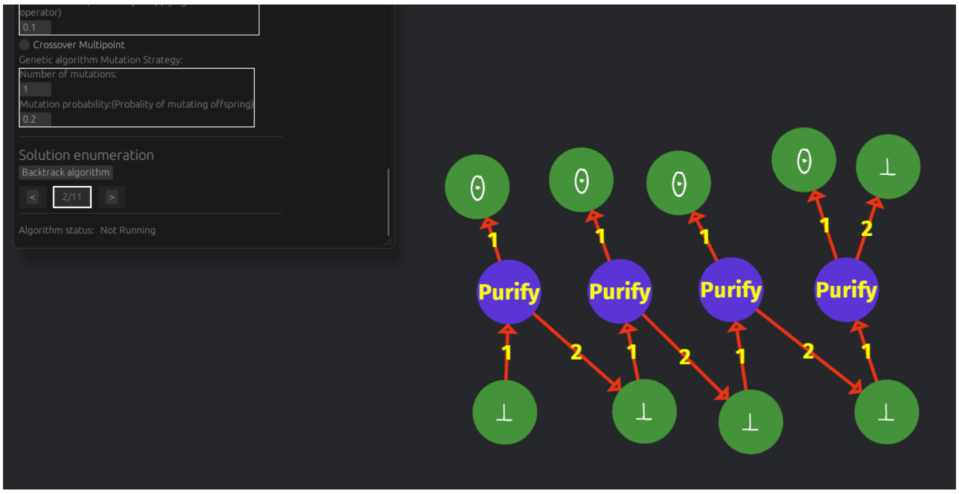
Solution 3

Bit Generator Example

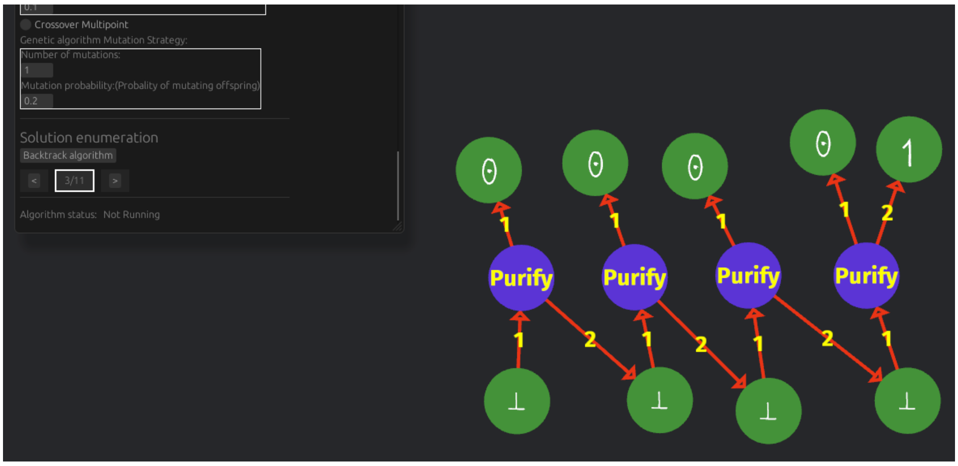
We provide the solution set for the bit generator circuit from section 3.2.2.



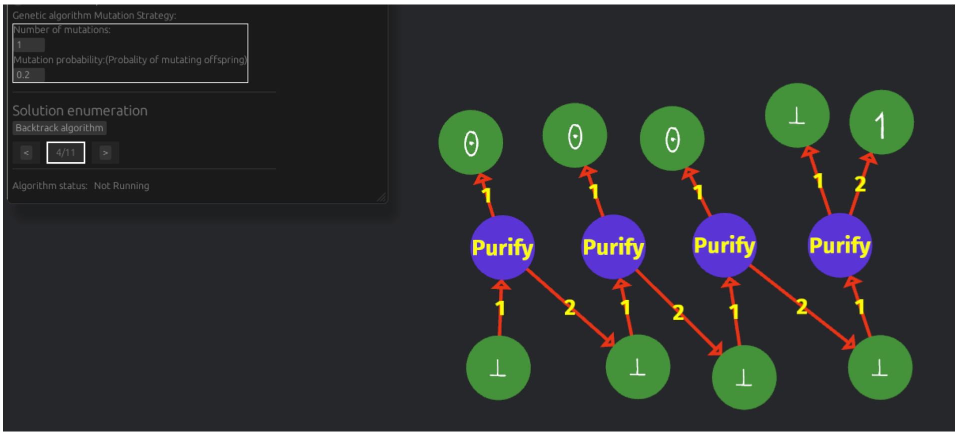
Solution 1



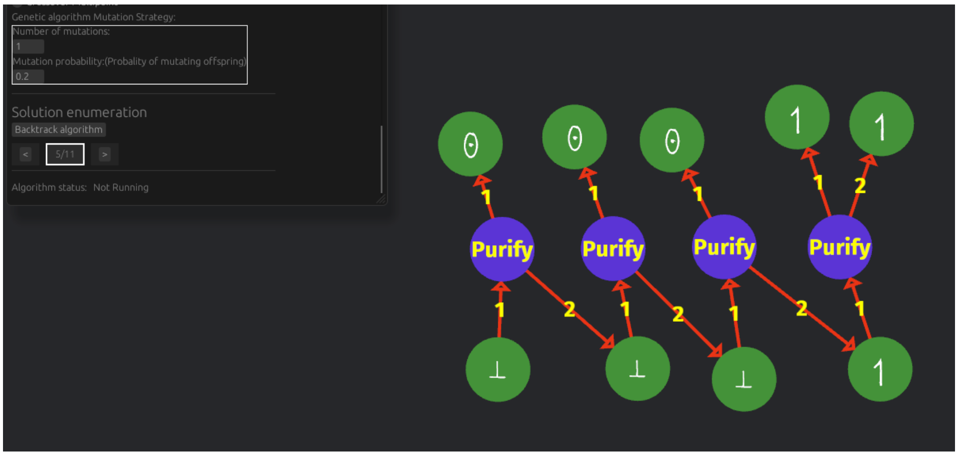
Solution 2



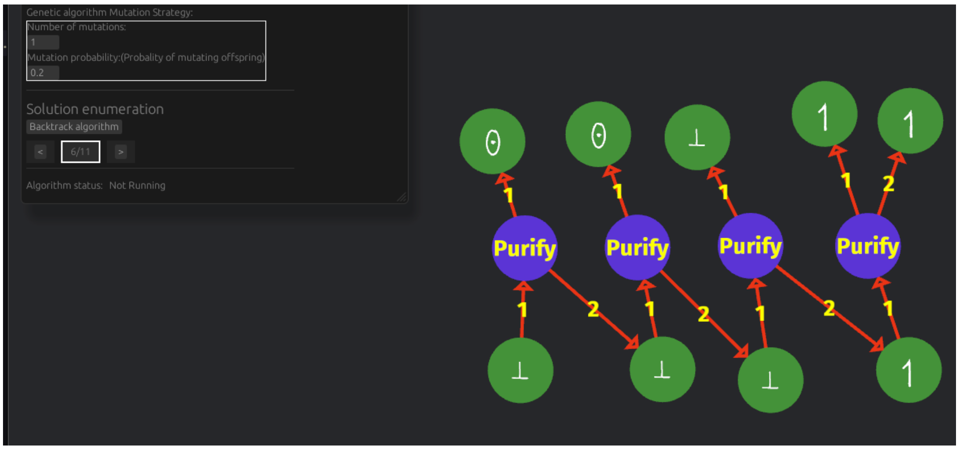
Solution 3



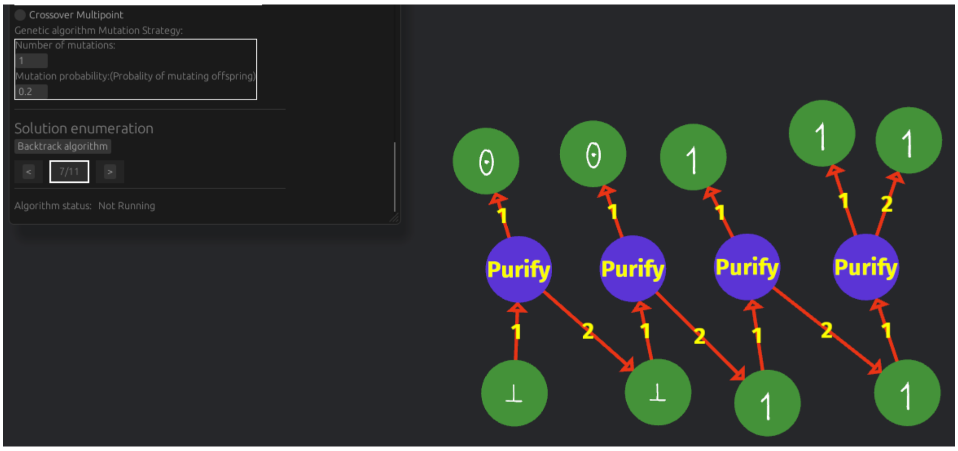
Solution 4



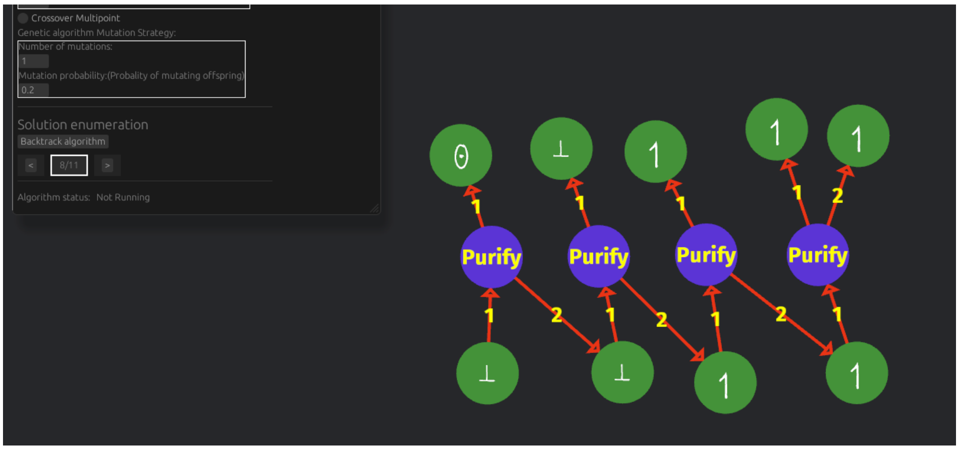
Solution 5



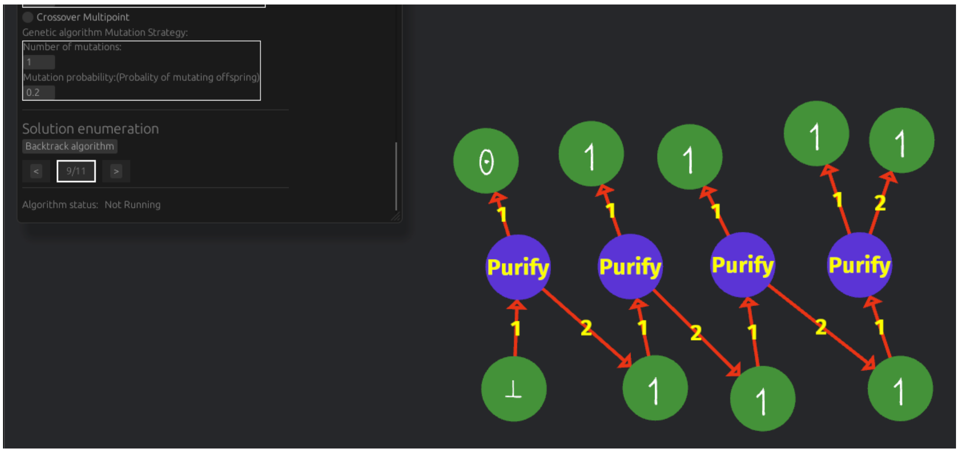
Solution 6



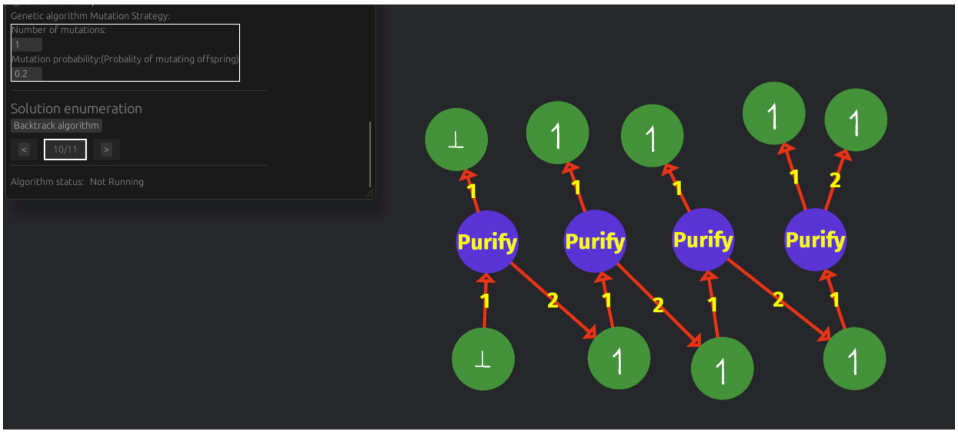
Solution 7



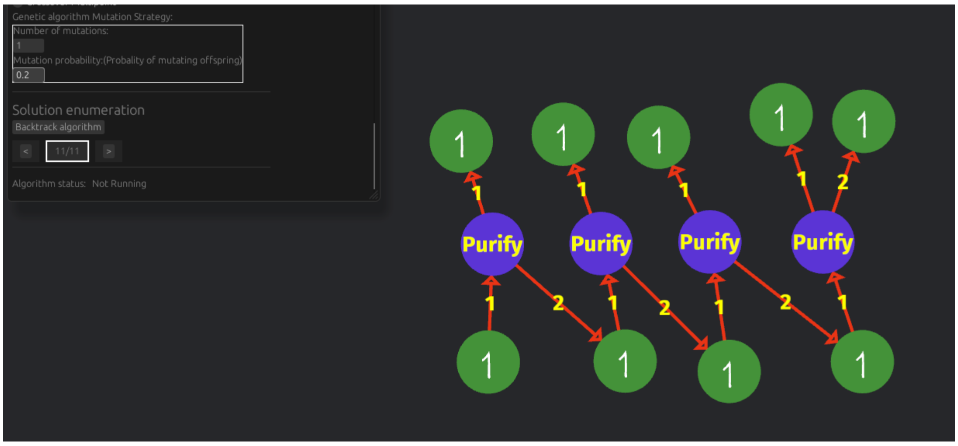
Solution 8



Solution 9



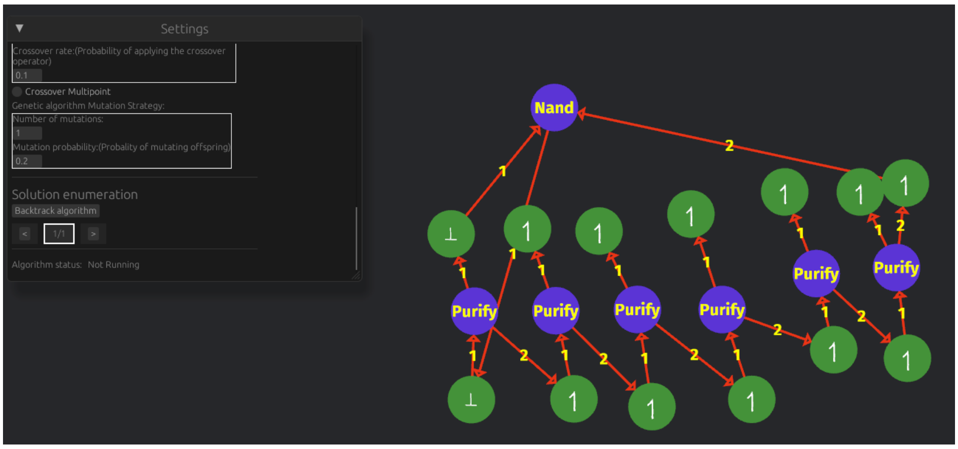
Solution 10



Solution 11

Complex circuit Example

We provide the solution set for the complex circuit from section 3.2.2.



Solution 1

Project Management

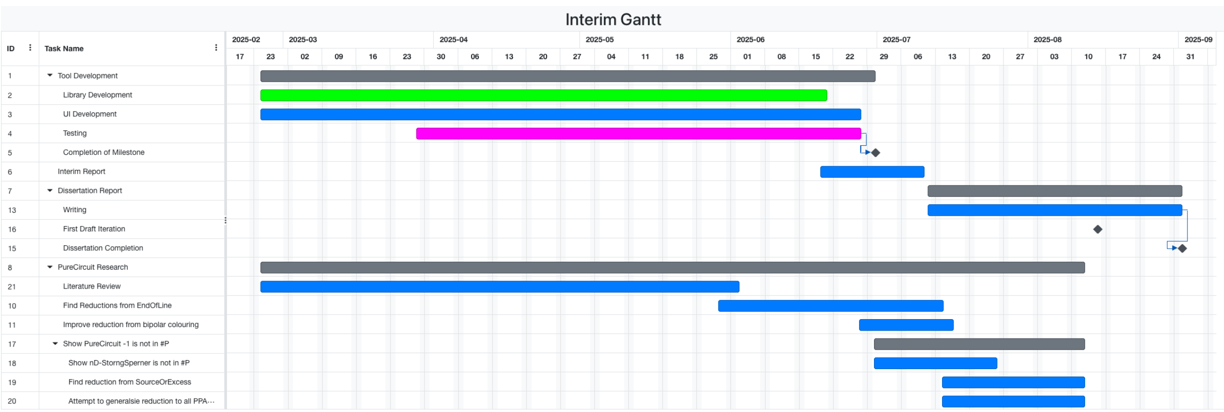
Interim Risk Assessment

Severity	Probability	Description	Mitigation	Address
High	Medium	Software may not be feasible within the remaining time frame.	We focus on the validation and creation of instances. The current project is primarily research-based, and therefore we are mainly prioritising the theoretical development aspect.	Not yet
Low	Medium	Software not identifying a correct solution.	Usage of TDD techniques and property testing to ensure correctness. Comparison with handmade instances.	Not yet
Medium	Low	Difficulty of showing the parsimonious reduction between PureCircuit and nD-StrongSperner.	Consult with the researchers that actively work on these problems. Lower the requirements to polynomially bounded reductions instead.	Not yet
High	High	Inability to make significant progress on the SourceOrExcess problems or the generalised statement.	Establish reductions from other problems in order to demonstrate #P hardness by starting from nD-StrongSperner.	Not yet

High	High	Incorrect proofs or reductions.	Analyse the problem under different constraints. Apply the duck method, where you attempt to explain the solution to a person who is not necessarily an expert. Validate proof with the supervisor or use the tool to investigate sample cases.	On-going
High	Medium	Develop combinatorial-friendly variants of PURECIRCUIT.	Apply robustness on the gate set of PURECIRCUIT. Develop new gates or variants are easier to work with or focus on a subset of solutions as explained in 1.2.1.	✓

Interim risk assessment table

Interim Report Timetable



Interim report gantt chart